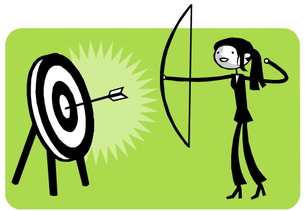
Chapter 5: Network Layer

Chapter goals:

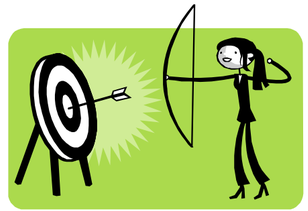
- understand principles behind network layer services:
  - Network layer service models
  - Forwarding versus routing
  - how a router works
  - routing (path selection)
  - dealing with scale
- instantiation and implementation in the Internet



第5章：网络层

章节目标：

- 理解网络层服务的原理：
  - 网络层服务模型 转发与路由 路由器的工作原理 路由（路径选择） 处理可扩展性
- 互联网中的实例化和实现



Chapter 5: Network Layer

- 5.1 Introduction
- 5.2 Virtual circuit and datagram networks
- 5.3 What's inside a router?
- 5.4 Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector
  - Hierarchical routing
- 5.5 Routing in the Internet
- 5.6 IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6

第 5 章：网络层

- 5.1 引言
- 5.2 虚拟电路与数据报网络
- 5.3 路由器里有什么?
- 5.4 路由算法：
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量 分层路由
- 5.5 互联网中的路由
- 5.6 IP：互联网协议
  - IPv4数据报格式 IPv4寻址 IP分片 网络地址转换 ARP ICMP IPv6

Keypoints and Difficulties

Keypoints:

- Link state routing algorithm
- Distance vector routing algorithm
- IPv4
- IPv6

Difficulties:

- Virtual circuit and datagram networks
- Subnet and Subnet Division
- IP fragment

关键点 和难点

关键点：

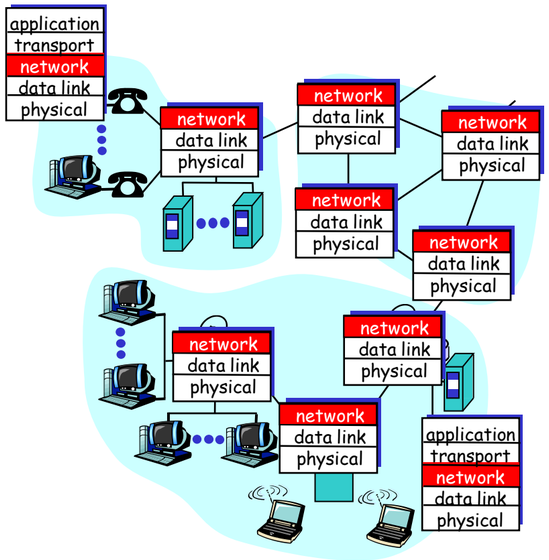
- 链路状态路由算法
- 距离矢量路由算法
- IPv4
- IPv6

难点：

- 虚拟电路与数据报网络
- 子网和子网划分
- IP分片

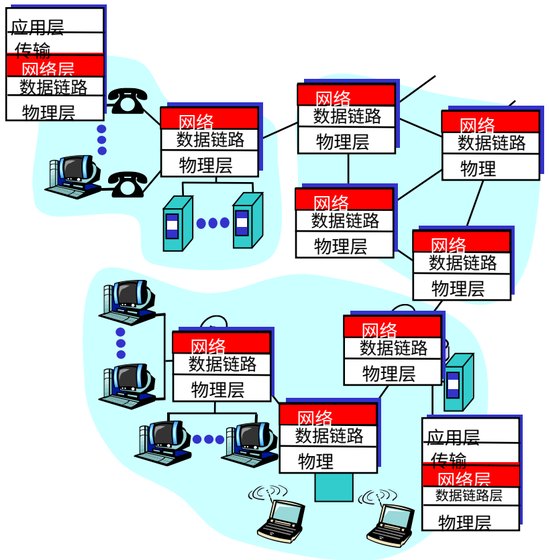
Network layer

- Transport packet from sending to receiving hosts
- Network layer protocols in *every* host, router
- Router examines header fields in all IP datagram passing through it



网络层

- 从发送主机到接收主机的传输数据包
- 网络层协议在<每个主机、路由器>中
- 路由器检查通过它的所有IP数据报的头部字段



Two key Network-layer functions



- Routing: determine route taken by packets from source to dest.
  - *Routing algorithms*
- Forwarding: move packets from router's input to appropriate router output

Analogy:

- *Routing:* process of planning trip from source to dest.
- *Forwarding:* process of getting through single interchange

Connection setup: some network architectures require router connection setup along path before data flows.



两个关键网络-层功能



- 路由：确定数据包从源节点到目的节点的路由路径
  - 路由算法
- 转发：将数据包从路由器的输入端移动到合适的路由器输出端

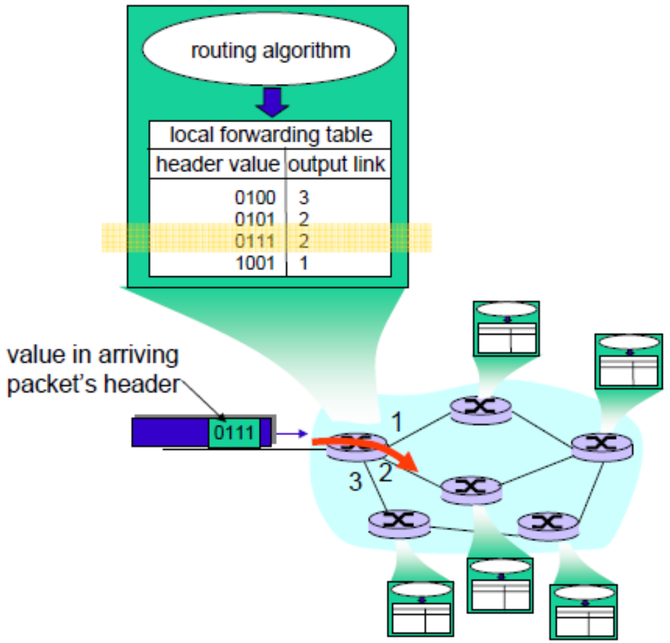
类比：

- 路由：从源节点到目的节点的规划过程。
- 转发：通过单个交换的过程。

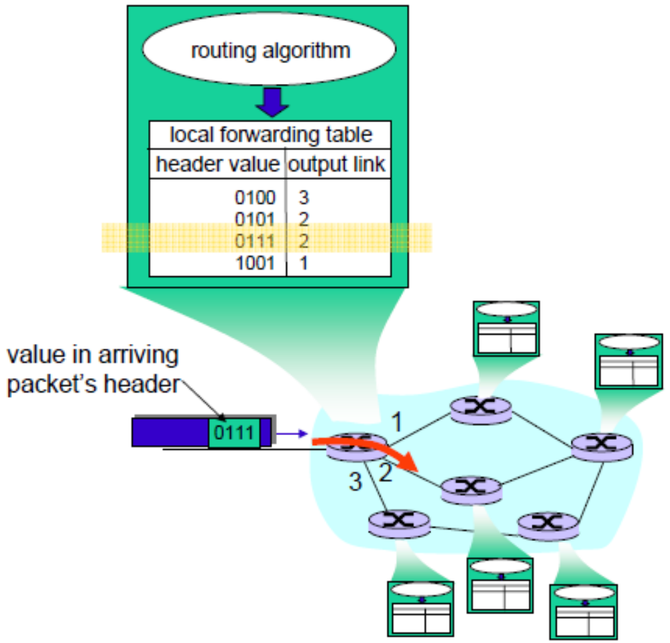
连接建立：某些网络架构需要在数据流之前沿路径建立路由器连接。



Interplay between routing and forwarding



路由与转发之间的交互



Network service model

Q: What *service model* for "channel" transporting packets from sender to receiver?

service abstraction

- guaranteed bandwidth?
- preservation of inter-packet timing (no jitter)?
- loss-free delivery?
- in-order delivery?
- congestion feedback to sender?

The most important abstraction provided by network layer:

virtual circuit or datagram?

网络服务模型

Q: 什么是将数据包从发送者传输到接收者的服务模型"信道"？

service abstraction

- 带宽保证？
- 保持数据包间时序（无抖动）？
- 无差错传输？
- 有序传输？
- 向发送者提供拥塞反馈？

网络层提供的最重要抽象：

virtual

电路还是数据报？

Network layer connection and connection-less service

- Datagram network provides network-layer connectionless service
- VC network provides network-layer connection service
- Analogous to the transport-layer services,but:
  - Service: host-to-host
  - No choice: network provides one to the other
  - Implementation: in network core

网络层连接和无连接服务

- 数据报网络提供网络层的无连接服务
- 虚电路网络提供网络层连接服务
- 类似于传输层服务，但是：
  - 服务：主机到主机
  - 没有选择：网络提供一对一
  - 实现：在网络核心

Network layer service models:

Network Architecture	Service Model	Guarantees ?				Congestion feedback
		Bandwidth	Loss	Order	Timing	
Internet	best effort	none	no	no	no	no (inferred via loss)
ATM	CBR	constant rate	yes	yes	yes	no congestion
ATM	VBR	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR	guaranteed minimum	no	yes	no	yes
ATM	UBR	none	no	yes	no	no

网络层服务模型：

网络架构	服务模型	保证?				拥塞反馈
		带宽	Loss	顺序	时间	
互联网	尽力而为	none	no	no	no	无（推断通过丢包）
ATM	CBR	恒定 rate	yes	yes	yes	no 拥塞
ATM	VBR	保证 rate	yes	yes	yes	no 拥塞
ATM	ABR	保证最小	no	yes	no	yes
ATM	UBR	none	no	yes	no	no

Chapter 5: Network Layer

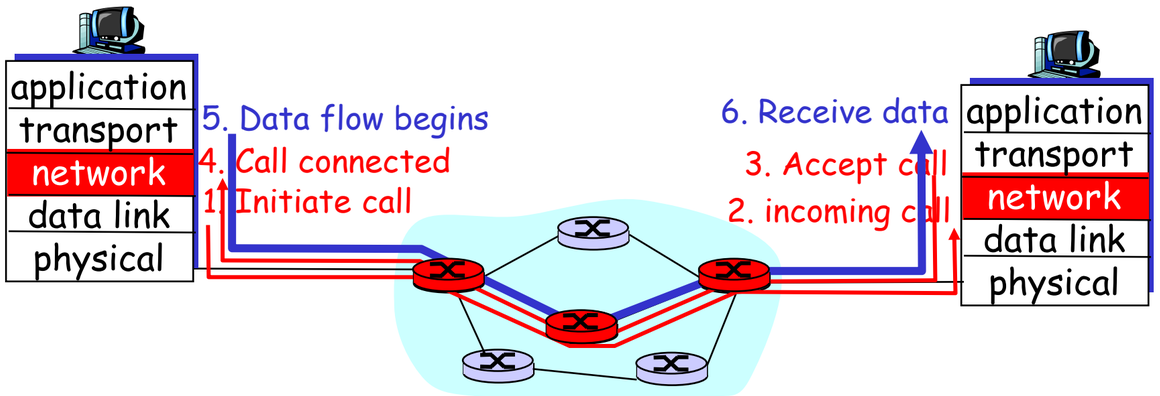
- 5.1 Introduction
- 5.2 Virtual circuit and datagram networks
- 5.3 What's inside a router?
- 5.4 Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector
  - Hierarchical routing
- 5.5 Routing in the Internet
- 5.6 IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6

5: 网络层

- 5.1 简介
- 5.2 虚拟电路与数据报网络
- 5.3 路由器里有什么?
- 5.4 路由算法:
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量 分层路由
- 5.5 互联网中的路由
- 5.6 IP: 互联网协议
  - IPv4数据报格式 IPv4寻址 IP分片 网络地址转换 ARP ICMP IPv6

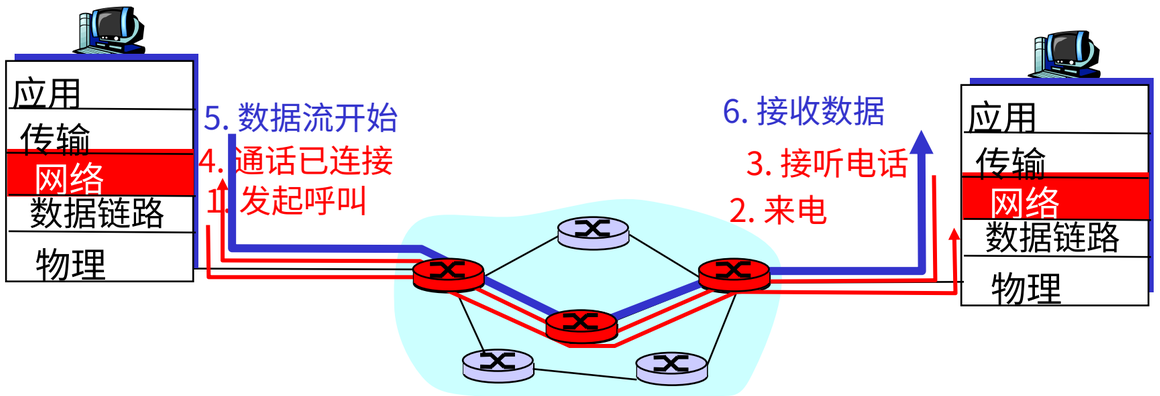
Virtual circuits: signaling protocols

- used to setup, maintain teardown VC
- used in ATM, frame-relay, X.25
- not used in today's Internet



虚拟电路：信令协议

- 用于建立、维护和拆除虚拟电路
- 用于ATM、帧中继、X.25
- 未在当今互联网中使用



Virtual circuits

"source-to-dest path behaves much like telephone circuit"
- performance-wise
- network actions along source-to-dest path

- call setup, teardown for each call *before* data can flow
- each packet carries VC identifier (not destination host OD)
- *every* router on source-dest path s maintain "state" for each passing connection
  - transport-layer connection only involved two end systems
- link, router resources (bandwidth, buffers) may be *allocated* to VC
  - to get circuit-like perf.

虚电路

"源节点到目的节点的路径表现得非常像电话电路"
- 性能方面 沿源节点到目的节点路径的网络动作

- 每个呼叫都需要进行呼叫建立和拆除 ，在数据可以流动
- 之前 每个数据包都携带虚电路标识符（而不是目的主机）OD)
- 每个源-目的路径上的路由器都为每个经过的连接维护 "状态"
  - 传输层连接仅涉及两个末端系统
- 链路、路由器资源（带宽、缓冲区）可以被分配给虚电路
  - 要获得电路般的性能。

VC Implementation



A VC consists of:

1. Path from source to destination
2. VC numbers, one number for each link along path
3. Entries in forwarding tables in routers along path

- Packet belonging to VC carries VC number(rather than dest. address)
- VC number can be changed on each link
  - New VC number comes from forwarding table

VC 实现

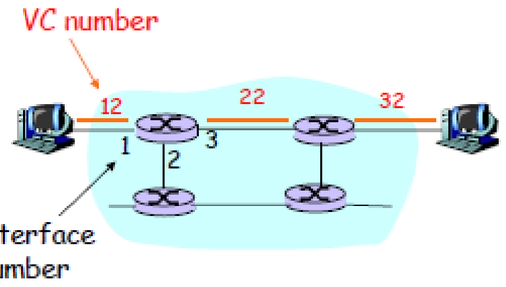


一个VC由：

1. 从源节点到目的节点的路径
2. VC 编号，每条链路使用一个编号
3. 沿路径中路由器转发表中的条目

- 属于 VC 的数据包携带 VC 编号（而不是目的地址）
- VC 编号可以在每条链路上更改
  - 新的VC数量来自转发表

Forwarding Table

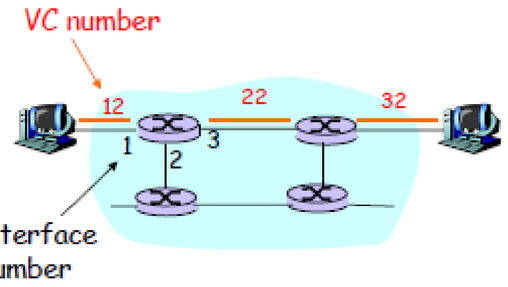


Forwarding table in northwest router:

Incoming interface	Incoming VC #	Outgoing interface	Outgoing VC #
1	12	3	22
2	63	1	18
3	7	2	17
1	97	3	87
...	...	...	...

Routers maintain connection state information!

Forwarding Table



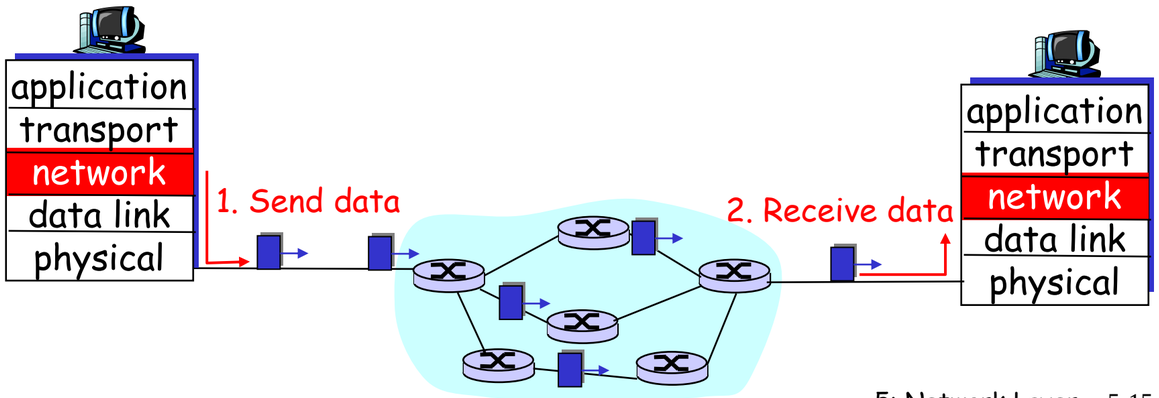
Forwarding table in northwest router:

Incoming interface	Incoming VC #	Outgoing interface	Outgoing VC #
1	12	3	22
2	63	1	18
3	7	2	17
1	97	3	87
...	...	...	...

Routers maintain connection state information!

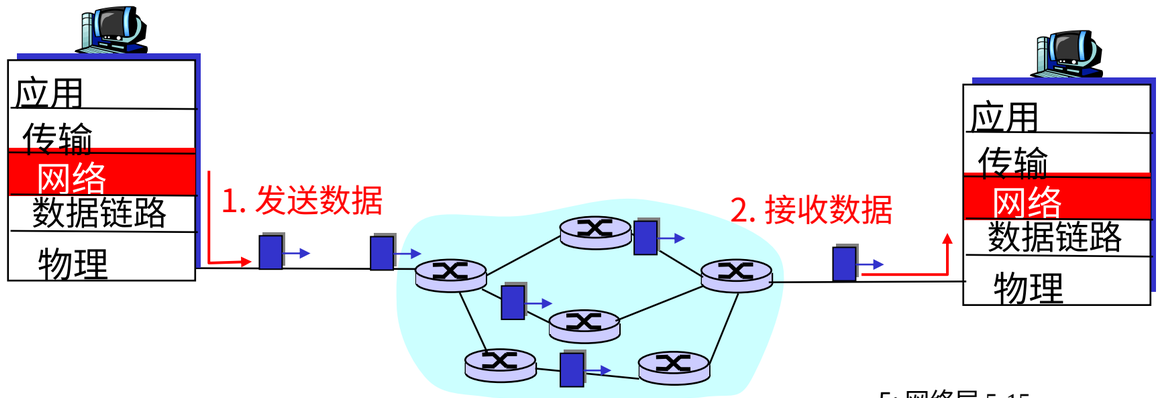
Datagram networks: the Internet model

- no call setup at network layer
- routers: no state about end-to-end connections
  - no network-level concept of "connection"
- packets typically routed using destination host ID
  - packets between same source-dest pair may take different paths



数据报网络： 互联网模型

- 网络层无呼叫建立过程
- 路由器：无端到端连接的状态信息
  - 无网络层的“连接”概念
- 数据包通常使用目的主机ID进行路由
  - 相同源-目的对之间的数据包可能走不同路径



Forwarding Table

4 billion possible entries

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Forwarding Table

4 billion possible entries

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Longest prefix matching

- when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address

Destination Address Range	Link interface
11001000 00010111 00010*** *********	0
11001000 00010111 00011000 *********	1
11001000 00010111 00011*** *********	2
otherwise	3

examples:

DA: 11001000 00010111 00010110 10100001 which interface?

DA: 11001000 00010111 00011000 10101010 which interface?

最长前缀匹配

- 当查找给定目的地址的转发表条目时，使用与目的地址匹配的<样式 id='2'>最长</样式>地址前缀

目的地址范围	链路接口
11001000 00010111 00010 *** *********	0
11001000 00010111 00011000 *********	1
11001000 00010111 00011 *** *********	2
否则	3

示例：

DA: 11001000 00010111 00010110 10100001 哪个接口?

DA: 11001000 00010111 00011000 10101010 哪个接口?

Datagram or VC network: why?

Internet

- data exchange among computers
  - "elastic" service, no strict timing req.
- "smart" end systems (computers)
  - can adapt, perform control, error recovery
  - simple inside network, complexity at "edge"
- many link types
  - different characteristics
  - uniform service difficult

ATM

- evolved from telephony
- human conversation:
  - strict timing, reliability requirements
  - need for guaranteed service
- "dumb" end systems
  - telephones
  - complexity inside network

数据报或虚电路网络：为什么?

互联网

- 计算机之间的数据交换
  - “弹性”服务，无严格时间要求。
- “智能”终端系统(计算机)。
  - 可以适应、执行控制、错误恢复。
  - 内部网络简单，复杂性在“边缘”。
- 多种链路类型
  - 不同特性
  - 统一服务困难

ATM

- 源自电话
- 人类对话：
  - 严格的时间要求、可靠性需求
  - 需要保证服务
- “哑”终端系统
  - 电话
  - 网络内部的复杂性

Chapter 5: Network Layer

- 5.1 Introduction
- 5.2 Virtual circuit and datagram networks
- 5.3 What's inside a router?
- 5.4 Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector
  - Hierarchical routing
- 5.5 Routing in the Internet
- 5.6 IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6

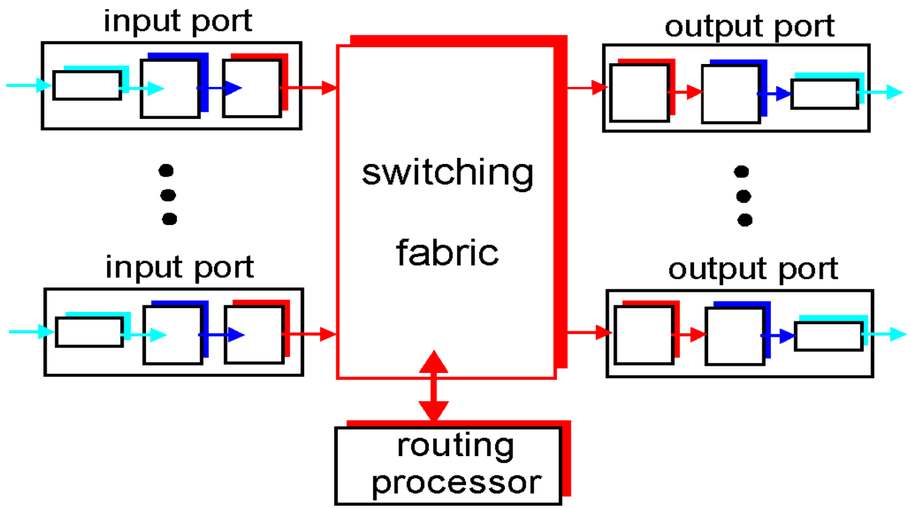
5: 网络层

- 5.1 简介
- 5.2 虚拟电路与数据报网络
- 5.3 路由器里有什么?
- 5.4 路由算法:
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量 分层路由
- 5.5 互联网中的路由
- 5.6 互联网协议: IP
  - IPv4数据报格式 IPv4寻址 IP分片 网络地址转换 ARP ICMP IPv6

Router Architecture Overview

Two key router functions:

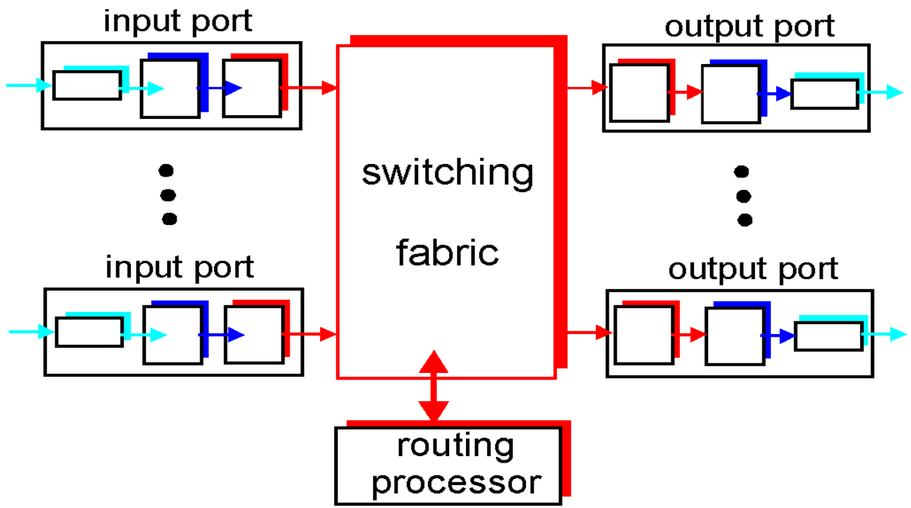
- run routing algorithms/protocol (RIP, OSPF, BGP)
- *switching* datagrams from incoming to outgoing link



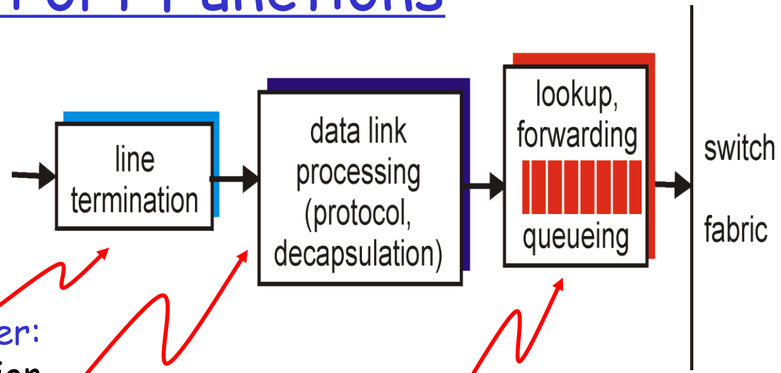
路由器架构概述

路由器的两个关键功能：

- 运行路由算法/协议（RIP、OSPF、BGP）
- 交换数据报从入站链路到出站链路



Input Port Functions



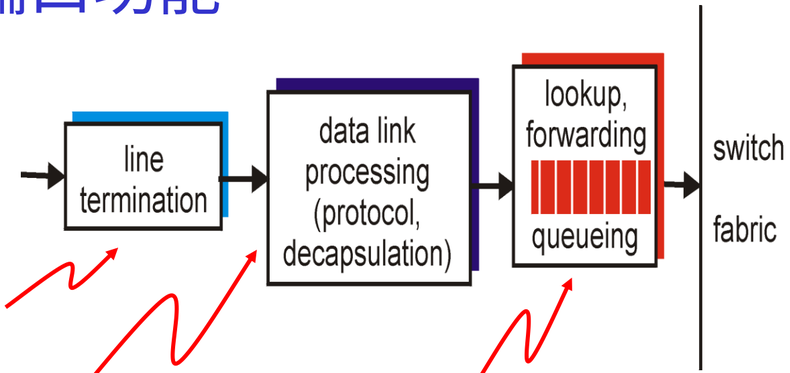
Physical layer:
bit-level reception

Data link layer:
e.g., Ethernet

Decentralized switching:

- given datagram dest., lookup output port using routing table in input port memory
- goal: complete input port processing at 'line speed'
- queuing: if datagrams arrive faster than forwarding rate into switch fabric

输入端口功能



物理层：
位级接收

数据链路层：
例如，以太网

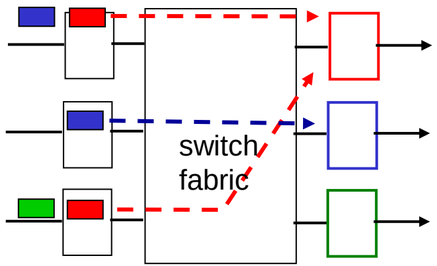
去中心化交换：

- 给定数据报目的，使用输入端口内存中的路由表查找输出端口
- 目标：在线速下完成输入端口处理
- 排队：如果数据包到达速度超过交换机矩阵的转发速率

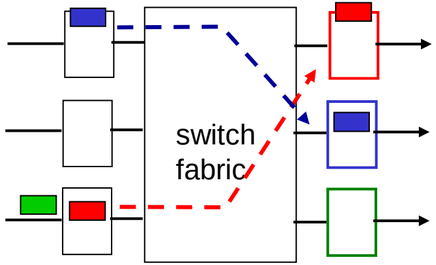
Input Port Queuing

- Fabric slower that input ports combined -> queueing may occur at input queues

  queueing delay and loss due to input buffer overflow!
- Head-of-the-Line (HOL) blocking: queued datagram at front of queue prevents others in queue from moving forward



output port contention:
only one red datagram can be transferred.
lower red packet is blocked

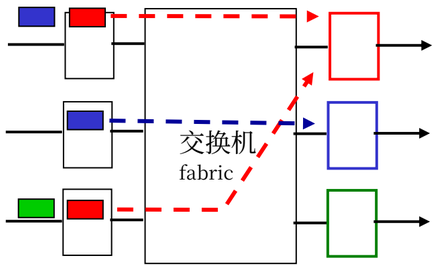


one packet time later:
green packet
experiences HOL
blocking

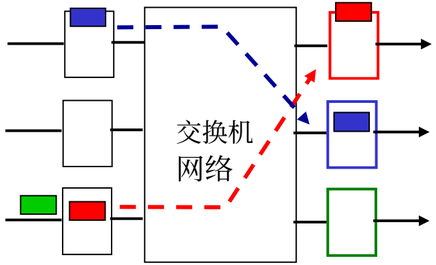
输入端口排队

- 矩阵比输入端口总和慢 -> 输入队列可能发生排队

  由于输入缓冲区溢出导致的排队延迟和丢包!
- 头部阻塞 (HOL)：队列前部的排队数据报会阻止队列中的其他数据报前进

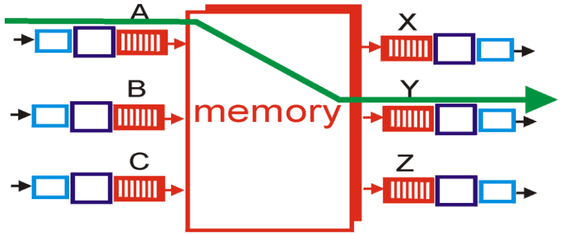


输出端口争用：只能有一个红色数据报被传输。低优先级红色数据包被阻塞

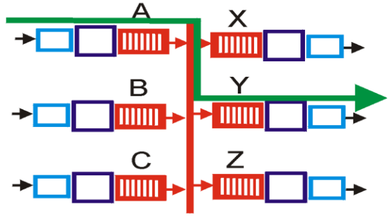


一个数据包时间后：
绿色数据包遭遇了
HOL阻塞

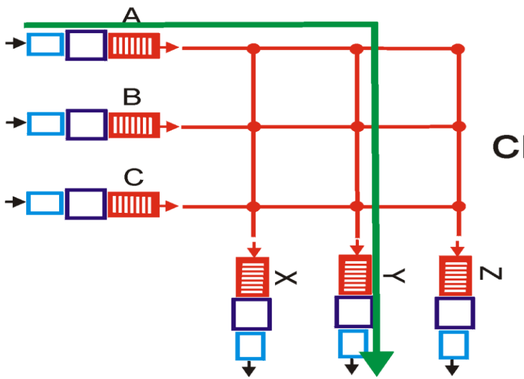
Three types of switching fabrics



memory

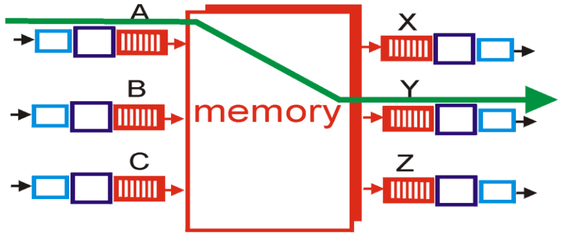


bus

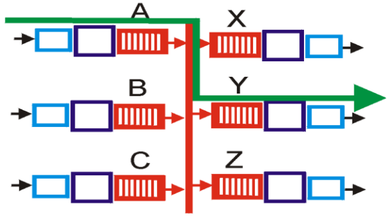


crossbar

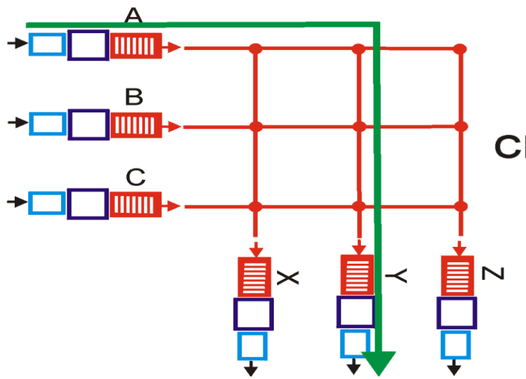
三种交换结构



memory



bus

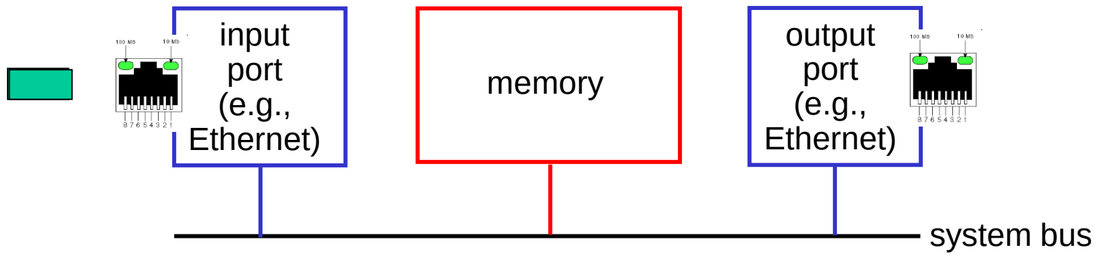


crossbar

Switching Via Memory

First generation routers:

- packet copied by system's (single) CPU
- speed limited by memory bandwidth (2 bus crossings per datagram)



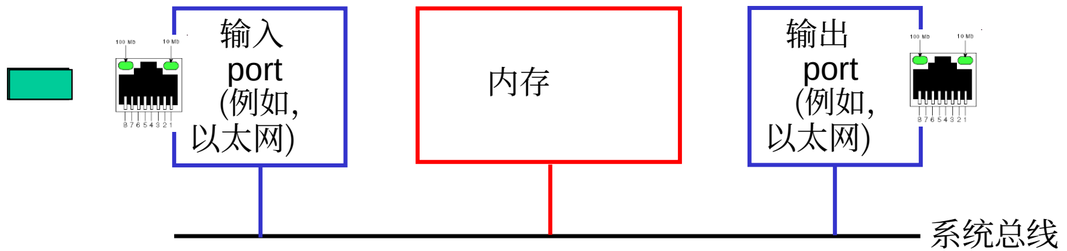
Modern routers:

- input port processor performs lookup, copy into memory
- Cisco Catalyst 8500

内存交换

第一代路由器：

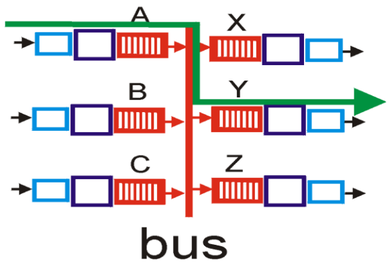
- 系统（单）CPU复制
- 速度受内存带宽限制（每个数据报2次总线穿越）



现代路由器：

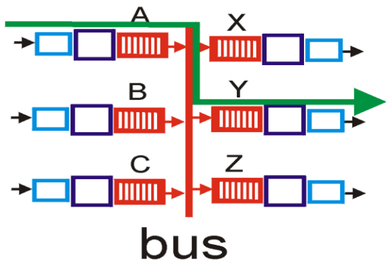
- 输入端口处理器执行查找，复制到内存
- 思科Catalyst 8500

Switching Via Bus



- datagram from input port memory to output port memory via a shared bus
- bus contention: switching speed limited by bus bandwidth
- 32 Gbps bus, Cisco 5600: sufficient speed for access and enterprise routers

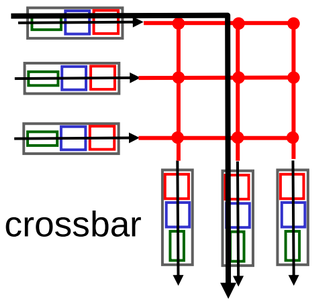
总线交换



- 数据报从输入端口内存通过共享总线传输到输出端口内存
- 总线争用： 交换速度受总线带宽限制
- 32 Gbps总线，Cisco 5600：足以满足接入和企业级路由器的速度需求

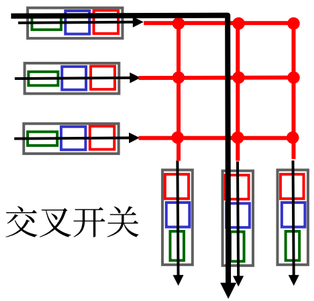
Switching Via An Interconnection Network

- overcome bus bandwidth limitations
- Banyan networks, crossbar, other interconnection nets initially developed to connect processors in multiprocessor
- Advanced design: fragmenting datagram into fixed length cells, switch cells through the fabric.
- Cisco 12000: switches 60 Gbps through the interconnection network

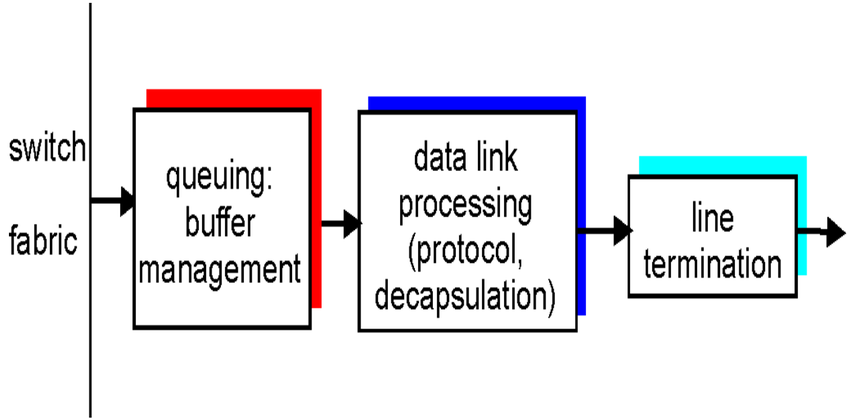


通过互连网络交换

- 克服总线带宽限制
- 百通网络、交叉开关、其他互连网络最初开发用于连接多处理器中的处理器
- 高级设计：将数据报分片为固定长度的信元，通过交换机在交换机之间传输信元
- 思科12000：通过互连网络交换 60 Gbps



Output Ports

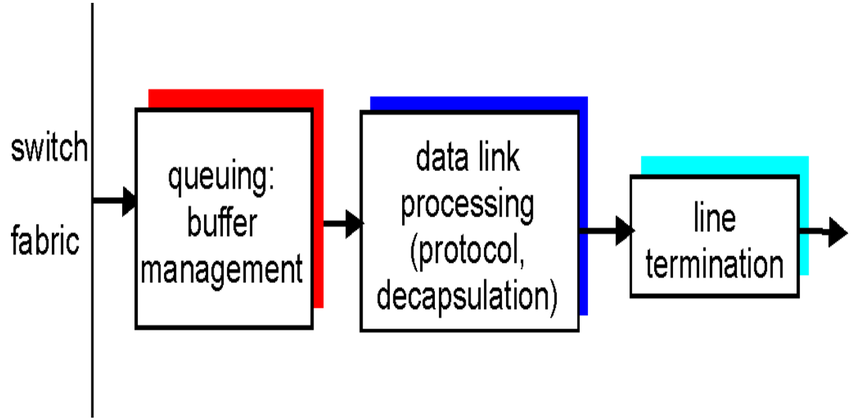


- *Buffering* required w
  fabric faster than th
- *Scheduling discipline* chooses among queued
  datagrams f

Datagram (packets) can be lost due to congestion, lack of buffers

Priority scheduling – who gets best performance, network neutrality

输出端口

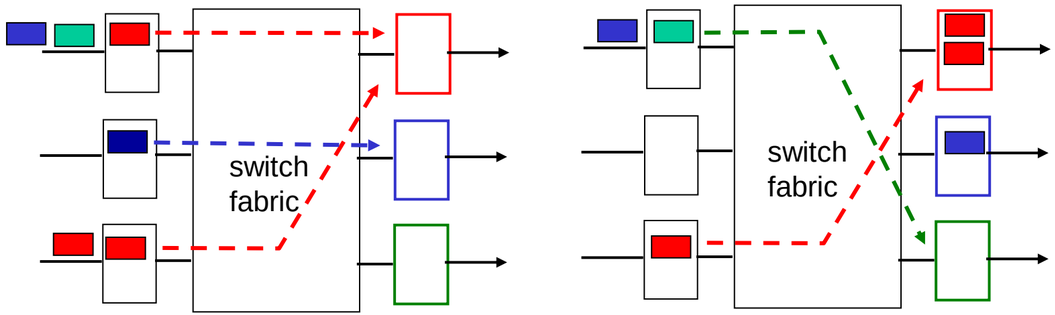


- 缓冲当数据报从
  网络结构比传输速率快
- 调度策略在排队
  数据报中选择

数据报（数据包）可能会丢失
由于拥塞，缓冲区不足

优先级调度——谁能获得最佳性能，网络中立性

Output port queueing

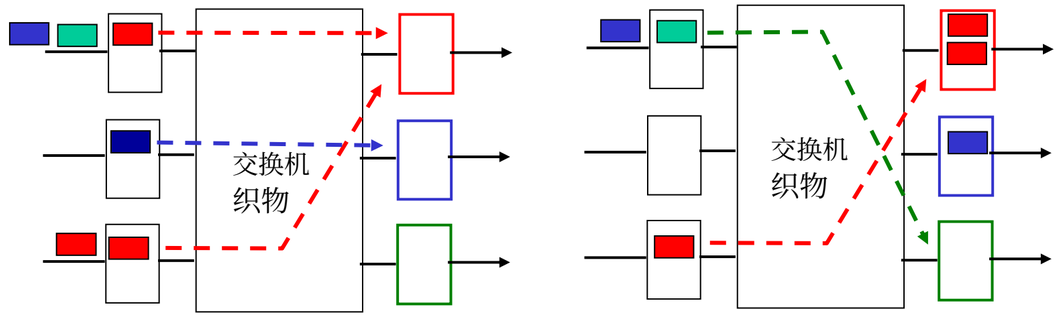


at *t*, packets more from input to output

one packet time later

- buffering when arrival rate via switch exceeeds ouput line speed
- *queueing (delay) and loss due to output port buffer overflow!*

输出端口排队



在t时刻，数据包更多从输入到输出

一个数据包时间后

- 当通过交换机的到达率超过输出线速时出现缓冲
- 排队（延迟）和因输出端口缓冲区溢出导致的丢包!

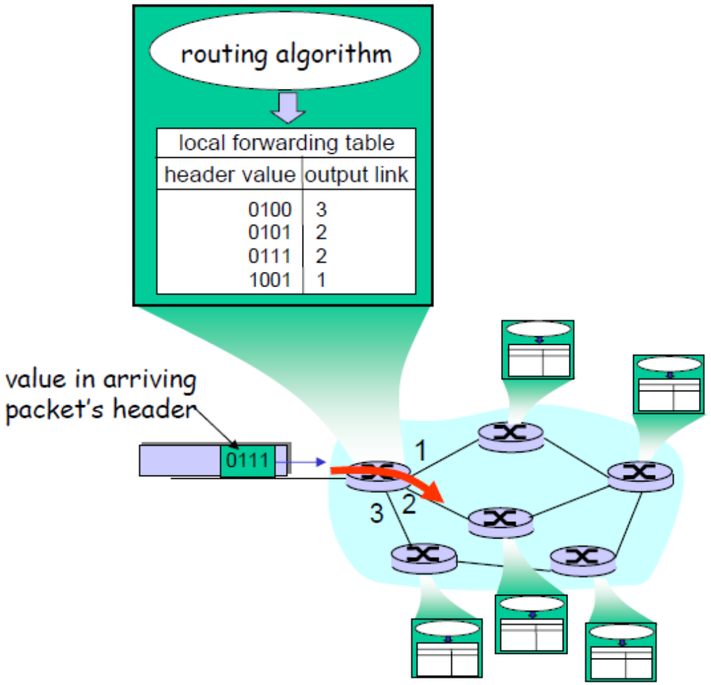
Chapter 5: Network Layer

- 5.1 Introduction
- 5.2 Virtual circuit and datagram networks
- 5.3 What's inside a router?
- 5.4 Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector
  - Hierarchical routing
- 5.5 Routing in the Internet
- 5.6 IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6

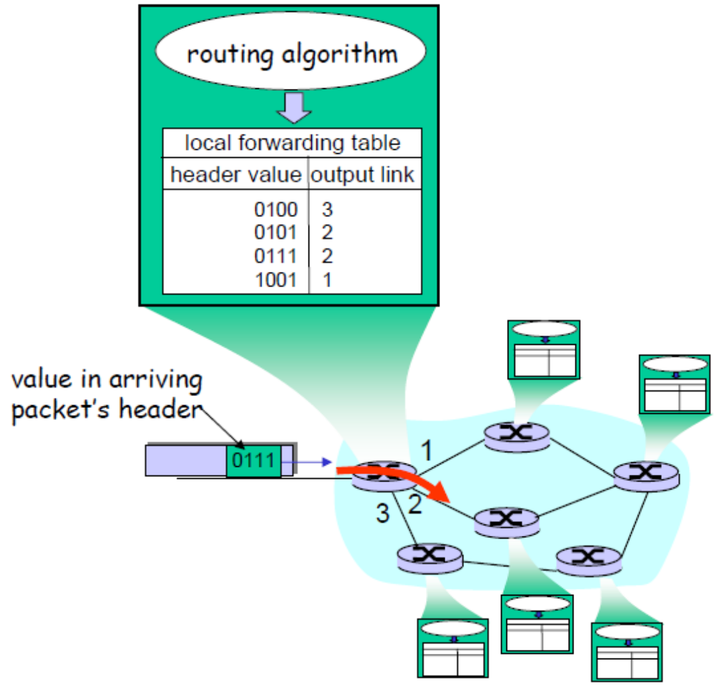
5: 网络层

- 5.1 简介
- 5.2 虚拟电路与数据报网络
- 5.3 路由器里有什么?
- 5.4 路由算法:
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量 分层路由
- 5.5 互联网中的路由
- 5.6 IP: 互联网协议
  - IPv4 数据报格式 IPv4 地址 IP 分片 网络地址转换 ARP ICMP IPv6

Interplay between routing and forwarding



路由与转发之间的相互作用



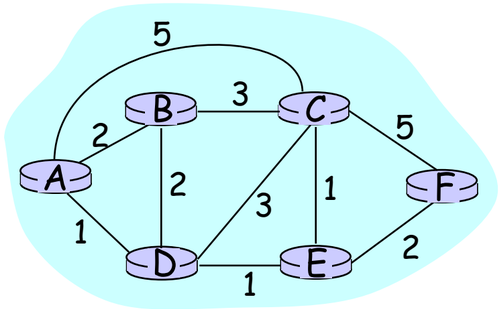
Routing

Routing protocol

Goal: determine "good" path (sequence of routers) thru network from source to dest.

Graph abstraction for routing algorithms:

- graph nodes are routers
- graph edges are physical links
  - link cost: delay, $ cost, or congestion level



- "good" path:
  - typically means minimum cost path
  - other def's possible

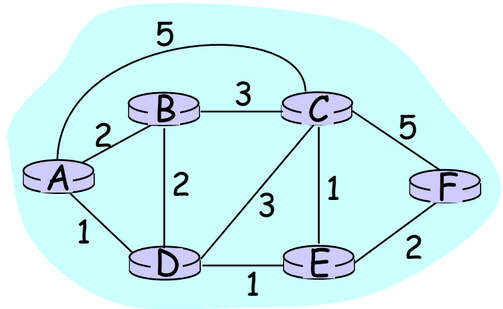
路由

路由协议

目标：确定从源节点到目的节点的网络中的"良好"路径（路由器的序列）。

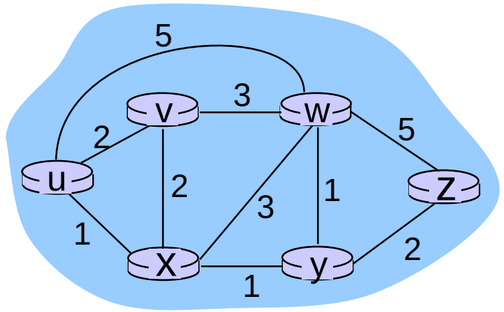
路由算法的图抽象：

- 图节点是路由器
- 图边是物理链路
  - 链路成本：延迟、$成本或拥塞水平



- "好"路径：通常表示最小成本路径其他定义也可能存在

Graph abstraction of the network



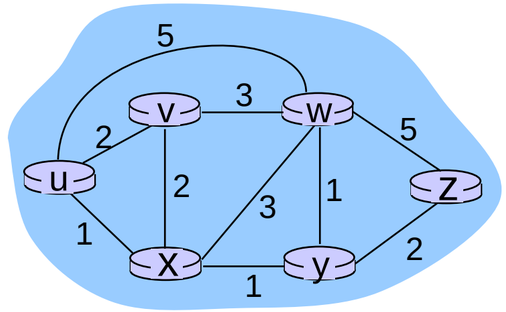
graph: G = (N,E)

N = set of routers = { u, v, w, x, y, z }

E = set of links ={ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) }

aside: graph abstraction is useful in other network contexts, e.g., P2P, where *N* is set of peers and *E* is set of TCP connections

网络图的抽象



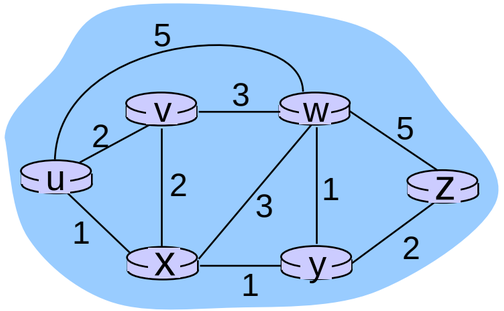
图：G = (N,E)

N = 一组路由器 = { u, v, w, x, y, z }

E = set of links ={ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) }

侧边栏：图抽象在网络其他上下文中也有用，例如P2P，其中N是节点集，E是TCP连接集

Graph abstraction: costs



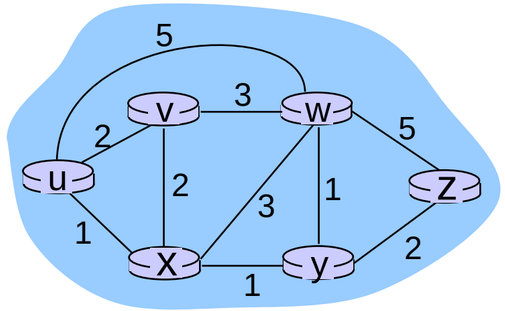
c(x,x') = cost of link (x,x')
e.g., c(w,z) = 5

cost could always be 1, or inversely related to bandwidth, or inversely related to congestion

cost of path $(x_1, x_2, x_3, \ldots, x_p) = c(x_1,x_2) + c(x_2,x_3) + \ldots + c(x_{p-1},x_p)$

key question: what is the least-cost path between u and z ?
routing algorithm: algorithm that finds that least cost path

图抽象：成本



c(x,x') = 链接(x,x')的成本,
例如c(w,z) = 5

成本始终为 1，或者与带宽成反比，或者与拥塞成反比

路径 $(x_1, x_2, x_3, \ldots, x_p) = c(x_1,x_2) + c(x_2,x_3) + \ldots + c(x_{p-1},x_p)$

关键问题： u 和 z 之间的最短路径是什么？路由算法： 找到最短路径的算法

Routing Algorithm classification

Global or decentralized information?

Global:

- all routers have complete topology, link cost info
- "link state" algorithms

Decentralized:

- router knows physically-connected neighbors, link costs to neighbors
- iterative process of computation, exchange of info with neighbors
- "distance vector" algorithms

Static or dynamic?

Static:

- routes change slowly over time

Dynamic:

- routes change more quickly
  - periodic update
  - in response to link cost changes

路由算法分类

全局或去中心化信息?

全局:

- 所有路由器都有完整的拓扑结构和链路成本信息
- "链路状态"算法

去中心化:

- 路由器知道物理连接的邻居，以及到邻居的链路成本
- 计算迭代过程，与邻居交换信息
- "距离向量"算法

静态还是动态?

静态:

- 路由随时间缓慢变化

动态:

- 路由变化更快
  - 周期性更新❍ 以响应链路成本变化

Dijkstra's algorithm

- Net topology,link costs known to all nodes
- Computes least cost paths from one node(source) to all other nodes
- Iterative: after k iterations, know least cost path to k dest.'s

Notation:

- c(x,y): link cost from node x to y; = ∞ if not direct neighbors
- D(v): current value of cost of path from source to dest. V
- p(v): predecessor node along path from source to v
- N': set of nodes whose least cost path definitively known

迪杰斯特拉算法

- 网络拓扑，链路成本对所有节点已知
- 计算从单个节点（源节点）到所有其他节点的<code style id='2'>最短路径</code>
- 迭代：在 k 次迭代后，知道到 k 个目的节点的最短路径

符号：

- c(x,y)：节点 x 到节点 y 的链路成本；= ∞ 如果它们不是直接邻居- D(v)：从源节点到目的节点的路径当前成本值。V- p(v)：路径上从源节点到 v 的前驱节点
- N'：已知最短路径的节点集

Dijkstra's algorithm



```
1  Initialization:
2    N' = {u}
3    for all nodes v
4      if v adjacent to u
5          then D(v) = c(u,v)
6      else D(v) = ∞
7
8  Loop
9    find w not in N' such that D(w) is a minimum
10   add w to N'
11   update D(v) for all v adjacent to w and not in N' :
12      D(v) = min( D(v), D(w) + c(w,v) )
13   /* new cost to v is either old cost to v or known
14    shortest path cost to w plus cost from w to v */
15 until all nodes in N'
```


迪杰斯特拉算法



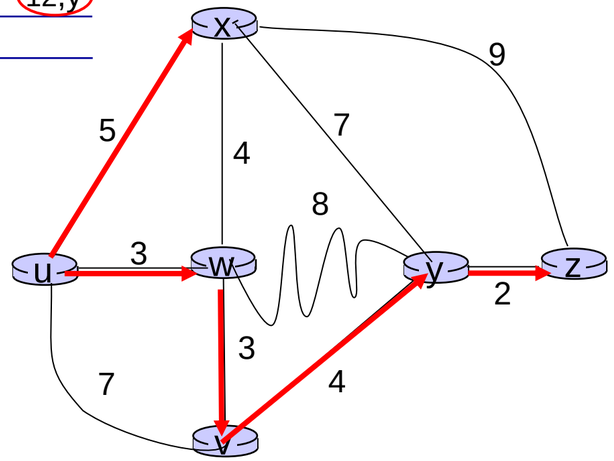
```
1  Initialization:
2    N' = {u}
3    for all nodes v
4      if v adjacent to u
5          then D(v) = c(u,v)
6      else D(v) = ∞
7
8  Loop
9    find w not in N' such that D(w) is a minimum
10   add w to N'
11   update D(v) for all v adjacent to w and not in N' :
12      D(v) = min( D(v), D(w) + c(w,v) )
13   /* new cost to v is either old cost to v or known
14    shortest path cost to w plus cost from w to v */
15 until all nodes in N'
```

Dijkstra's algorithm: example

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

notes:

- construct shortest path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)

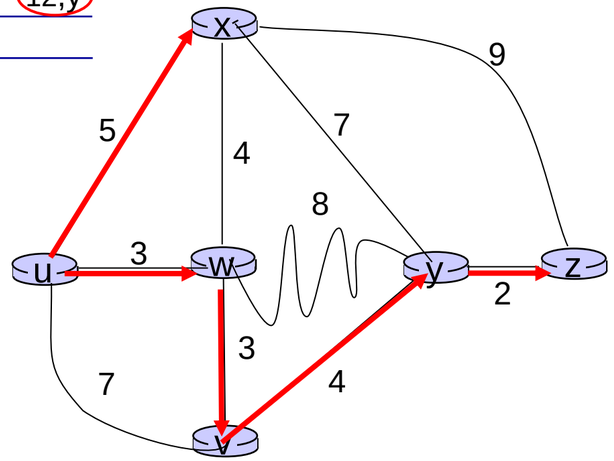


迪杰斯特拉算法：示例

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			14,x 11,w	
3	uwxv				14,x 10,v	
4	uwxvy					12,y
5	uwxvyz					

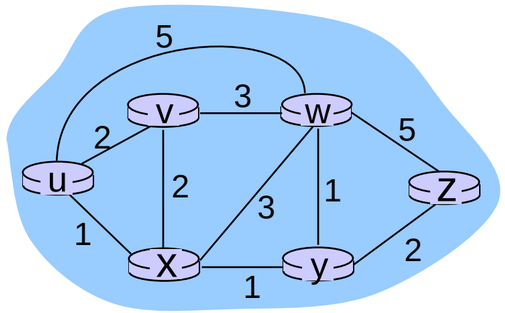
笔记：

- 通过追踪前驱节点构建最短路径树- 关系可以存在（可以任意断开）



Dijkstra's algorithm: another example

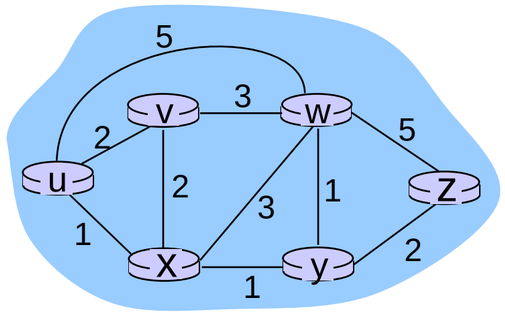
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

迪科斯彻的算法：另一个例子

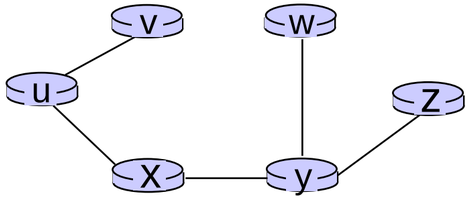
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



* 查看在线互动练习以获取更多示例：
http://gaia.cs.umass.edu/kurose_ross/interactive/

Dijkstra's algorithm: example (2)

resulting shortest-path tree from u:

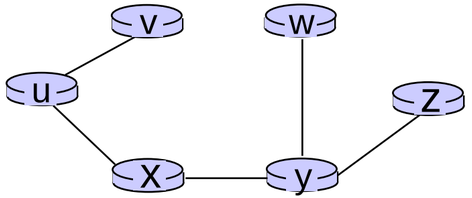


resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

迪杰斯特拉的算法：示例 (2)

从 u 生成的最短路径树：

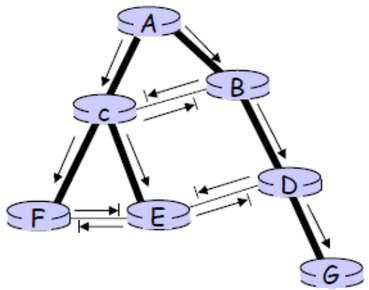


u: 结果转发表

目的	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

Broadcast routing

- Flooding: Deliver packets from source to all other nodes
  - Broadcast storm

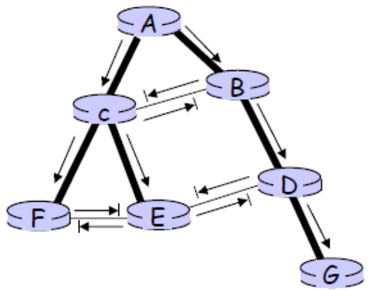


- Controlled flooding:
  - Sequence-number-controlled flooding: add *ID + broadcast sequence number* into the broadcast packets
  - reverse path forwarding, RPF
- Be able to avoid the broadcast storm

RFP: Reverse path forwarding

广播路由

- 泛洪： 将数据包从源节点发送到所有其他节点 广播风暴

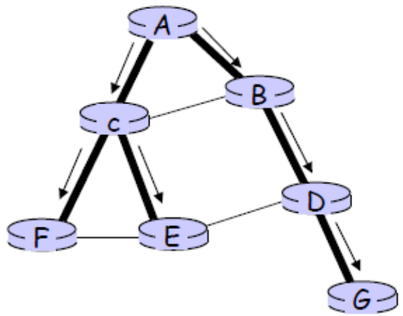


- 控制泛洪：
  - 序列号- 控制泛洪： 向广播数据包中添加*ID* + 广播序列号
  - 反向路径转发，RPF
- 能够 避免广播风暴

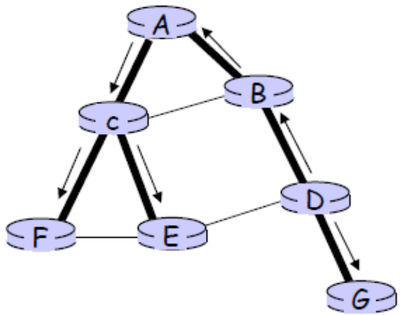
RFP: Reverse path forwarding

Broadcast routing

- Spanning-tree broadcast
  - Suppose you have a connected undirected graph
  - …then a spanning tree of the graph is a connected subgraph in which there are no cycles
- Be able to avoid the transmission of redundancy broadcast packets



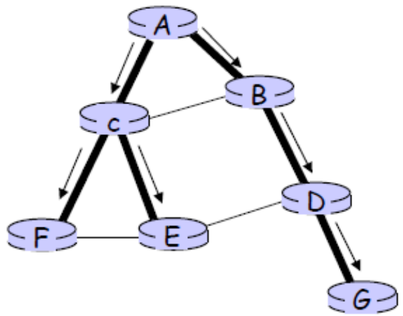
(a) Broadcast initiated at A



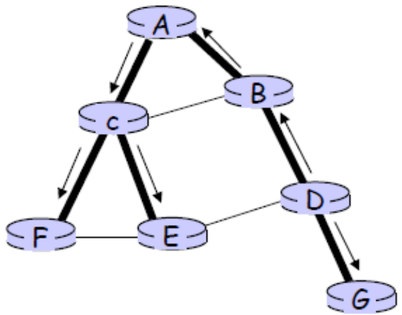
(b) Broadcast initiated at D

广播路由

- 生成树广播
  - 假设你有一个连通的无向图
  - ...然后，图的生成树是一个连通的子图，其中不包含任何环路
- 能够 避免传输冗余广播数据包



(a) Broadcast initiated at A



(b) Broadcast initiated at D

Link state algorithm

Broadcast routing + Dijkstra's algorithm

- Having each node broadcast link-state packets to all other nodes → all nodes have an identical and complete view of the network.
- Using Dijkstra's algorithm compute the least-cost path from one node to all other nodes in the network
- If a link cost changes, re-broadcast and re-compute

链路状态算法

广播路由 + 迪杰斯特拉算法

- 每个节点 广播链路状态数据包给 所有 其他节点 → 所有节点对网络都有一个相同且完整的视图。
- 使用迪杰斯特拉算法 计算从一节点到网络中所有其他节点的最短路径
- 如果链路成本发生变化，则重新广播并重新计算

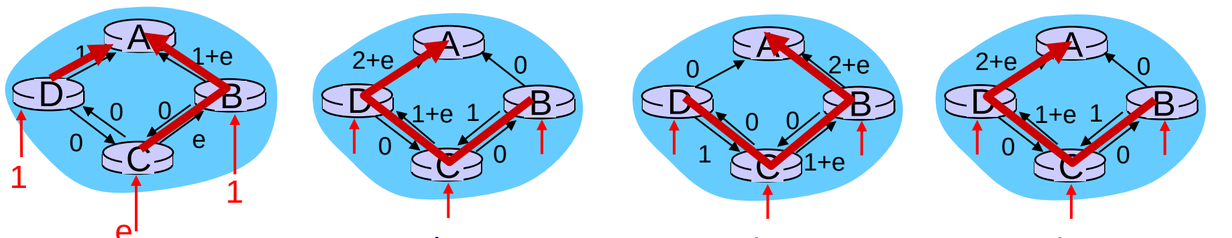
Link state algorithm,discussion

Algorithm complexity: n nodes

- Each iteration: need to check all nodes, w, not in N
- N(n+1)/2 comparisons:$O(n^2)$
- More efficient implementations possible: O(nlogn)

Oscillations possible:

- E.g., link cost = amount of carried traffic



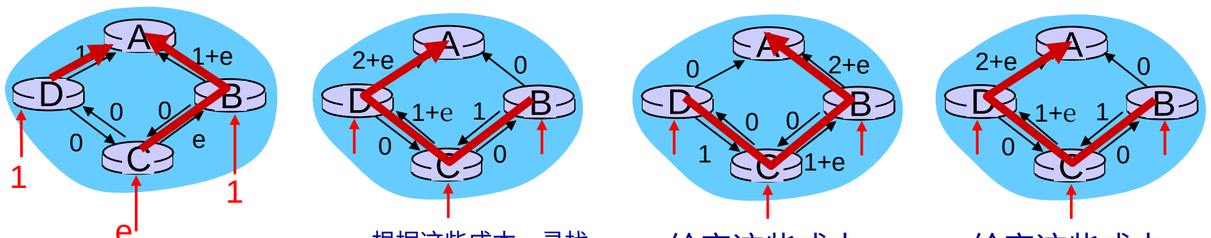
链路状态算法，讨论

算法复杂度：n个节点

- 每次迭代：需要检查所有不在节点集N中的节点w，进行N(n+1)/2次比较：$O(n^2)$
- 可以实现更高效的实现：O(nlogn)

可能出现振荡：

- 例如，链路成本= 承载的流量大小



Distance vector algorithm

Bellman-Ford equation (dynamic programming)

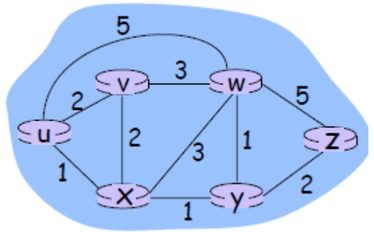
Define

$d_x(y)$:= cost of least-cost path from x to y

Then

$d_x(y)=\min_v \{c(x,v)+d_v(y)\}$

where min is taken over all neighbors v of x



For example:

Clearly, $d_v(z)=5$, $d_w(z)=3$, $d_x(z)=3$

B-F equation says:

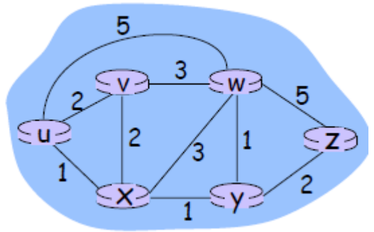
$d_u(z)=\min\{c(u,v)+d_v(z), c(u,x)+d_x(z), c(u,w)+d_w(z)\}$
$=\min(2+5,1+3,5+3)=4$

距离向量算法

贝尔曼-福特方程（动态规划）

定义 $d_x(y)$：= 从 x 到 y 的最短路径成本

然后$d_x(y)=$最小$_v\{c(x,v)+d_v(y)\}$，其中最小值是在x的所有邻居v上取的



例如：显然，$d_v(z)=5$，$d_w(z)=3$，$d_x(z)=3$

B-F方程式说明：$d_u(z)=\min\{c(u,v)+d_v(z), c(u,x)+d_x(z), c(u,w)+d_w(z)\}=\min(2+5,1+3,5+3)=4$

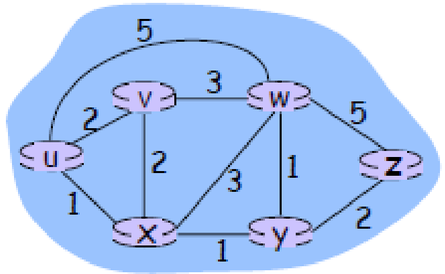
Distdance vector algorithm

- $D_x(y)$ = estimate of least cost from x to y
  - x maintains distance vector $\mathbf{D}_x = [D_x(y): y \in N]$
- node x:
  - knows cost to each neighbor v: c(x,v)
  - maintains its neighbors' distance vectors. For each neighbor v, x maintains $\mathbf{D}_v = [D_v(y): y \in N]$

Distdance向量算法

- $D_x(y)$ = 从x到y的最小成本估计
  - x维护距离向量<code>D</code>$_x$ = [<code>D</code>$_x$ <code>(y): y </code><code>∈ </code><code>N</code>]
- 节点x：
  - <code>知道到每个邻居v的成本：</code><code>c(x,v)</code>
  - <code>维护其邻居</code><code>’ </code><code>距离向量。对于</code><code>每个邻居v，x维护</code><code>D</code>$_v$ = $[D_v(y): y \in N]$

Bellman-Ford Example



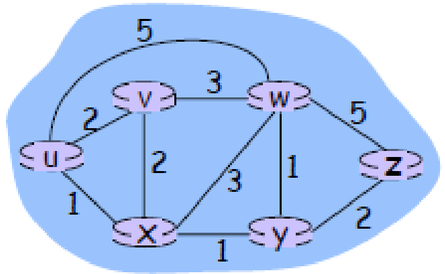
Distance vectors stored at node x

		Cost to x	y	z	u	v	w
from	x	0	1	3	1	2	2
	y	1	0	2	2	3	1
	u	1	2	4	0	2	5
	v	2	3	5	3	0	3
	w	2	1	3	5	3	0

Routing table at node x

	destination y	z	u	v	w
hop, cost	y,1	y,1	u,1	v,2	y,2

贝尔曼-福特示例



Distance vectors stored at node x

		Cost to x	y	z	u	v	w
from	x	0	1	3	1	2	2
	y	1	0	2	2	3	1
	u	1	2	4	0	2	5
	v	2	3	5	3	0	3
	w	2	1	3	5	3	0

Routing table at node x

	destination y	z	u	v	w
hop, cost	y,1	y,1	u,1	v,2	y,2

Distdance vector algorithm

Basic idea:

- Each node periodically sends its own distance vector estimate to neighbors

- When a node x receives new DV estimate from neighbor, it updates its own DV using B-F equation

$D_x(y) \leftarrow \min_v\{c(x,v)+D_v(y)\}$ for each node $y \in N$

- Under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

Distdance向量算法

基本思想：

- 每个节点周期性地将其自身的距离向量估计发送给邻居

- 当节点x从邻居处接收到新的DV估计时，它使用B-F方程更新自身的DV

$D_x(y) \leftarrow \min_v\{c(x,v)+D_v(y)\}$ 对于每个节点 $y \in N$

- 在轻微、自然条件下，估计 $D_x(y)$ 收敛到实际最小成本 $d_x(y)$

Distance vector algorithm

Iterative, asynchronous:

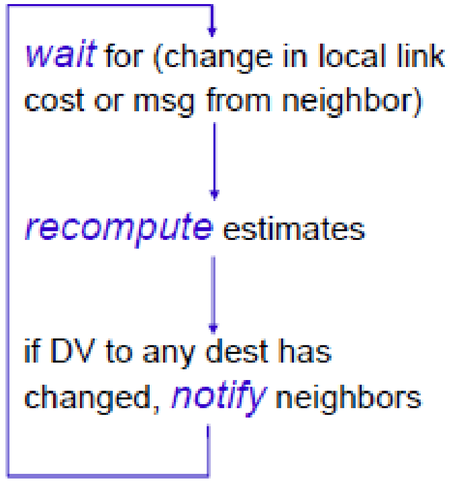
Each local iteration caused by:

- Local link cost change
- DV update message from neighbor

Distributed:

- Each node notifies neighbors only when its DV change
- Neighbors then notify their neighbors if necessary

Each node:



距离向量算法

迭代、异步：

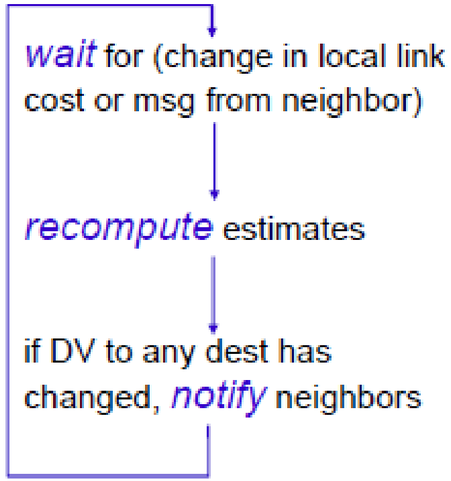
每次局部迭代由：

- 本地链路成本变化
- 来自邻居的DV更新消息

分布式：

- 每个节点仅在其DV变化时通知邻居 才通知
- 邻居在必要时会通知它们的邻居

Each node:



$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

node x table

cost to

from	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

cost to

from	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

node y table

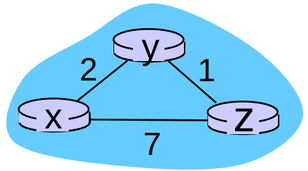
cost to

from	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

node z table

cost to

from	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0



time

$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

节点x表

成本至

从	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

到

从	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

节点 y 表

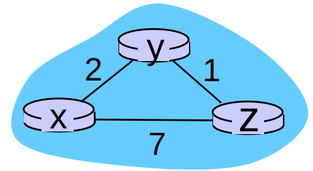
成本至

从	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

节点 z 表

成本到

从	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0



time

$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

node x table

cost to

from	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

from	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

from	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

node y table

cost to

from	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

from	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

from	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

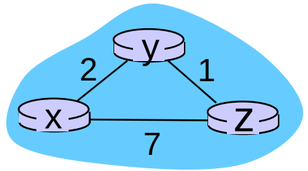
node z table

cost to

from	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

from	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0



time

$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

节点 x 表

成本到

来自	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

来自	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

来自	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

节点 y 表

成本至

来自	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

来自	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

来自	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

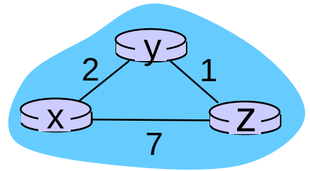
节点 z 表

成本至

来自	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

来自	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

来自	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0



time

Distance vector: link cost changes

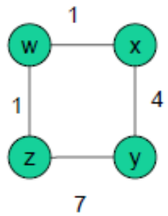
Link cost changes:

- Node detects local link cost change
- Updates routing info, recalculates distance vector
- If DV changes, notigy neighbors

"good news travels fast, bad news travels slow"

"good news travels fast"





node w table

from	cost to x	y	z	w
w	1	5	1	0
x	0	4	2	1
z	2	6	0	1

node x table

from	cost to x	y	z	w
x	0	4	2	1
w	1	5	1	0
y	4	0	6	5

node y table

from	cost to x	y	z	w
y	4	0	6	5
x	0	4	2	1
z	2	6	0	1

node z table

from	cost to x	y	z	w
z	2	6	0	1
w	1	5	1	0
y	4	0	6	5

距离向量：链路成本变化

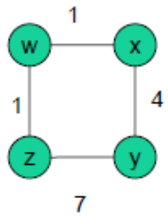
链路成本变化：

- 节点检测到本地链路成本变化
- 更新路由信息，重新计算距离向量
- 如果距离向量变化，则通知邻居

“好消息传得快，坏消息传得慢”

"good news travels fast"





node w table

from	cost to x	y	z	w
w	1	5	1	0
x	0	4	2	1
z	2	6	0	1

node x table

from	cost to x	y	z	w
x	0	4	2	1
w	1	5	1	0
y	4	0	6	5

node y table

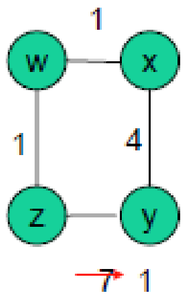
from	cost to x	y	z	w
y	4	0	6	5
x	0	4	2	1
z	2	6	0	1

node z table

from	cost to x	y	z	w
z	2	6	0	1
w	1	5	1	0
y	4	0	6	5

"good news travels fast"

Algorithm converges in 3 steps.



node w table

from \ cost to	x	y	z	w
w	1	5	1	0
x	0	4	2	1
z	2	6	0	1

from \ cost to	x	y	z	w
w	1	**2**	1	0
x	0	4	2	1
z	2	1	0	1

from \ cost to	x	y	z	w
w	1	2	1	0
x	0	4	2	1
z	2	1	0	1

node x table

from \ cost to	x	y	z	w
x	0	4	2	1
w	1	5	1	0
y	4	0	6	5

from \ cost to	x	y	z	w
x	0	4	2	1
w	1	5	1	0
y	3	0	1	2

from \ cost to	x	y	z	w
x	0	**3**	2	1
w	1	2	1	0
y	3	0	1	2

node y table

from \ cost to	x	y	z	w
y	**3**	0	**1**	**2**
x	0	4	2	1
z	2	6	0	1

from \ cost to	x	y	z	w
y	3	0	1	2
x	0	4	2	1
z	2	1	0	1

from \ cost to	x	y	z	w
y	3	0	1	2
x	0	4	2	1
z	2	1	0	1

node z table

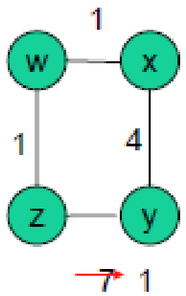
from \ cost to	x	y	z	w
z	2	**1**	0	1
w	1	5	1	0
y	4	0	6	5

from \ cost to	x	y	z	w
z	2	1	0	1
w	1	5	1	0
y	4	0	6	5

from \ cost to	x	y	z	w
z	2	1	0	1
w	1	2	1	0
y	3	0	1	2

"good news travels fast"

Algorithm converges in 3 steps.



node w table

from \ cost to	x	y	z	w
w	1	5	1	0
x	0	4	2	1
z	2	6	0	1

from \ cost to	x	y	z	w
w	1	**2**	1	0
x	0	4	2	1
z	2	1	0	1

from \ cost to	x	y	z	w
w	1	2	1	0
x	0	4	2	1
z	2	1	0	1

node x table

from \ cost to	x	y	z	w
x	0	4	2	1
w	1	5	1	0
y	4	0	6	5

from \ cost to	x	y	z	w
x	0	4	2	1
w	1	5	1	0
y	3	0	1	2

from \ cost to	x	y	z	w
x	0	**3**	2	1
w	1	2	1	0
y	3	0	1	2

node y table

from \ cost to	x	y	z	w
y	**3**	0	**1**	**2**
x	0	4	2	1
z	2	6	0	1

from \ cost to	x	y	z	w
y	3	0	1	2
x	0	4	2	1
z	2	1	0	1

from \ cost to	x	y	z	w
y	3	0	1	2
x	0	4	2	1
z	2	1	0	1

node z table

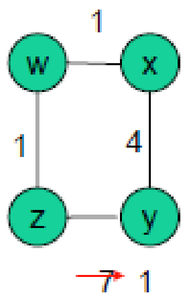
from \ cost to	x	y	z	w
z	2	**1**	0	1
w	1	5	1	0
y	4	0	6	5

from \ cost to	x	y	z	w
z	2	1	0	1
w	1	5	1	0
y	4	0	6	5

from \ cost to	x	y	z	w
z	2	1	0	1
w	1	2	1	0
y	3	0	1	2

"good news travels fast"

Algorithm converges in 3 steps.



node w table

from	cost to x	y	z	w
w	1	2	1	0
x	0	4	2	1
z	2	1	0	1

from	cost to x	y	z	w
w	1	2	1	0
x	0	3	2	1
z	2	1	0	1

node x table

from	cost to x	y	z	w
x	0	3	2	1
w	1	2	1	0
y	3	0	1	2

from	cost to x	y	z	w
x	0	3	2	1
w	1	2	1	0
y	3	0	1	2

node y table

from	cost to x	y	z	w
y	3	0	1	2
x	0	4	2	1
z	2	1	0	1

from	cost to x	y	z	w
y	3	0	1	2
x	0	3	2	1
z	2	1	0	1

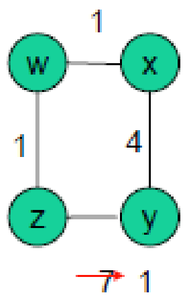
node z table

from	cost to x	y	z	w
z	2	1	0	1
w	1	2	1	0
y	3	0	1	2

from	cost to x	y	z	w
z	2	1	0	1
w	1	2	1	0
y	3	0	1	2

"good news travels fast"

Algorithm converges in 3 steps.



node w table

from	cost to x	y	z	w
w	1	2	1	0
x	0	4	2	1
z	2	1	0	1

from	cost to x	y	z	w
w	1	2	1	0
x	0	3	2	1
z	2	1	0	1

node x table

from	cost to x	y	z	w
x	0	3	2	1
w	1	2	1	0
y	3	0	1	2

from	cost to x	y	z	w
x	0	3	2	1
w	1	2	1	0
y	3	0	1	2

node y table

from	cost to x	y	z	w
y	3	0	1	2
x	0	4	2	1
z	2	1	0	1

from	cost to x	y	z	w
y	3	0	1	2
x	0	3	2	1
z	2	1	0	1

node z table

from	cost to x	y	z	w
z	2	1	0	1
w	1	2	1	0
y	3	0	1	2

from	cost to x	y	z	w
z	2	1	0	1
w	1	2	1	0
y	3	0	1	2

Count to infinity problem

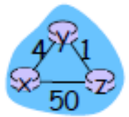
Link cost changes:

- good news travels fast
- Bad news travels slow - "count to infinity" problem!

- for example:
  44 iterations before algorithm stabilizes

"bad news travels slow"

"count to infinity" problem



node x table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

node y table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

node z table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

计数到无穷问题

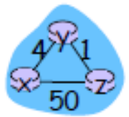
链路成本变化：

- 好消息传得快
- 坏消息 传得慢- "计数到无穷"问题！

- 例如：算法稳定前需要44次迭代

"bad news travels slow"

"count to infinity" problem



node x table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

node y table

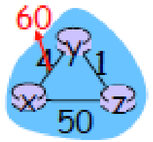
from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

node z table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

"bad news travels slow"

Algorithm converges in 44 steps.



Cost of link xy changes

node x table

from \ cost to	x	y	z
x	0	51	50
y	4	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	6	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	6	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	51	50
y	8	0	1
z	7	1	0

node y table

from \ cost to	x	y	z
x	0	4	5
y	6	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	6	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	8	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	51	50
y	8	0	1
z	7	1	0

node z table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	6	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	51	50
y	6	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	51	50
y	8	0	1
z	9	1	0

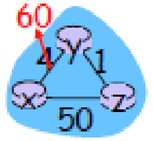
"bad news travels slow"

Algorithm converges in 44 steps.

Cost of link xy changes

"bad news travels slow"

Algorithm converges in 44 steps.



Cost of link xy changes

node x table

from \ cost to	x	y	z
x	0	51	50
y	48	0	1
z	49	1	0

node x table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	49	1	0

node x table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	50	1	0

node x table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	49	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	49	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	48	0	1
z	49	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	50	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	50	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

"bad news travels slow"

Algorithm converges in 44 steps.

Cost of link xy changes

node x table

from \ cost to	x	y	z
x	0	51	50
y	48	0	1
z	49	1	0

node x table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	49	1	0

node x table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	50	1	0

node x table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	49	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	49	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node y table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	48	0	1
z	49	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	50	1	0

node z table

from \ cost to	x	y	z
x	0	51	50
y	50	0	1
z	50	1	0

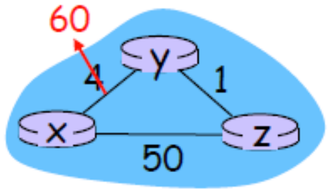
node z table

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

How to solve count to infinity problem?

Poisoned reverse:

- If Z routes through Y to get to X:
- Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- Will this completely solve count to infinity problem?



node x table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

node y table

from \ cost to	x	y	z
x	0	4	∞
y	4	0	1
z	∞	1	0

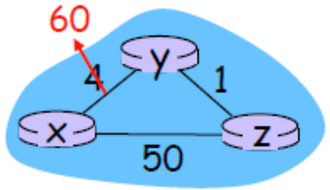
node z table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

如何解决计数到无穷问题?

中毒反向:

- 如果 Z 通过 Y 路由到 X:
- Z 告知 Y 其（Z 的）到 X 的距离是无限（因此 Y 不会通过 Z 路由到 X)
- 这能完全解决计数到无穷问题吗?



node x table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

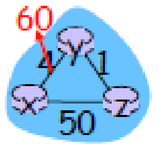
node y table

from \ cost to	x	y	z
x	0	4	∞
y	4	0	1
z	∞	1	0

node z table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

Algorithm converges in 3 steps.



Cost of link xy changes to 60

node x table

from \ cost to	x	y	z
x	0	51	50
y	4	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	60	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	60	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node y table

from \ cost to	x	y	z
x	0	4	∞
y	60	0	1
z	∞	1	0

from \ cost to	x	y	z
x	0	51	50
y	60	0	1
z	∞	1	0

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node z table

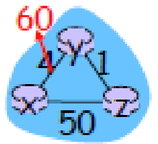
from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	∞	50
y	60	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	∞	50
y	60	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	∞	50
y	∞	0	1
z	50	1	0

Algorithm converges in 3 steps.



Cost of link xy changes to 60

node x table

from \ cost to	x	y	z
x	0	51	50
y	4	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	60	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	51	50
y	60	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node y table

from \ cost to	x	y	z
x	0	4	∞
y	60	0	1
z	∞	1	0

from \ cost to	x	y	z
x	0	51	50
y	60	0	1
z	∞	1	0

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

node z table

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

from \ cost to	x	y	z
x	0	∞	50
y	60	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	∞	50
y	60	0	1
z	50	1	0

from \ cost to	x	y	z
x	0	∞	50
y	∞	0	1
z	50	1	0

Comparison of LS and DV algorithms

Message complexity
- LS: with n nodes, E links, O(nE) msgs sent
- DV: exchange between neighbors only
  - Convergence time varies

Speed of convergence
- LS: $O(n^2)$ algorithm requires O(nE) msgs
  - may have oscillations
- DV:convergence time varies
  - may be routing loops
  - count-to-infinity problem

Robustness: what happens if router malfunctions?

LS:
- Node can advertise incorrect link cost
- Each node computes only its own table

DV:
- DV node can advertise incorrect path cost
- Each node's table used by others
- Error propagate thru network

LS和DV算法的比较

消息复杂度
- LS: 对于n个节点、E条链路，发送O(nE)条消息
- DV: 仅在邻居之间交换
  - 收敛时间变化

收敛速度
- LS: $O(n^2)$ 算法需要 O(nE) 消息 可能会有振荡 DV: 收敛时间变化 可能会有路由环路 计数到无穷问题

鲁棒性： 如果路由器故障会怎样?

LS:
- 节点可以发布不正确的链路成本 每个节点仅计算自己的表Dv: Dv节点可以发布不正确的路径成本 每个节点的表被其他节点使用 错误通过网络传播

Hierarchical Routing

Our routing study thus far - idealization

- all routers identical
- network "flat"

… *not* true in practice

scale: with billions of destinations:

- can't store all dest's in routing tables!
- routing table exchange would swamp links!

administrative autonomy

- internet = network of networks
- each network admin may want to control routing in its own network

分层路由

我们目前的路由研究——理想化

- 所有路由器都相同
- 网络"扁平"… 并非

实际如此

缩放：与数十亿个目的地址：

- 无法在路由表中存储所有目的地的信息！
- 路由表交换将使链路不堪重负！

行政自主权

- 互联网 = 网络中的网络
- 每个网络管理员可能都希望控制自己网络中的路由

Hierarchical Routing

- aggregate routers into regions, "autonomous systems" (AS)
- routers in same AS run same routing protocol
  - "intra-AS" routing protocol
  - routers in different AS can run different intra-AS routing protocol

gateway routers

- special routers in AS
- run intra-AS routing protocol with all other routers in AS
- *also* responsible for routing to destinations outside AS
  - run *inter-AS routing* protocol with other gateway routers

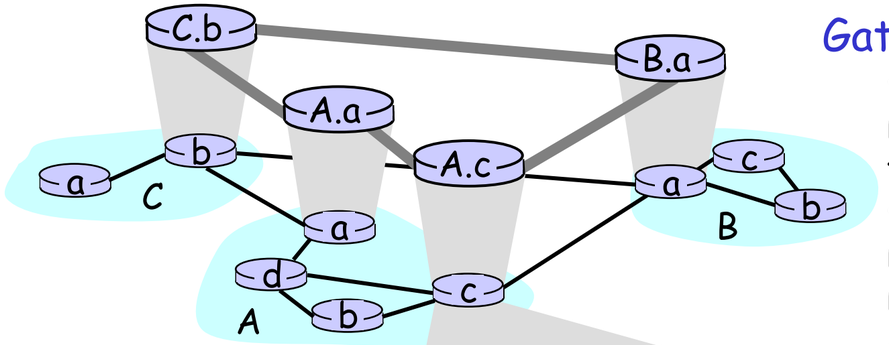
分层路由

- 将路由器聚合为 区域，"自治系统" （AS）
- 同一自治系统中的路由器运行相同路由协议
  - "域内"-路由协议 不同自治系统中的路由器可以运行不同的域内路由协议

网关路由器

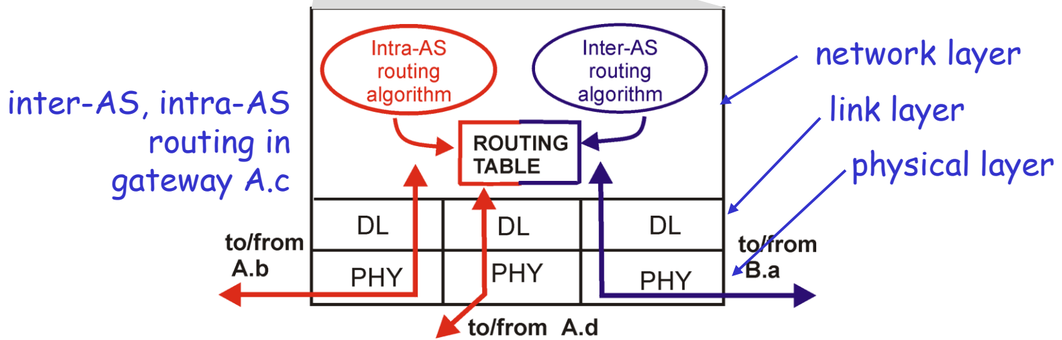
- 自治系统 中的特殊路由器 域内路由 协议，与自治系统 中所有其他路由器一起，也负责路由到自治系统外的目的地
  - 与其他网关路由器运行 域间路由 协议

Intra-AS and Inter-AS routing

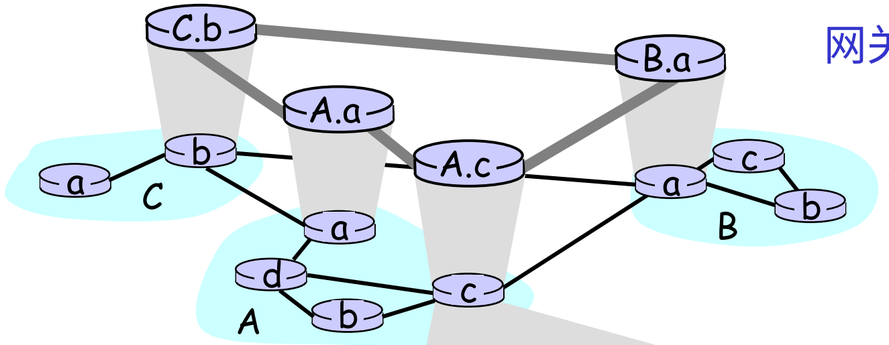


Gateways:

- perform inter-AS routing amongst themselves
- perform intra-AS routers with other routers in their AS

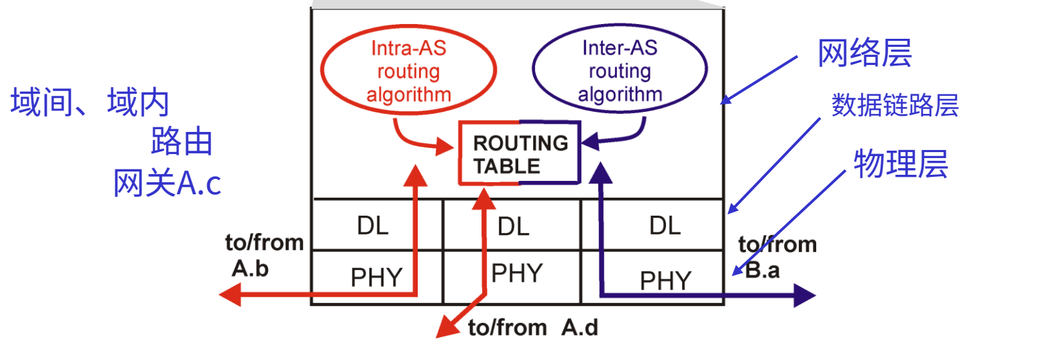


域内-自治系统与域间路由

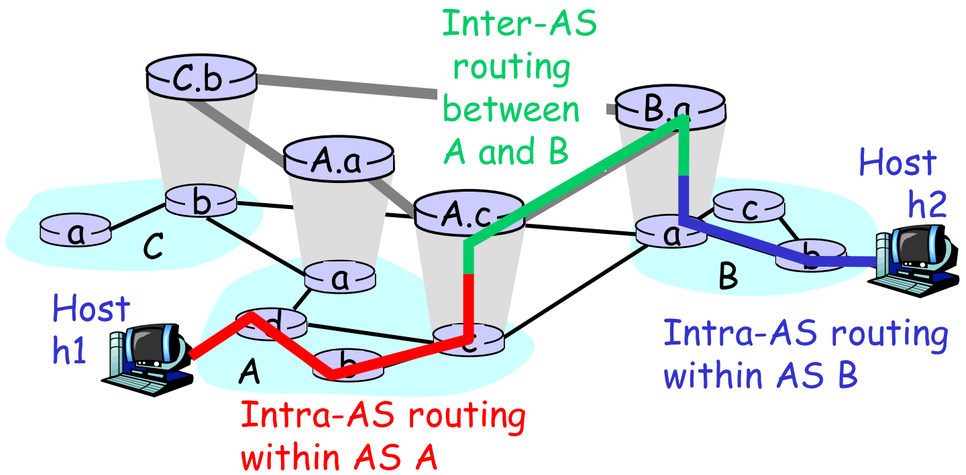


网关：

- 执行域间路由彼此之间
- 执行域内与其他路由器在其域内AS

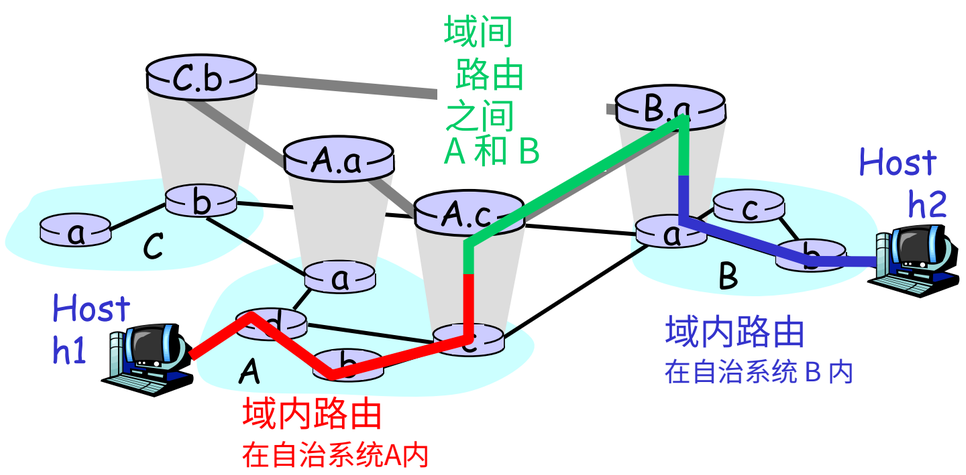


Intra-AS and Inter-AS routing



- We'll examine specific inter-AS and intra-AS Internet routing protocols shortly

域内-自治系统与域间路由



- 我们将很快研究具体的—自治系统间和自治系统内互联网路由协议

Routing in the Internet

- The Global Internet consists of Autonomous Systems (AS) interconnected with each other:
  - **Stub AS**: small corporation
  - **Multihomed AS**: large corporation (no transit)
  - **Transit AS**: provider

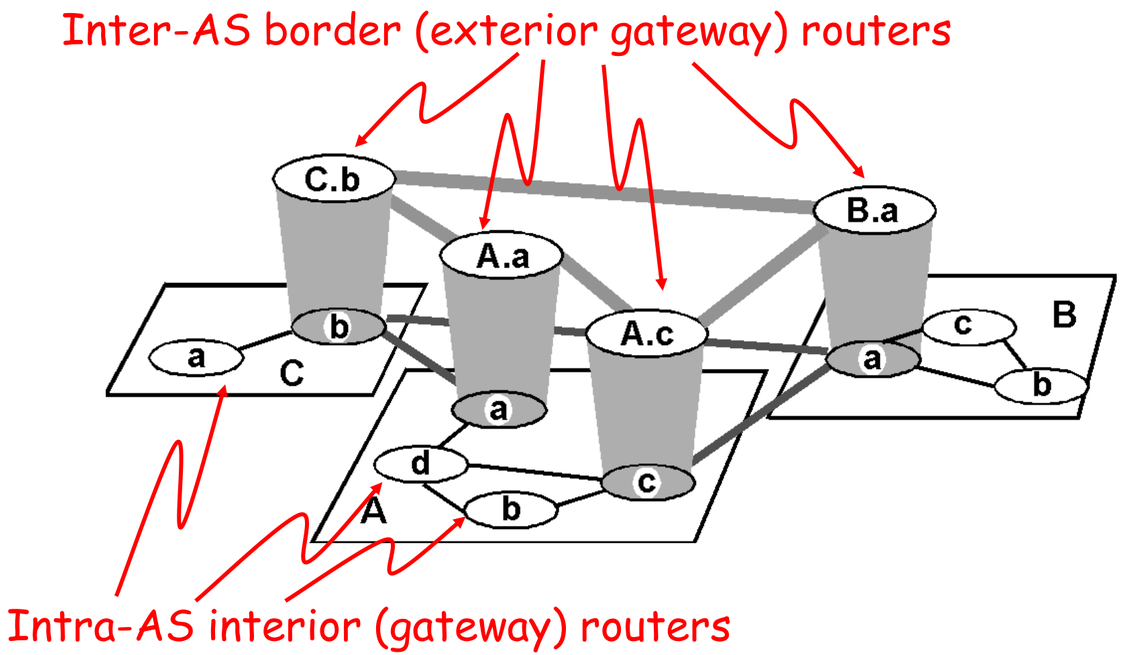
- Two-level routing:
  - **Intra-AS**: administrator is responsible for choice
  - **Inter-AS**: unique standard

互联网中的路由

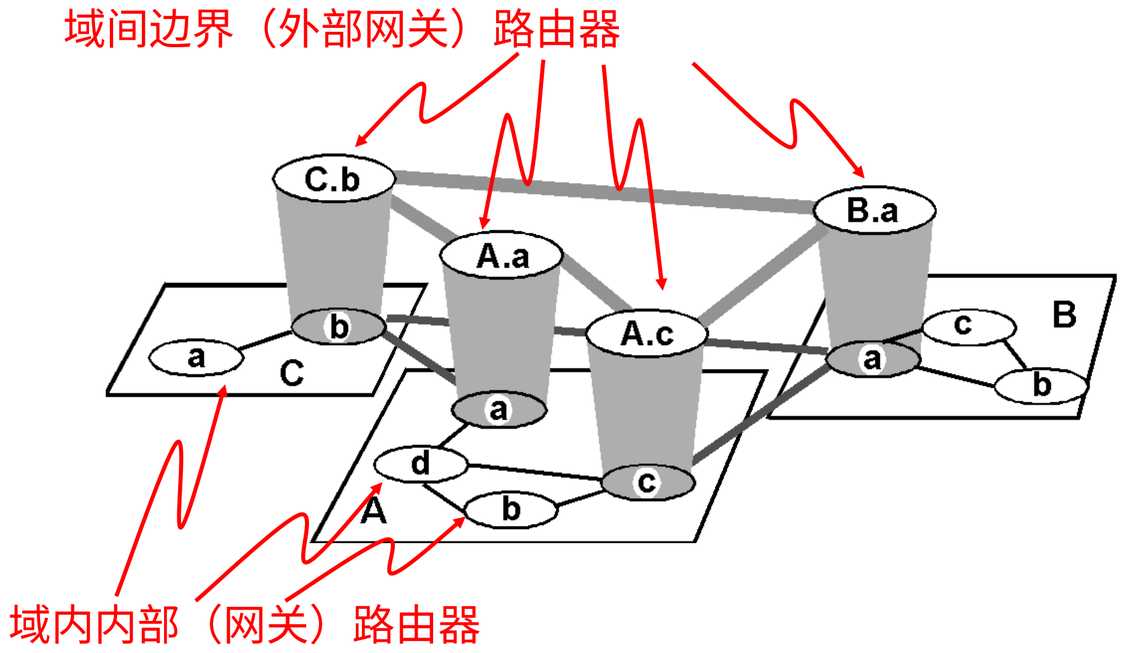
- 全球互联网由自治系统(AS)相互连接而成：
  - **小型企业**：小型公司 **多宿主企业**：大型公司（无中转） **中转企业**：服务提供商

- 双层路由：
  - **域内**：管理员负责选择 **域间**：唯一标准

Internet AS Hierarchy



互联网自治系统层次结构



Chapter 5: Network Layer

- 5.1 Introduction
- 5.2 Virtual circuit and datagram networks
- 5.3 What's inside a router?
- 5.4 Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector
  - Hierarchical routing
- 5.5 Routing in the Internet
- 5.6 IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6

5: 网络层

- 5.1 简介
- 5.2 虚拟电路与数据报网络
- 5.3 路由器里有什么?
- 5.4 路由算法:
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量 分层路由
- 5.5 互联网中的路由
- 5.6 IP: 互联网协议
  - IPv4数据报格式 IPv4寻址 IP分片 网络地址转换 ARP ICMP IPv6

Intra-AS Routing

- Also known as Interior Gateway Protocols (IGP)
- Most common intra-AS routing protocols:
  - RIP: Routing Information Protocol
  - OSPF: Open Shortest Path First
  - IGRP: Interior Gateway Routing Protocol (Cisco proprietary for decades, until 2016)

域内-自治系统路由

- 也称为 内部网关协议 (IGP)
- 最常见的域内路由协议：
  - RIP: 路由信息协议
  - OSPF: 开放最短路径优先
  - IGRP: 内部网关路由协议（思科专有，持续数十年，直至2016年）

RIP (Routing Information Protocol)

- Distance vector algorithm
- Included in BSD-UNIX Distribution in 1982
- Distance metric: # of hops (max = 15 hops)
  - *Can you guess why?*

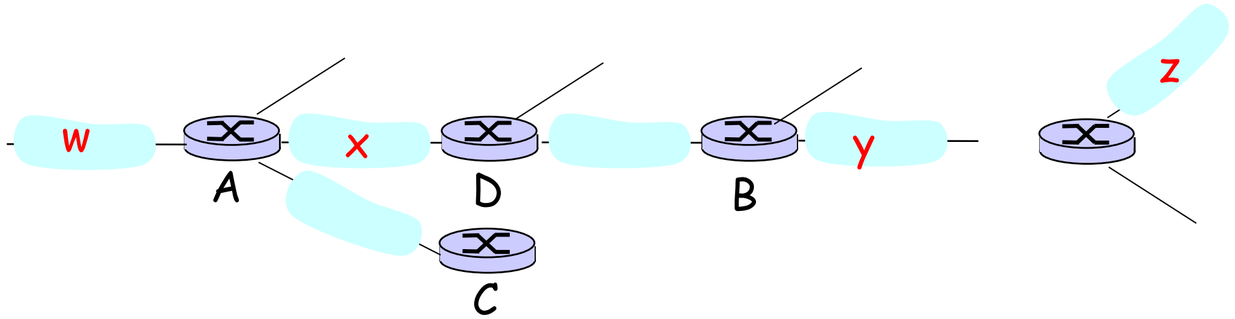
- Distance vectors: exchanged every 30 sec via Response Message (also called **advertisement**)
- Each advertisement: route to up to 25 destination nets

RIP（路由信息协议）

- 距离向量算法- 1982年包含在BSD-UNIX发行版中- 距离度量：跳数（最大= 15 跳数）
  - 你能猜到为什么吗？

- 距离向量：通过响应消息（也称为<样式 id='2'>发布</样式>）每30秒交换
- 每个发布：最多包含25个目的网络的路由

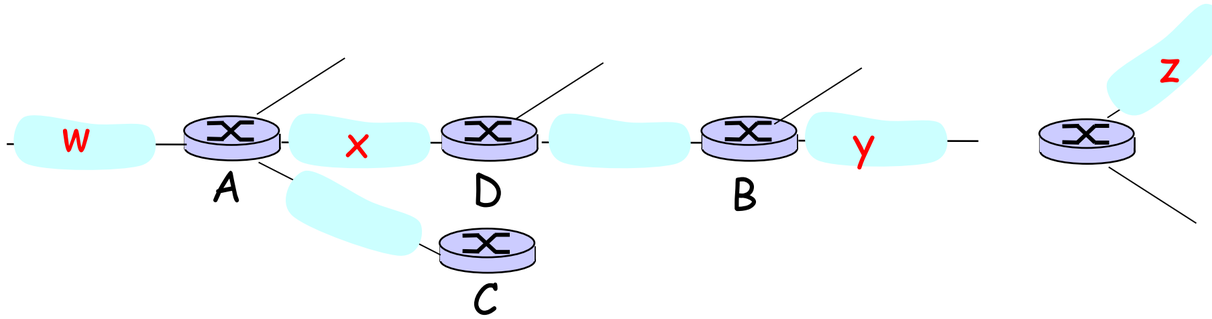
RIP (Routing Information Protocol)



Destination Network	Next Router	Num. of hops to dest.
w	A	2
y	B	2
z	B	7
x	--	1
….	….	....

Routing table in D

RIP (路由信息协议)



目标网络	下一跳路由器	到目标网络跳数
w	A	2
y	B	2
z	B	7
x	--	1
….	….	....

D中的路由表

RIP: Link Failure and Recovery

If no advertisement heard after 180 sec --> neighbor/link declared dead

- routes via neighbor invalidated
- new advertisements sent to neighbors
- neighbors in turn send out new advertisements (if tables changed)
- link failure info quickly propagates to entire net
- poison reverse used to prevent ping-pong loops (infinite distance = 16 hops)

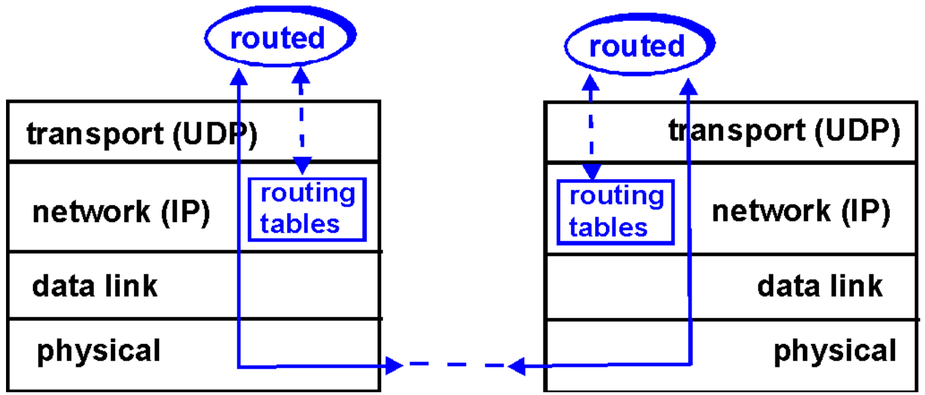
RIP：链路故障与恢复

如果在180秒内未收到任何发布信息——>邻居/链路被宣告为死亡

- 通过邻居的路径失效
- 向邻居发送新的发布信息
- 邻居依次发送新的发布信息（如果表发生变化）
- 链路故障信息迅速传播到整个网络
- poison reverse 用于防止 ping-pong 循环（无限距离 = 16 跳）

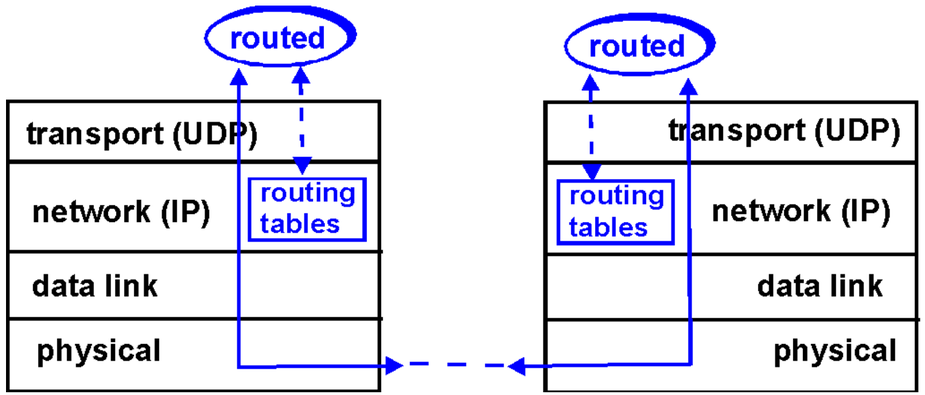
RIP Table processing

- RIP routing tables managed by **application-level** process called *route-d (daemon)*
- advertisements sent in UDP packets, periodically repeated



RIP 表处理

- RIP 路由表由称为 **应用层** 的进程 route-d (守护进程) 管理
- 通过<code>UDP</code>数据包发送的发布信息，会周期性地重复



RIP Table example (continued)

Router: *giroflee.eurocom.fr*

Destination	Gateway	Flags	Ref	Use	Interface
127.0.0.1	127.0.0.1	UH	0	26492	lo0
192.168.2.	192.168.2.5	U	2	13	fa0
193.55.114.	193.55.114.6	U	3	58503	le0
192.168.3.	192.168.3.5	U	2	25	qaa0
224.0.0.0	193.55.114.6	U	3	0	le0
default	193.55.114.129	UG	0	143454	

- Three attached class C networks (LANs)
- Router only knows routes to attached LANs
- Default router used to "go up"
- Route multicast address: 224.0.0.0
- Loopback interface (for debugging)

RIP 表示例（续）

路由器： giroflee.eurocom.fr

目的网关 标志 引用 使用 接口

127.0.0.1 127.0.0.1 UH 0 26492 lo0
192.168.2. 192.168.2.5 U 2 13 fa0
193.55.114. 193.55.114.6 U 3 58503 le0
192.168.3. 192.168.3.5 U 2 25 qaa0
224.0.0.0 193.55.114.6 U 3 0 le0
默认 193.55.114.129 UG 0 143454

- 三个连接的C类网络（局域网）
- 路由器仅知道连接的局域网的路由
- 默认路由器用于“向上”连接
- 路由组播地址：224.0.0.0
- 回环接口（用于调试）

OSPF (Open Shortest Path First)

- "open": publicly available
- Uses Link -State algorithm
  - LS packet dissemination
  - Topology map at each node
  - Route computation using Dijkstra's algorithm
- OSPF advertisements are disseminated to entire AS (via flooding)
- Link state advertisements are periodically updated
- OSPF advertisements in OSPF messages are carried directly by IP

OSPF (开放最短路径优先)

- "open"：公开可用
- 使用 链路-状态 算法
  - 链路状态分组传播 每个节点的拓扑图 使用迪杰斯特拉算法进行路径计算
- OSPF通告会传播到 整个自治系统（通过泛洪）
- 链路状态通告会周期性更新
- OSPF消息中的OSPF发布由直接由IP承载

OSPF "advanced" features (not in RIP)

- Security: all OSPF messages authenticated (to prevent malicious intrusion); TCP connections used
- Multiple same-cost paths allowed (only one path in RIP)
- For each link, multiple cost metrics for different TOS (eg, satellite link cost set "low" for best effort; high for real time)
- Integrated uni- and multicast support:
  - Multicast OSPF (MOSPF) uses same topology data base as OSPF
- Hierarchical OSPF in large domains.

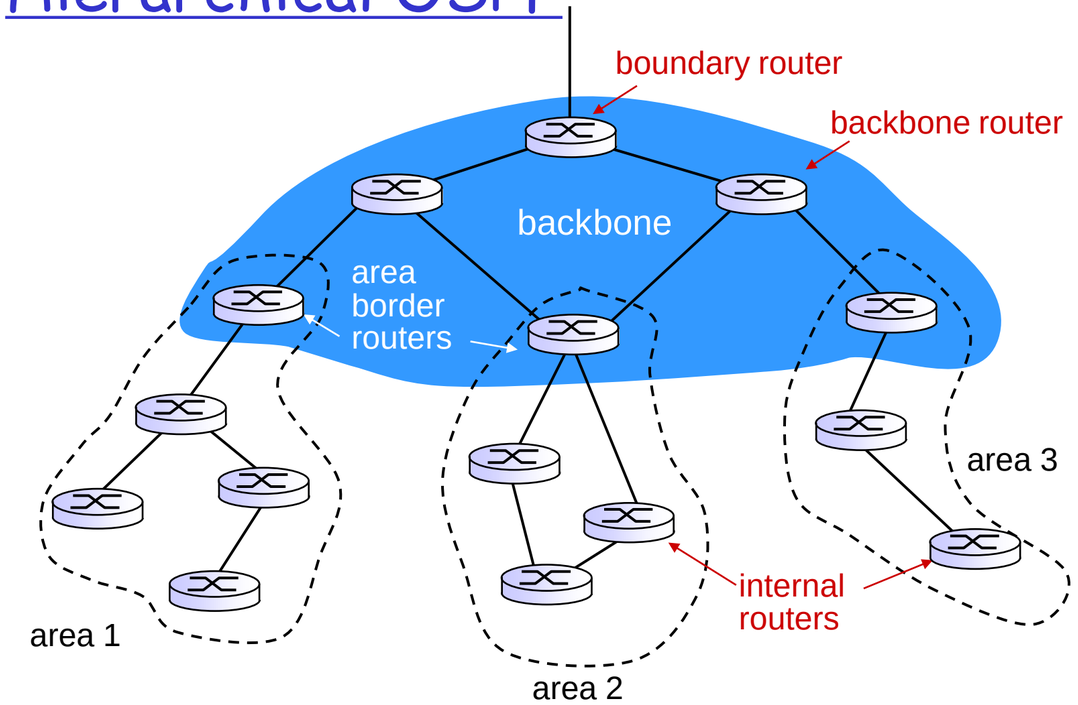
OSPF "高级"功能（RIP中不存在）

- 安全：所有OSPF消息都经过认证（以防止恶意入侵）；使用TCP连接
- 允许多条相同成本路径（RIP中仅允许一条路径）

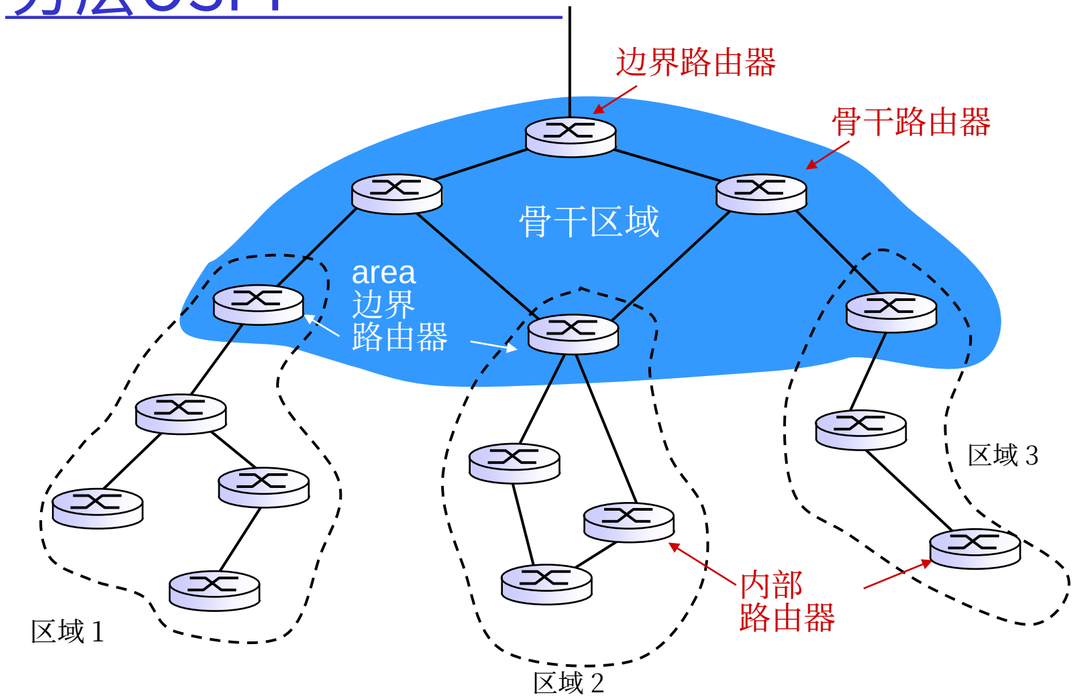
- 对于每条链路，针对不同 TOS （例如，卫星链路成本设置为"低"以实现尽力而为； 高优先级用于实时传输）可配置多个成本指标
- 集成单播和 组播支持：
  - 组播OSPF（MOSPF）使用与OSPF相同的拓扑数据库

- 大型域中的分层OSPF

Hierarchical OSPF



分层OSPF



Hierarchical OSPF

- Two-level hierarchy: local area, backbone.
  - Link-state advertisements only in area
  - each nodes has detailed area topology; only know direction (shortest path) to nets in other areas.
- **Area border routers:** "summarize" distances to nets in own area, advertise to other Area Border routers.
- **Backbone routers:** run OSPF routing limited to backbone.
- **Boundary routers:** connect to other ASs.

分层OSPF

- 两级层次结构：本地区域，骨干区域。
  - 仅在区域内发布链路状态通告；每个节点拥有详细的区域拓扑；仅知道到其他区域网络的方向（最短路径）。
- **区域边界路由器：** "汇总"到本区域网络的距离，向其他区域边界路由器发布。
- **骨干路由器：** 运行仅限于骨干区域的OSPF路由。
- **边界路由器：** 连接到其他自治系统。

IGRP (Interior Gateway Routing Protocol)

- CISCO proprietary; successor of RIP (mid 80s)
- Distance Vector, like RIP
- several cost metrics (delay, bandwidth, reliability, load etc)
- uses TCP to exchange routing updates
- Loop-free routing via Distributed Updating Alg. (DUAL) based on *diffused computation*

IGRP（内部网关路由协议）

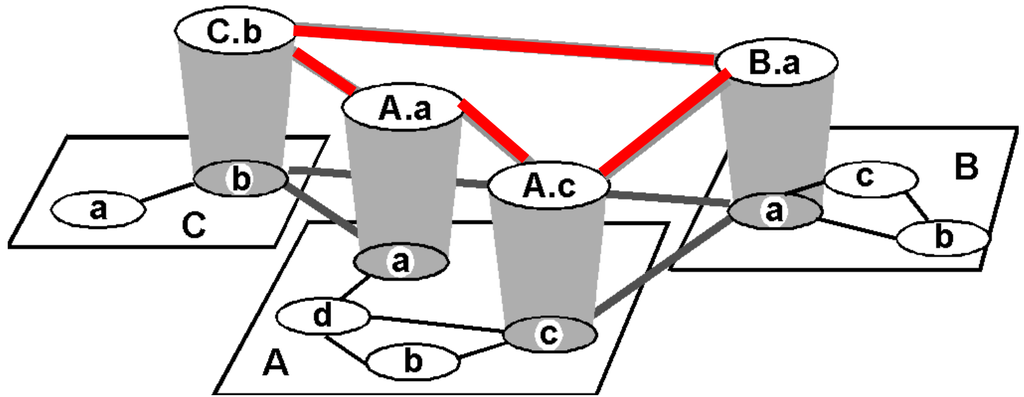
- 思科专有；RIP的继任者（80年代中期）　　）
- 距离向量，如RIP❒

  多个成本指标（延迟、带宽、可靠性、负载等）

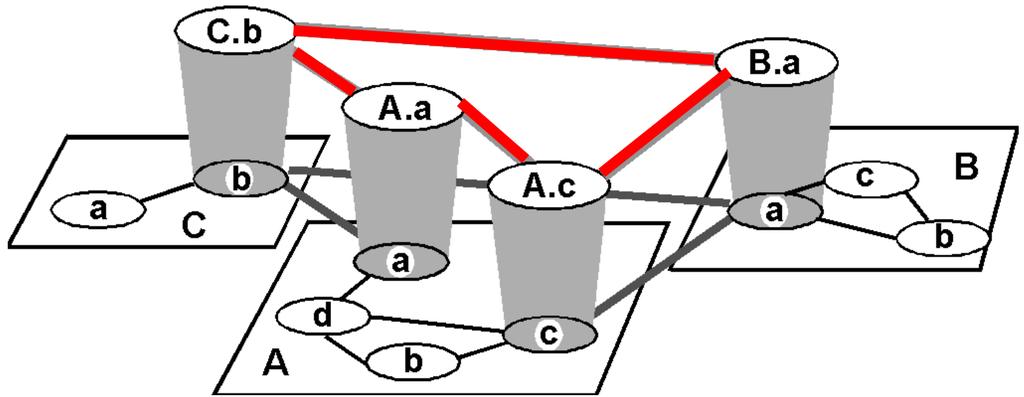
- 使用TCP交换路由更新
- 通过分布式更新算法（DUAL）实现无环路路由，基于

<样式 id='2'>扩散计算</样式>

Inter-AS routing



互联-自治系统路由



Internet inter-AS routing: BGP

- BGP (Border Gateway Protocol): *the* de facto inter-domain routing protocol
- **Path Vector** protocol:
  - similar to Distance Vector protocol
  - each Border Gateway broadcast to neighbors (peers) *entire path* (I.e, sequence of ASs) to destination
  - E.g., Gateway X may send its path to dest. Z:

Path (X,Z) = X,Y1,Y2,Y3,…,Z

互联网间-自治系统路由：BGP

- BGP（边界网关协议）：事实上的域间路由协议
- **路径向量**协议：
  - 类似于距离向量协议
  - 每个边界网关向邻居（对等体）广播到目的地的整个路径（即，自治系统序列）
  - 例如，网关X可能会将其路径发送到目的Z：

Path (X,Z) = X,Y1,Y2,Y3,…,Z

Internet inter-AS routing: BGP

- BGP (Border Gateway Protocol): *the* de facto standard
- BGP provides each AS a means to:
  - Obtain subnet reachability information from neighboring ASs.
  - Propagate reachability information to all AS-internal routers.
  - Determine good routes to subnets based on reachability information and policy.
- Allows subnet to advertise its existence to the rest of Internet: "I am here"

互联网间-自治系统路由：BGP

- BGP（边界网关协议）：事实上的标准
- BGP 为每个自治系统提供一种方法：
  - 获取来自相邻子网的可达性信息：ng
    自治系统。 向所有自治系统内部路由器传播可达性信息。 根据可达性信息及策略确定到子网的良好路由
    ty
    信息和政策。
- 允许子网向互联网发布其存在信息：“我在这里”其余：“我在这里”

BGP route selection

- Router may learn about more than 1 route to some prefix. Router must select route
- Elimination rules:
  - Local preference value attribute: policy decision
  - Shortest AS-PATH
  - Closest NEXT-HOP router: hot potato routing
  - Additional criteria

BGP 路由选择

- 路由器可能了解到某个前缀有多条路由。路由器必须选择路由
- 消除规则：
  - 本地优先级值属性：策略决策 最短AS-PATH 最近下一跳路由器：热土豆路由 附加标准

Internet inter-AS routing: BGP

Suppose: gateway X send its path to peer gateway W

- W may or may not select path offered by X
  - cost, policy (don't route via competitors AS), loop prevention reasons.
- If W selects path advertised by X, then:

  Path (W,Z) = w, Path (X,Z)
- Note: X can control incoming traffic by controling it route advertisements to peers:
  - e.g., don't want to route traffic to Z -> don't advertise any routes to Z

互联网间-自治系统路由：BGP

假设：网关X将其路径发送给对等网关W

- W可能选择X提供的路径，也可能不选择
  - 成本、策略（不通过竞争对手的自治系统路由）、环路p预防原因。
- 如果W选择X发布的路径，则：

  Path (W,Z) = w, Path (X,Z)
- 注意：X可以通过控制向对等方发布的路由信息来控制传入流量
  - 例如，不想将流量路由到Z -> 不想 发布任何路由到Z

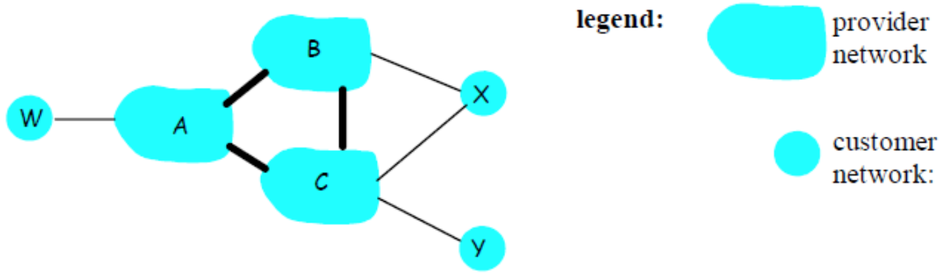
Internet inter-AS routing: BGP

- BGP messages exchanged using TCP.
- BGP messages:
  - OPEN: opens TCP connection to peer and authenticates sender
  - UPDATE: advertises new path (or withdraws old)
  - KEEPALIVE keeps connection alive in absence of UPDATES; also ACKs OPEN request
  - NOTIFICATION: reports errors in previous msg; also used to close connection

互联网间-自治系统路由：BGP

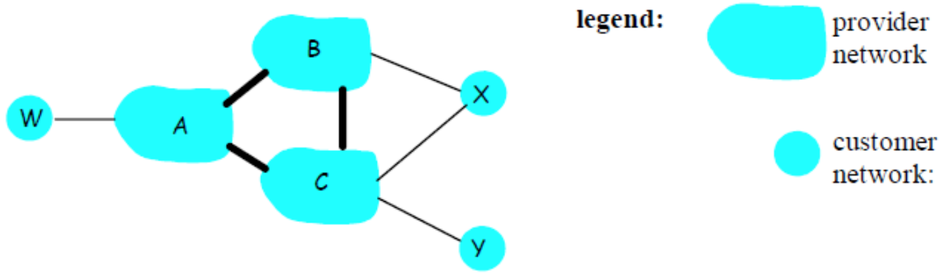
- 使用 TCP的BGP消息。
- BGP消息：
  - OPEN： 打开到对端的TCP连接并验证发送方
  - UPDATE： 发布新路径（或撤回旧路径）
  - KEEPALIVE 在没有UPDATE的情况下保持连接活跃；也确认OPEN请求
  - NOTIFICATION: 报告前一条消息中的错误；也用于关闭连接

BGP Routing Policy



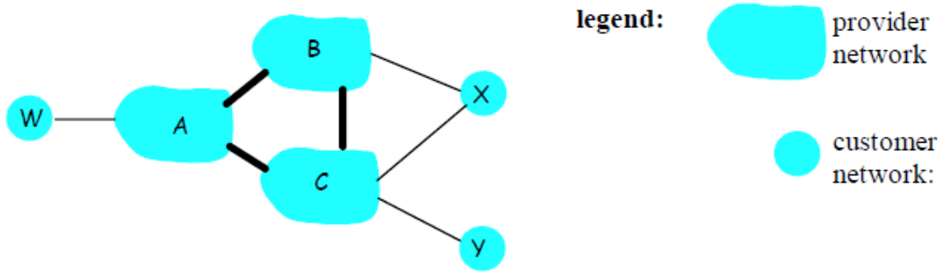
- A,B,C are provider networks
- X,W,Y are customer (of provider networks)
- X is dual-homed: attached to two networks
  - X does not want to route from B via X to C
  - …so X will not advertise to B a route to C

BGP路由策略



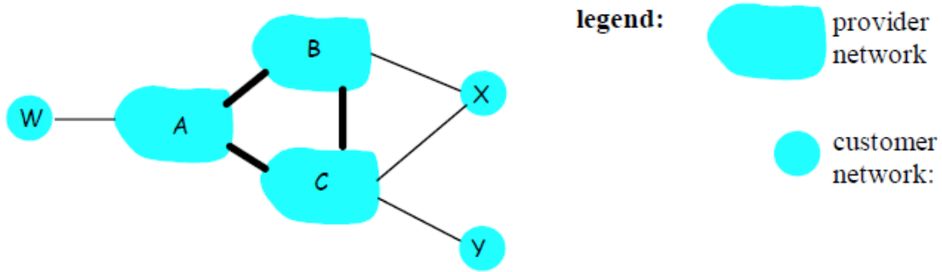
- A、B、C 是 提供者网络
- X、W、Y是客户（属于提供者网络）
- X是双归属：连接到两个网络
  - X不希望从B通过X路由到C
  - …所以X不会向B发布到C的路由

BGP Routing Policy



- A advertises to B the path AW
- B advertises to X the path BAW
- Should B advertise to C the path BAW?
  - No way! B gets no "revenue" for routing CBAW since neither W nor C are B's customers
  - B wants to force C to route to W via A
  - B wants to route only to/from its customers!

BGP路由策略



- A向B发布路径AW
- B向X发布路径BAW
- B应该向C发布路径BAW吗?
  - 不可能！B为路由CBAW没有得到“收入”，因为W和C都不是B的客户
  - B想迫使C通过A路由到W
  - B 想要为 仅 其客户进行路由！

Why different Intra- and Inter-AS routing ?

Policy:

- Inter-AS: admin wants control over how its traffic routed, who routes through its net.
- Intra-AS: single admin, so no policy decisions needed

Scale:

- hierarchical routing saves table size, reduced update traffic

Performance:

- Intra-AS: can focus on performance
- Inter-AS: policy may dominate over performance

为什么域内- 和域间路由?

策略：

- 域间：管理员希望控制其流量如何路由，谁通过其网络进行路由。
- 域内：单一管理员，因此无需策略决策

规模：

- 分层路由节省表大小，减少更新流量

性能：

- 域内：可以专注于性能
- 域间：策略可能优先于性能

Exercises-1

1.The protocols for directly encapsulating RIP、OSPF and BGP messages are

A. TCP、UDP、IP
B. TCP、IP、UDP
C. UDP、TCP、IP
D. UDP、IP、TCP

练习-1

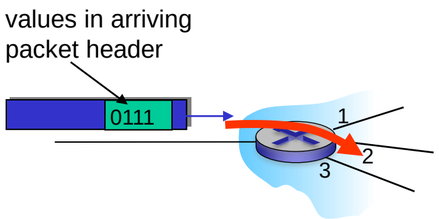
1.直接封装RIP、OSPF和BGP消息的协议是

A. TCP、UDP、IP
B. TCP、IP、UDP
C. UDP、TCP、IP
D. UDP、IP、TCP

*Network layer: data plane, control plane

Data plane

- local, per-router function
- determines how datagram arriving on router input port is forwarded to router output port
- forwarding function



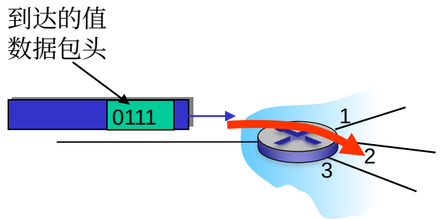
Control plane

- network-wide logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
  - *traditional routing algorithms:* implemented in routers
  - *software-defined networking (SDN):* implemented in (remote) servers

*网络层：数据平面，控制平面

数据平面

- 本地，每路由器功能
- 确定数据报如何从路由器输入端口转发到输出端口
- 转发功能

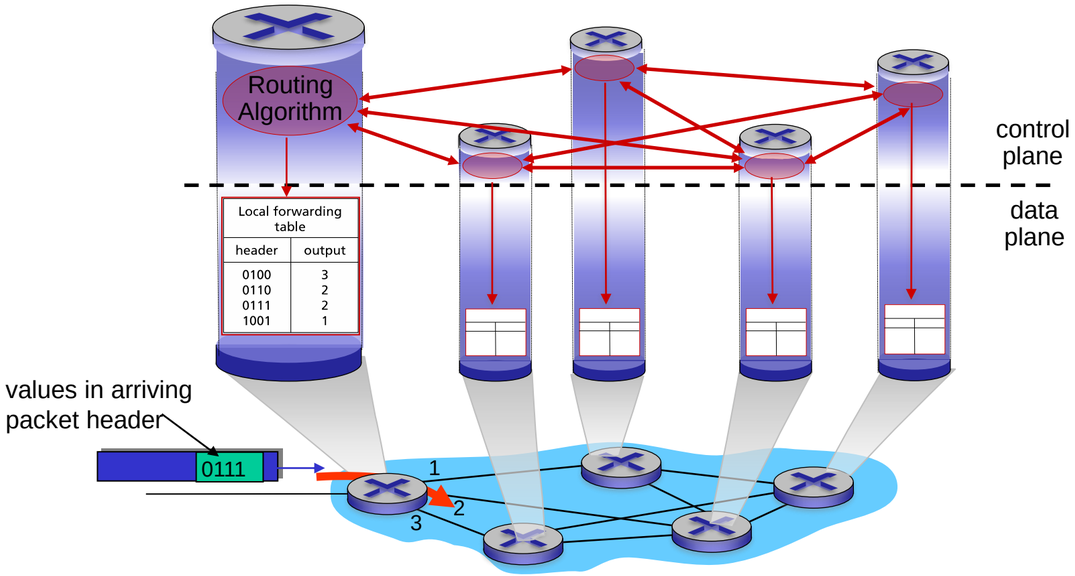


控制平面

- 全局网络逻辑
- 确定数据报如何在源主机到目的主机的端到端路径上，在路由器之间进行路由
- 两种控制平面方法：•传统路由算法：在路由器中实现•软件定义网络(SDN)：在（远程）服务器中实现

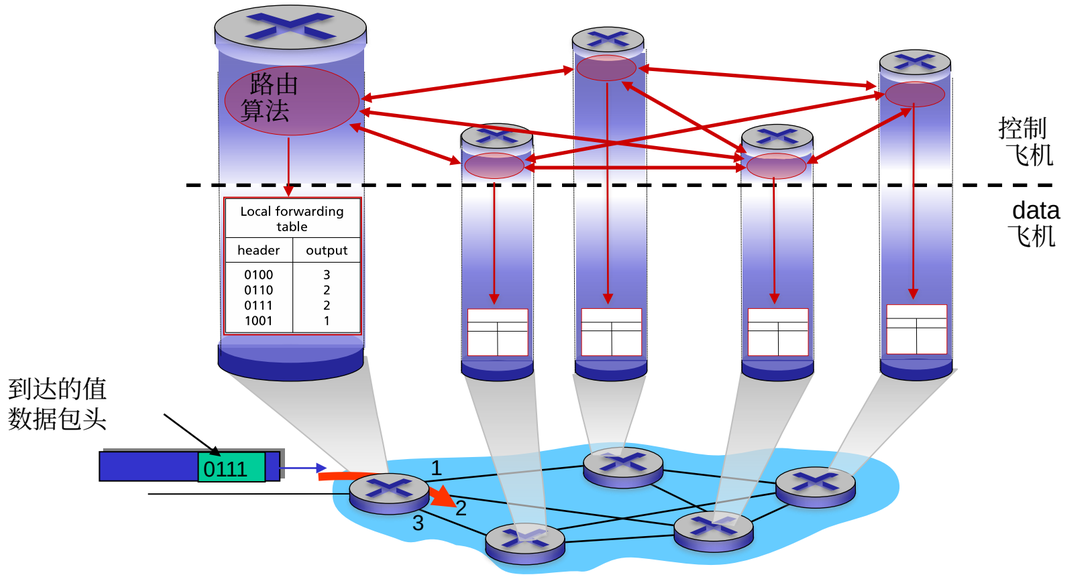
*Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane



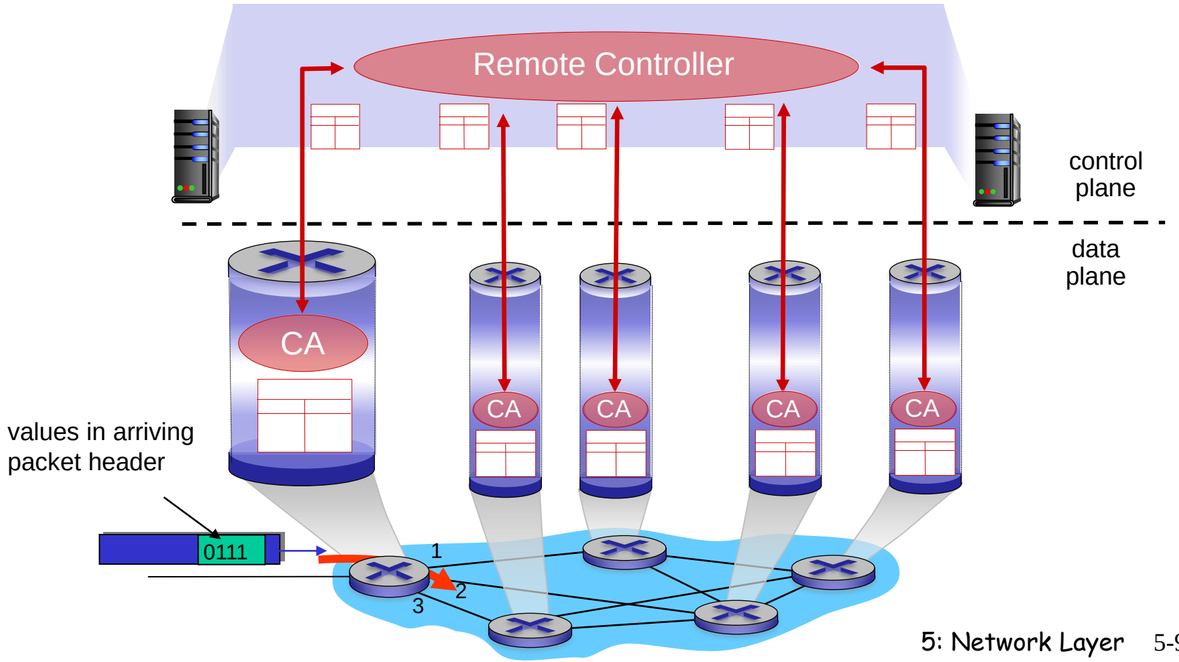
*每路由器控制平面

每个路由算法的组件在每个和每个路由器都在控制平面中交互



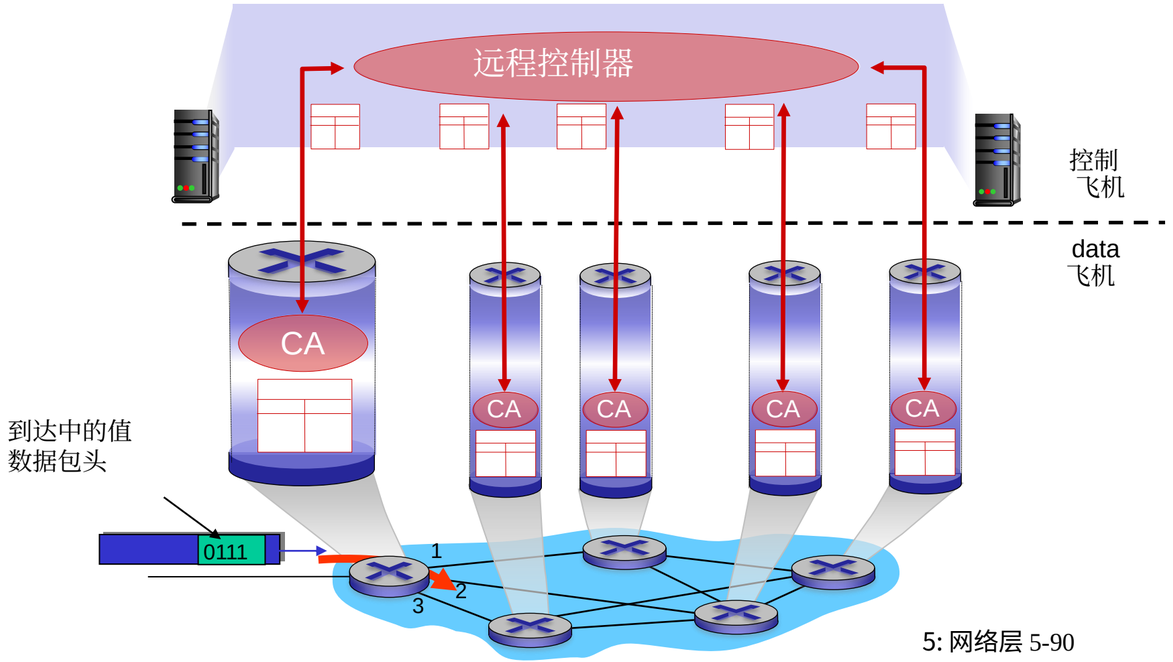
*Logically centralized control plane

A distinct (typically remote) controller interacts with local control agents (CAs)



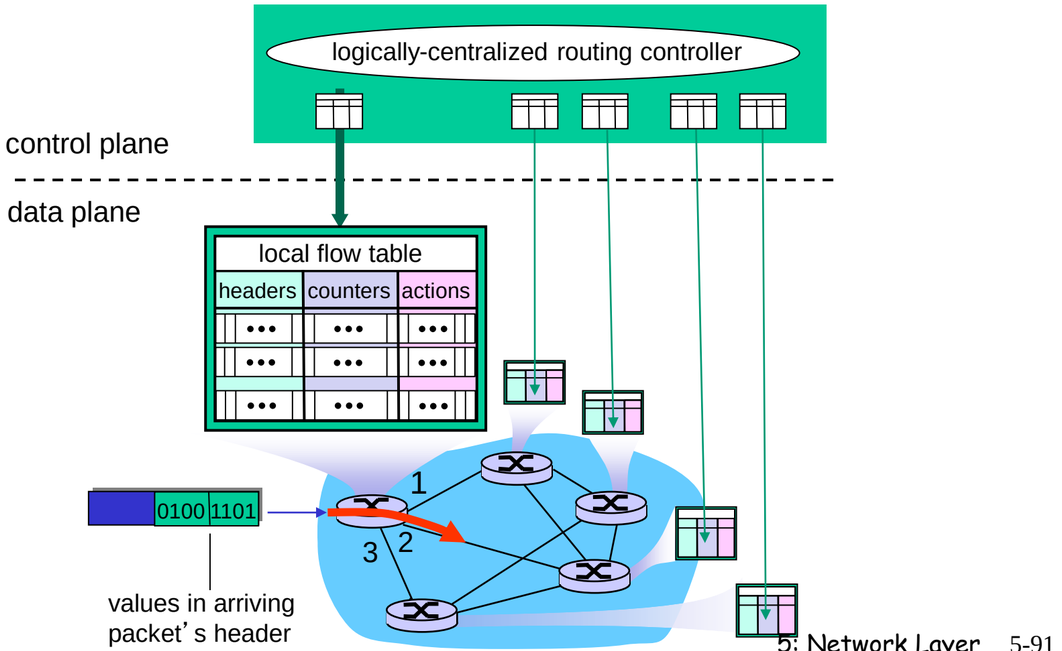
*逻辑集中式控制平面

一个独立的（通常是远程的）控制器与本地控制代理（CAs）交互



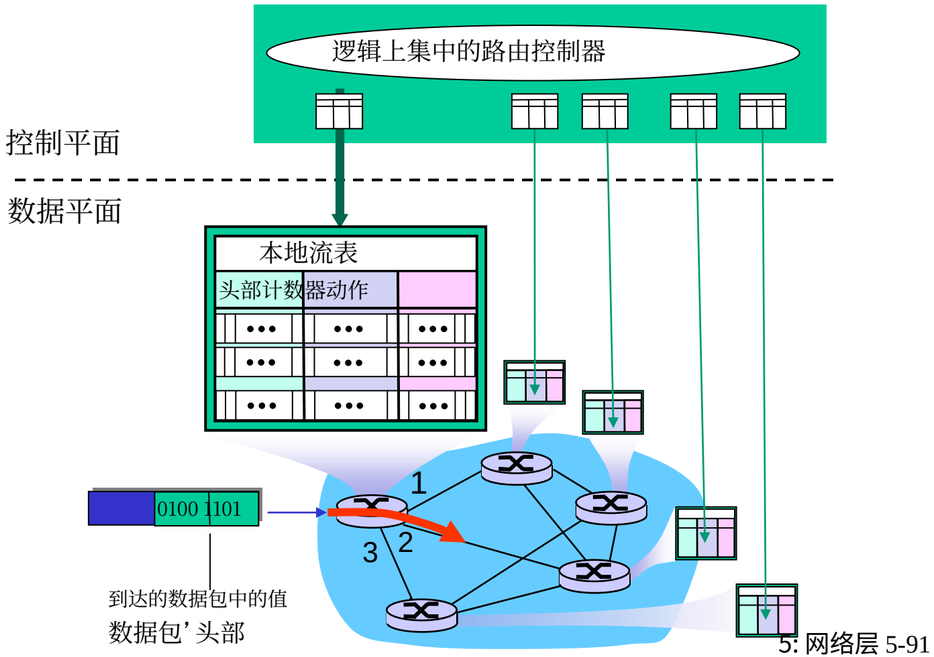
*Generalized Forwarding and SDN

Each router contains a *flow table* that is computed and distributed by a *logically centralized* routing controller

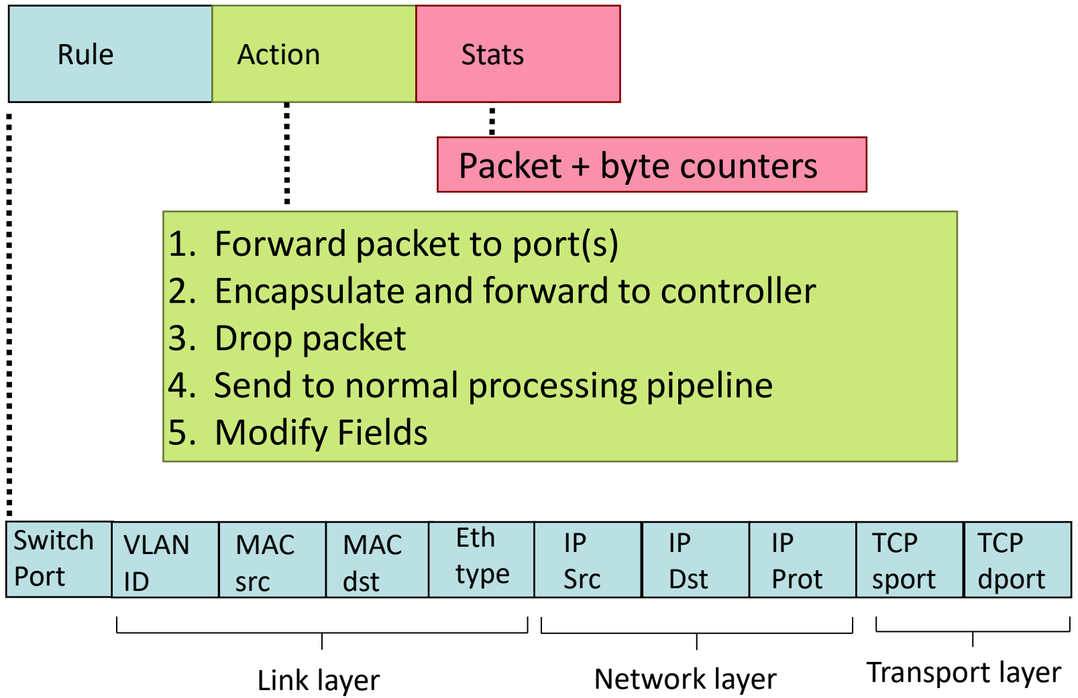


*通用转发与SDN

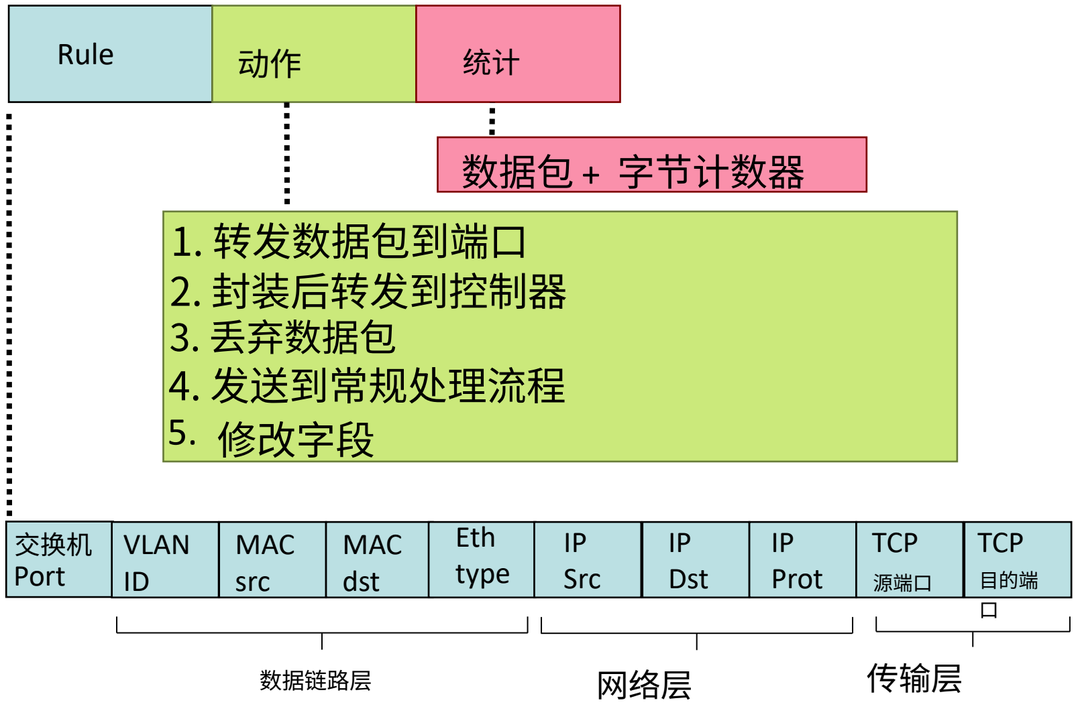
每个路由器都包含一个由逻辑上集中的路由控制器计算和分发的 流表



*OpenFlow: Flow Table Entries



*OpenFlow: 流表条目



*OpenFlow abstraction

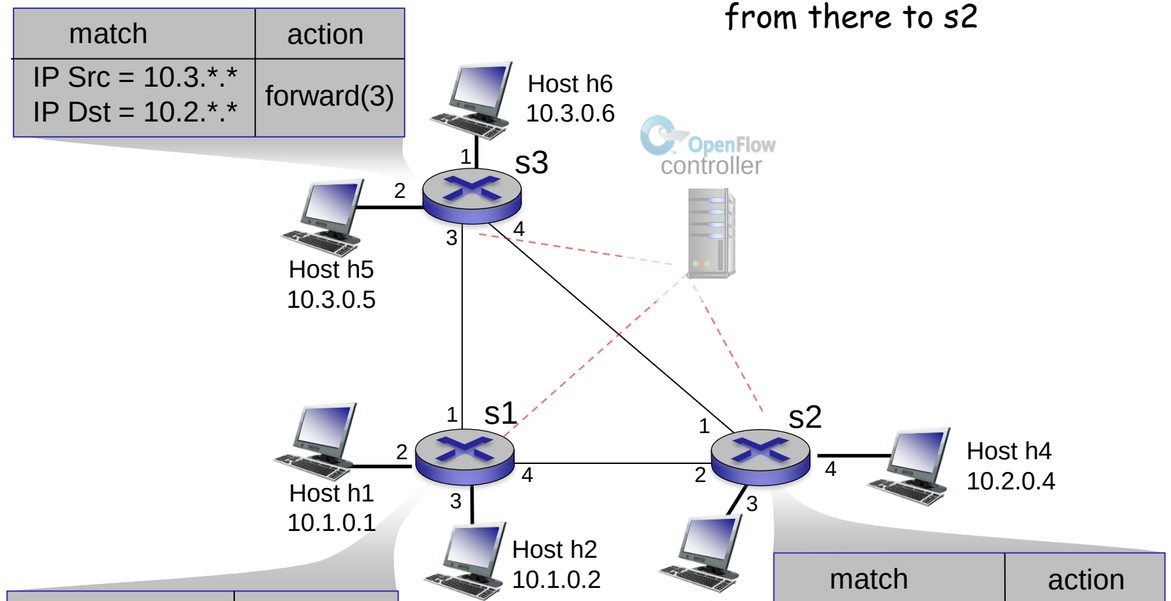
- *match+action:* unifies different kinds of devices
  - Router
    - *match:* longest destination IP prefix
    - *action:* forward out a link
  - Switch
    - *match:* destination MAC address
    - *action:* forward or flood
  - Firewall
    - *match:* IP addresses and TCP/UDP port numbers
    - *action:* permit or deny
  - NAT
    - *match:* IP address and port
    - *action:* rewrite address and port

*OpenFlow抽象

- 匹配+动作： 统一不同种类的设备
  - 路由器
    - 匹配： 最长目的IP前缀
    - 动作： 从链路转发出去
  - 交换机
    - 匹配： 目标MAC地址
    - 动作： 转发或洪泛
  - 防火墙
    - 匹配： IP地址和TCP/UDP端口编号
    - 动作：允许或拒绝
  - 网络地址转换
    - 匹配：IP地址和端口
    - 动作：重写地址和端口

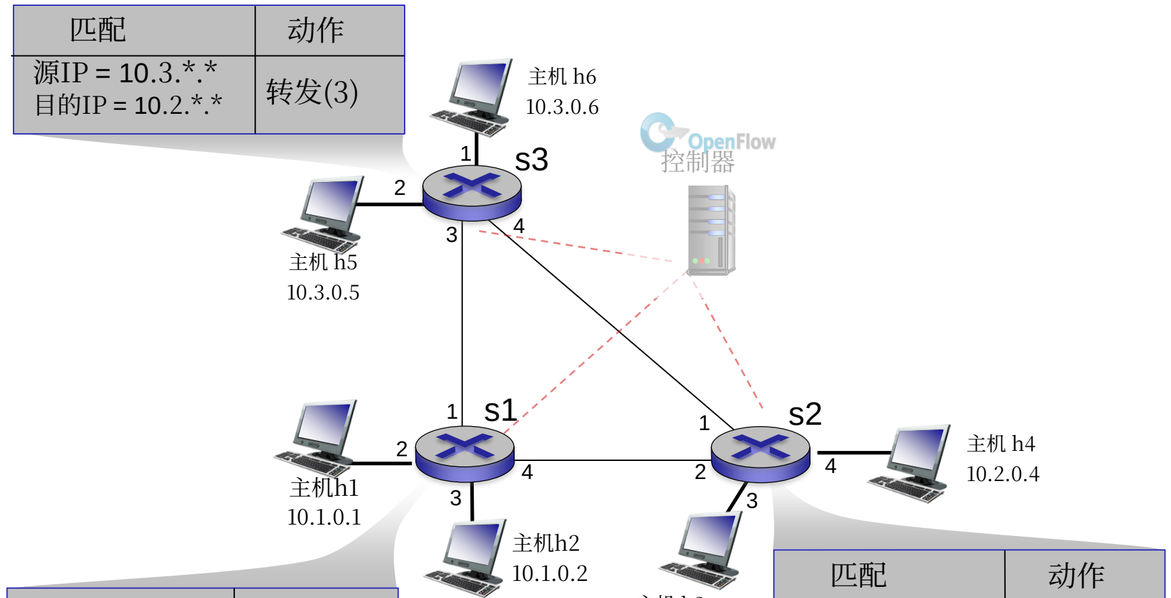
*OpenFlow example

Example: datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2



*OpenFlow示例

示例：来自主机 h5 和 h6 的数据包应通过 s1 发送到 h3 或 h4，然后从那里发送到 s2



Homework

- P455:R4,R5
- P457:P3,P5
- P458:P11
- P396:P14

作业

- P455:R4,R5
- P457:P3,P5
- P458:P11
- P396:P14

Chapter 5: Network Layer

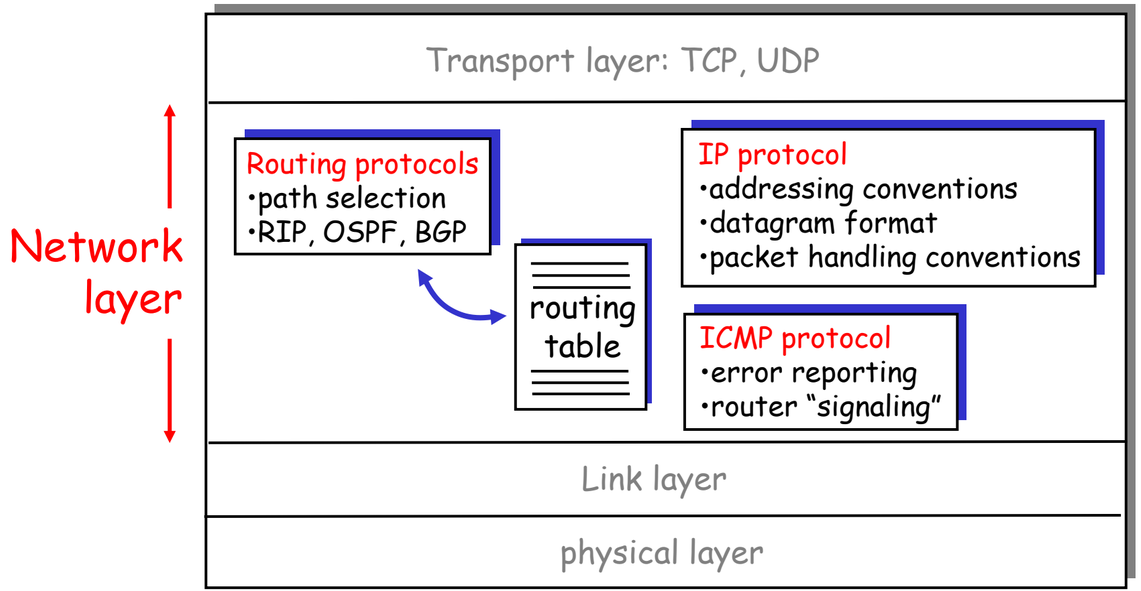
- 5.1 Introduction
- 5.2 Virtual circuit and datagram networks
- 5.3 What's inside a router?
- 5.4 Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector
  - Hierarchical routing
- 5.5 Routing in the Internet
- 5.6 IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6

5: 网络层

- 5.1 简介
- 5.2 虚拟电路与数据报网络
- 5.3 路由器里有什么?
- 5.4 路由算法:
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量 分层路由
- 5.5 互联网中的路由
- 5.6 IP: 互联网协议
  - IPv4数据报格式 IPv4寻址 IP分片 网络地址转换 ARP ICMP IPv6

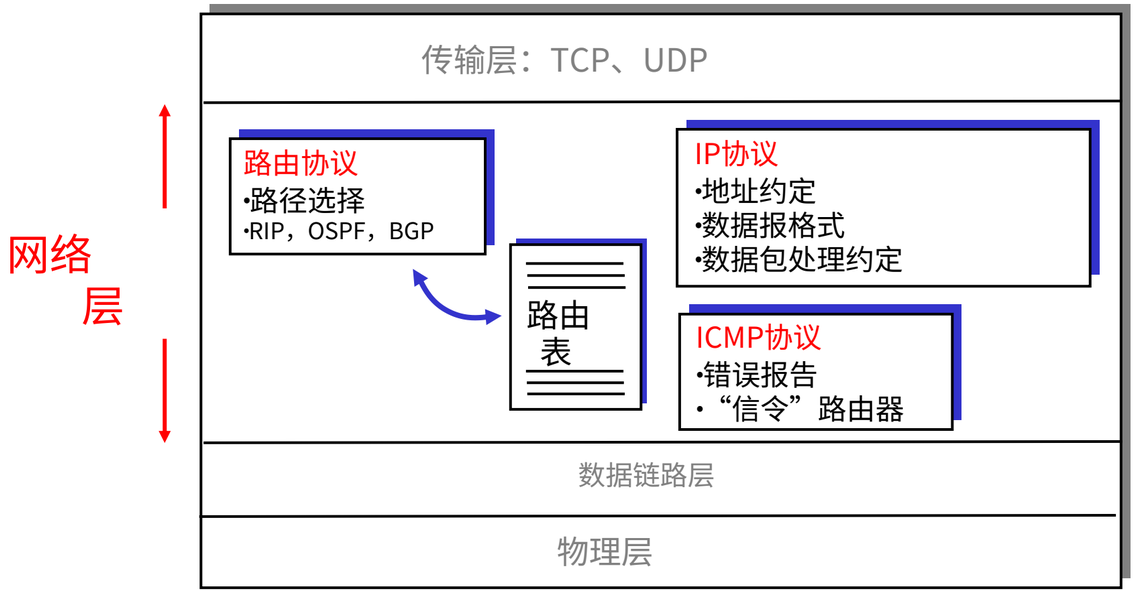
The Internet Network layer

Host, router network layer functions:

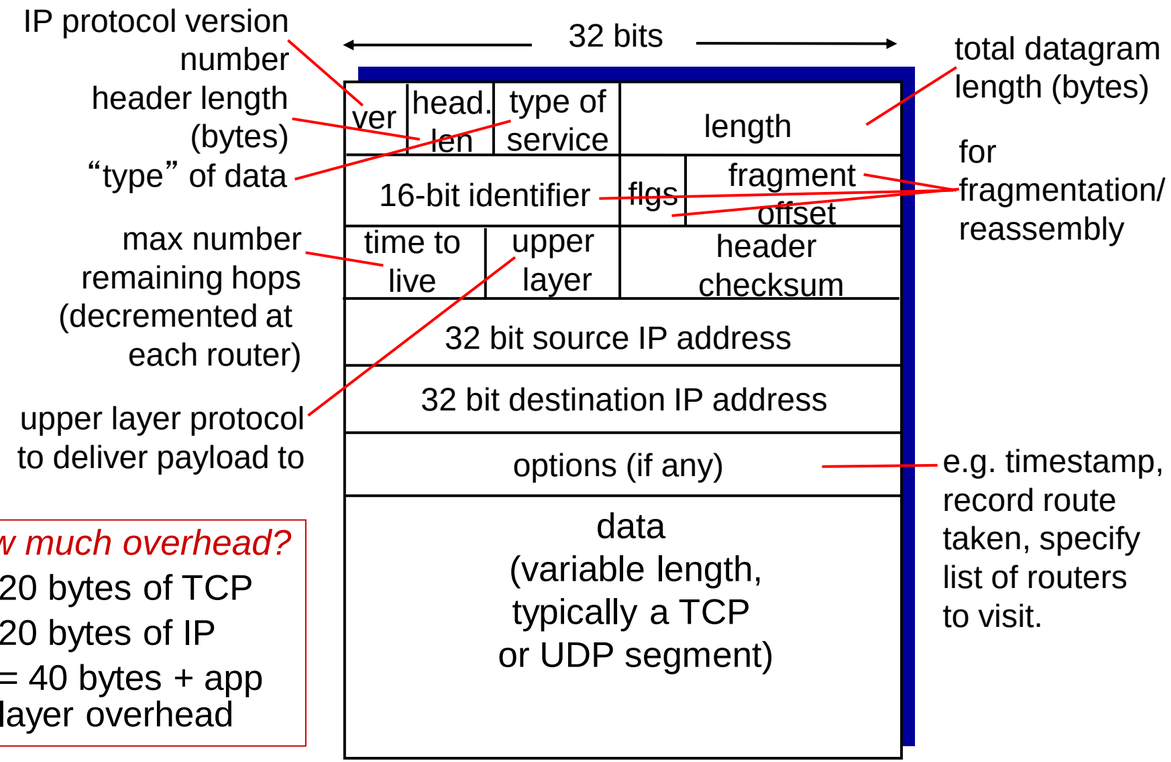


互联网网络层

主机、路由器网络层功能：



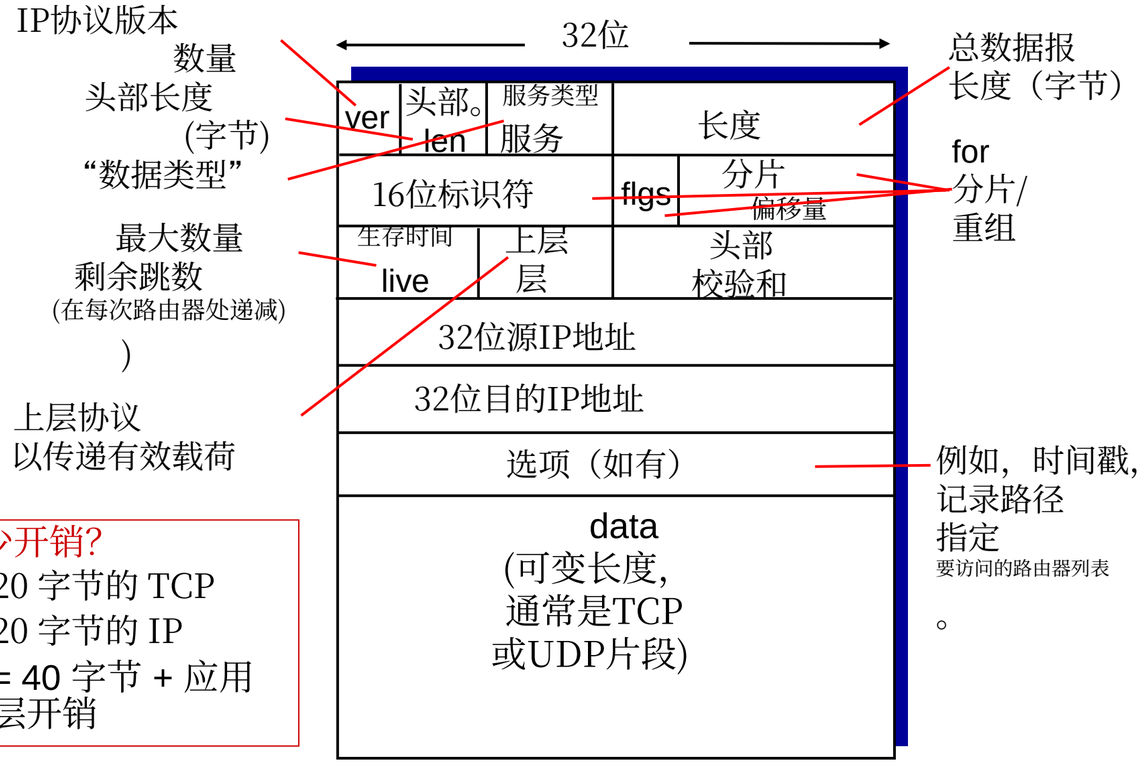
IP datagram format



how much overhead?

- 20 bytes of TCP
- 20 bytes of IP
- = 40 bytes + app layer overhead

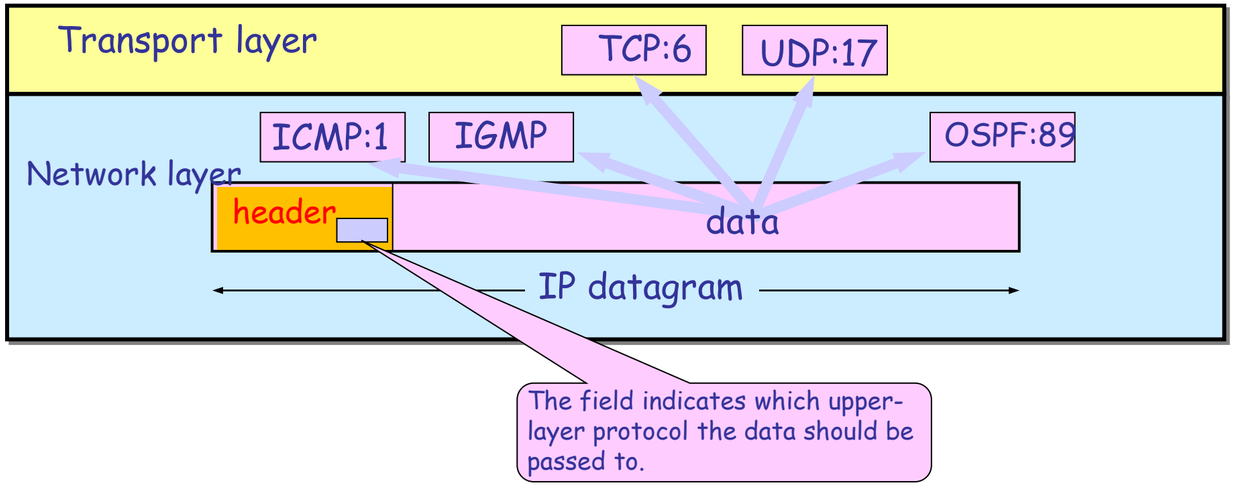
IP数据报格式



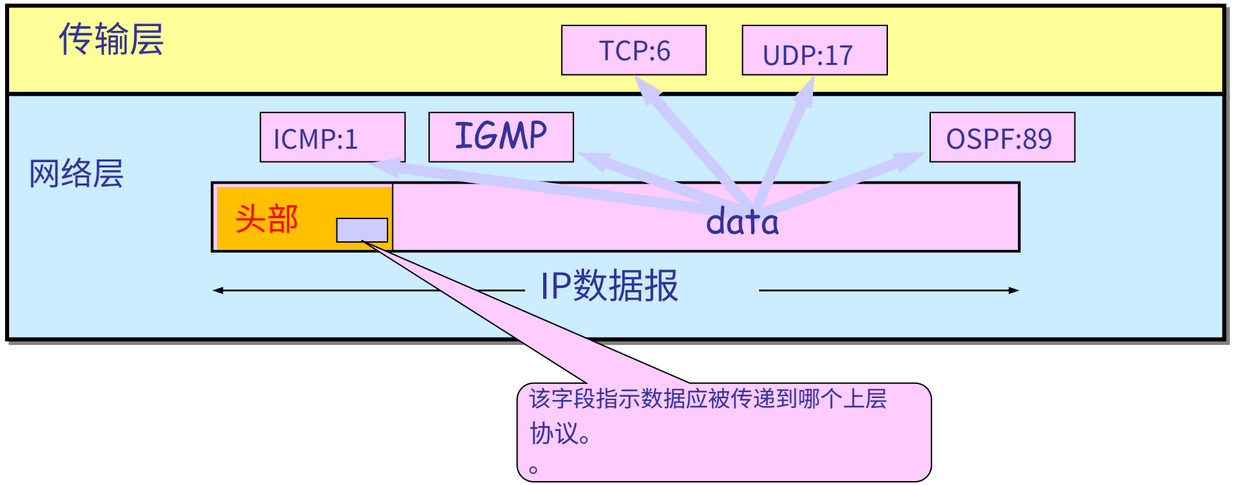
多少开销?

- 20 字节的 TCP
- 20 字节的 IP
- = 40 字节 + 应用层开销

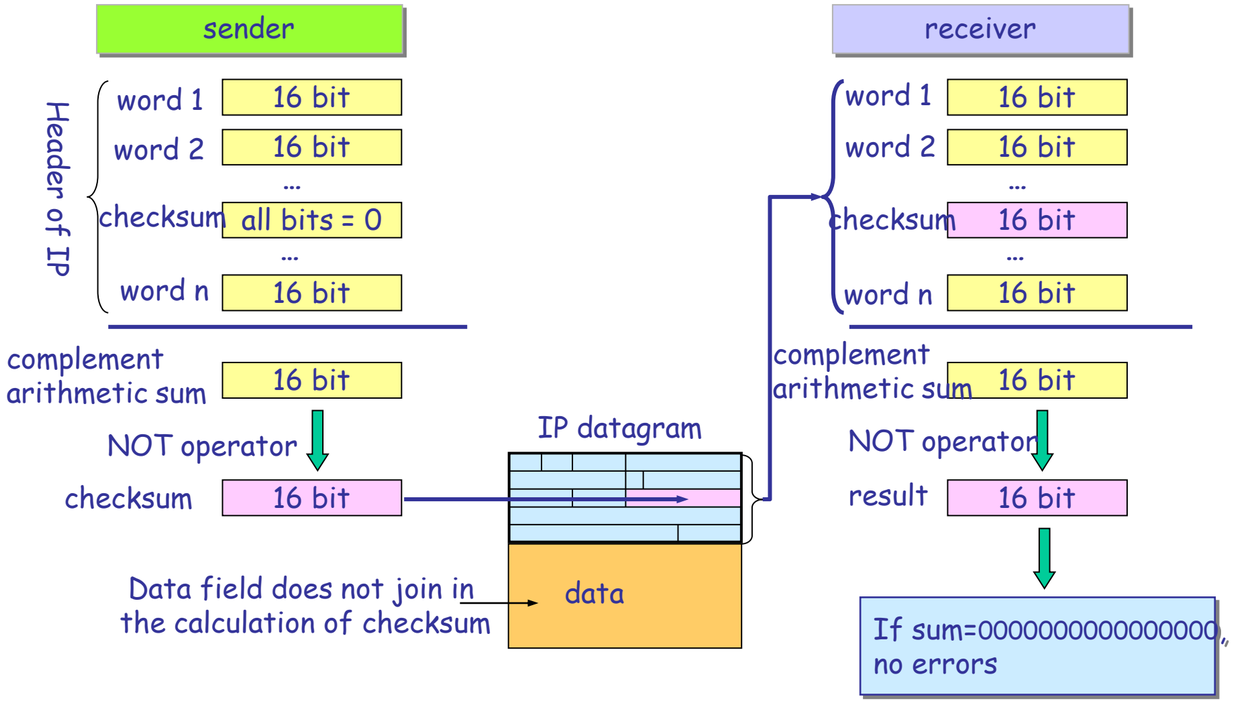
Upper-layer protocol field



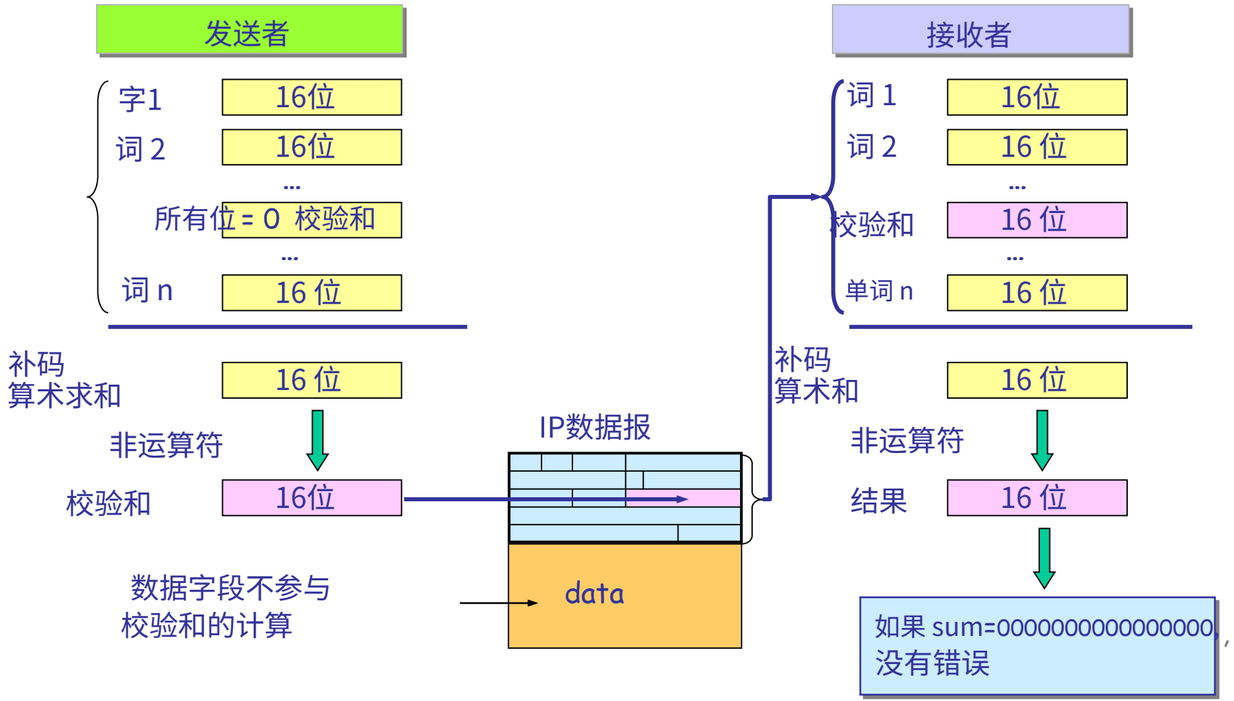
上层-层协议字段



Header checksum

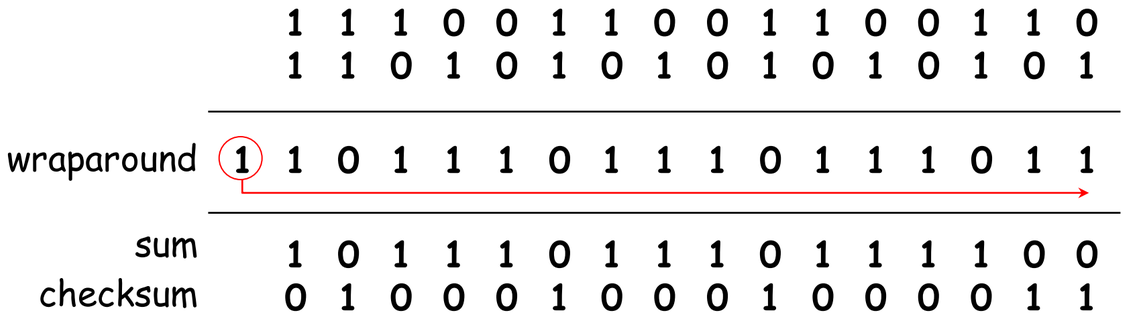


头部校验和



Internet checksum: example

example: add two 16-bit integers

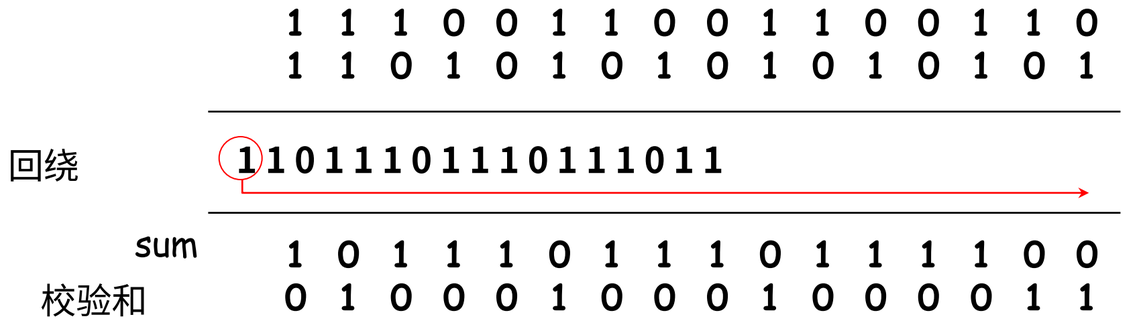


Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

互联网校验和：示例

示例：添加两个16位整数

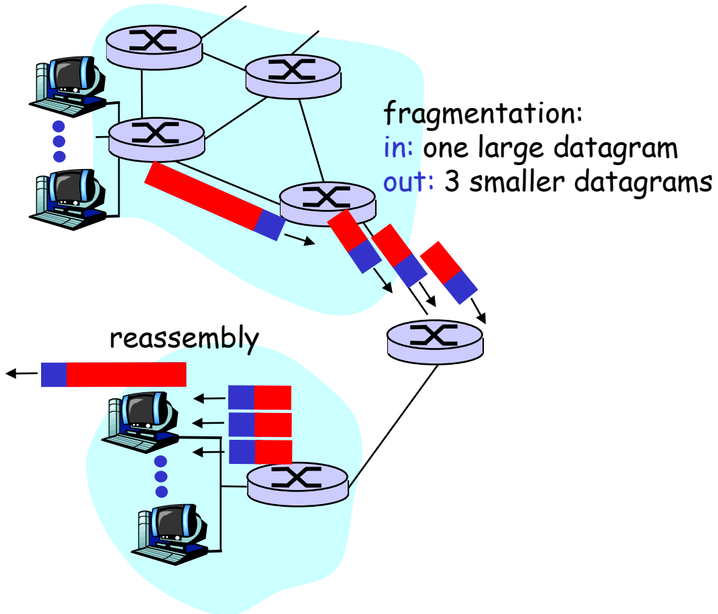


注意：在加法运算中，需要将最高有效位产生的进位加到结果中

* 请访问在线互动练习获取更多示例：
http://gaia.cs.umass.edu/kurose_ross/interactive/

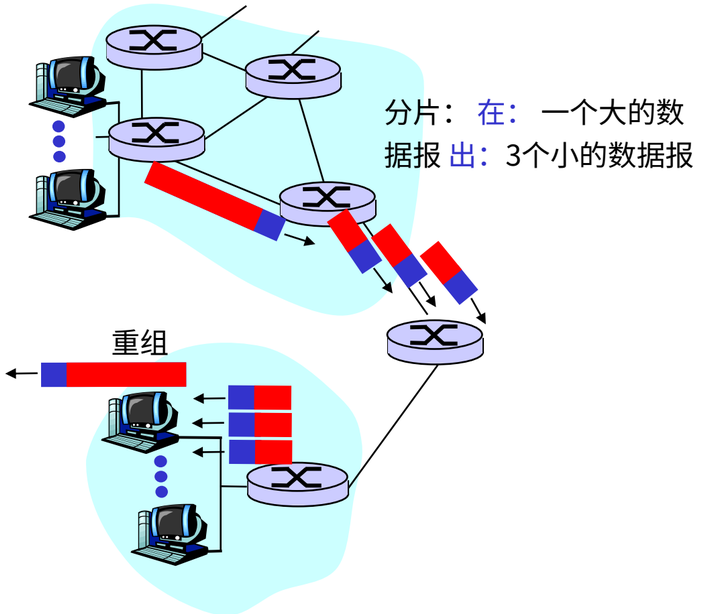
IP Fragmentation & Reassembly

- network links have MTU (max.transfer size) - largest possible link-level frame.
  - different link types, different MTUs
- large IP datagram divided ("fragmented") within net
  - one datagram becomes several datagrams
  - "reassembled" only at final destination
  - IP header bits used to identify, order related fragments

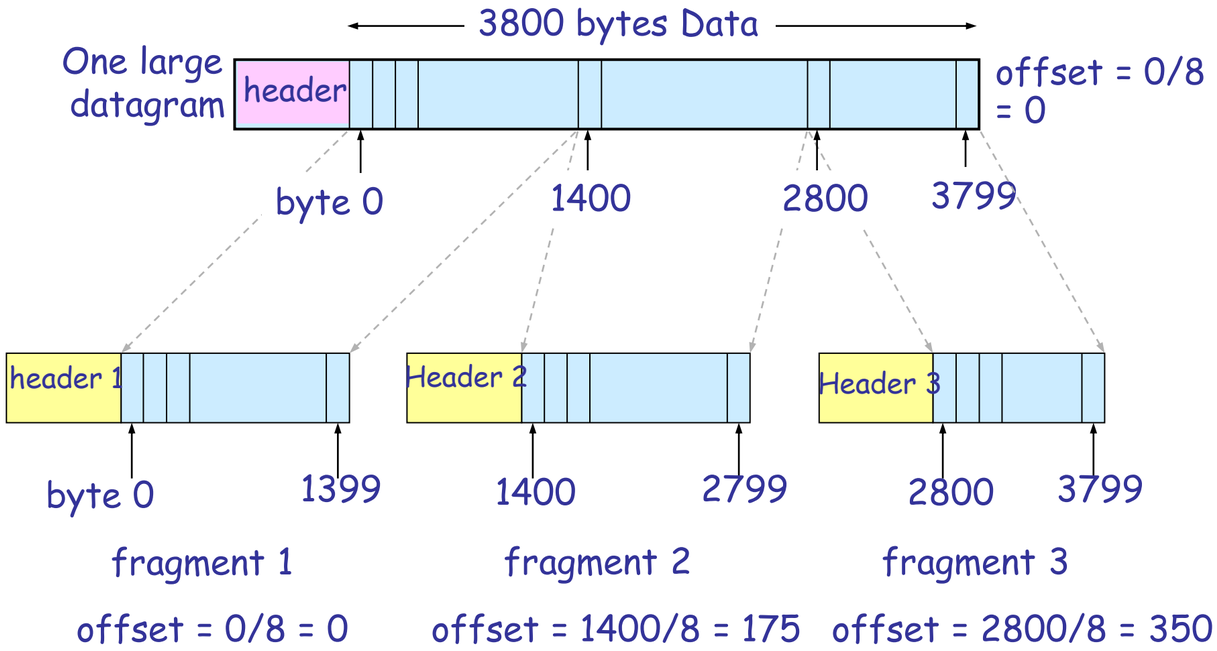


IP分片与重组

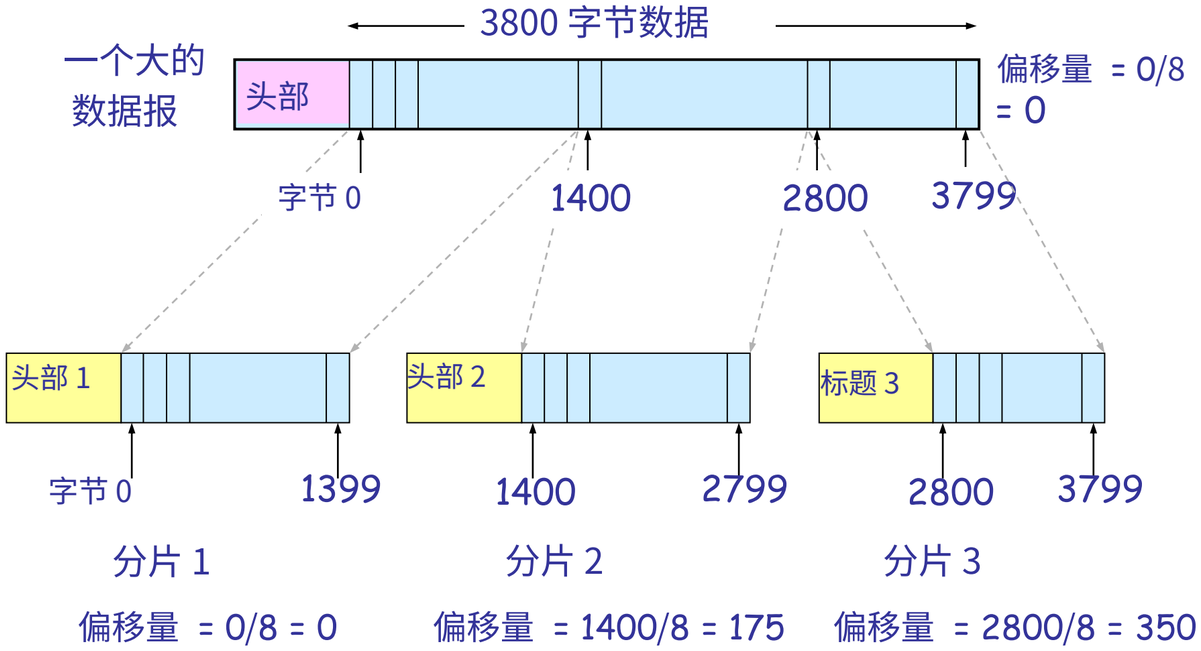
- 网络链路具有MTU（最大传输单元）- 即最大的链路级帧
  - 不同的链路类型，不同的MTU
- 一个大的IP数据报在网内被分片（fragmented）
  - 一个数据报变成多个数据报
  - “重组”仅在最终目的
  - IP头部比特用于识别、排序相关分片



IP Fragmentation



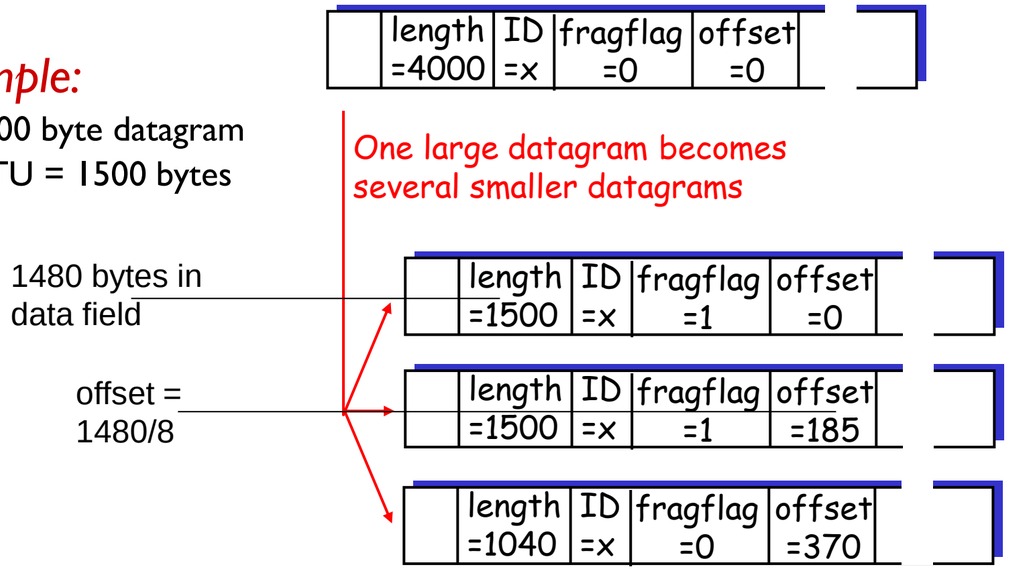
IP分片



IP Fragmentation and Reassembly

example:

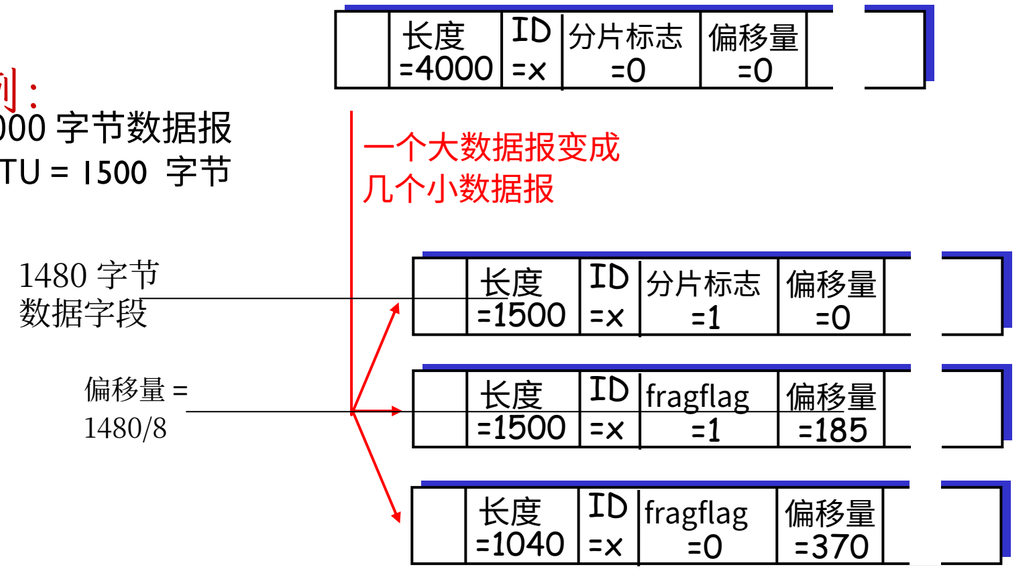
- 4000 byte datagram
- MTU = 1500 bytes



IP分片与重组

示例：

- 4000 字节数据报
- MTU = 1500 字节



IP datagram

- Now assume a IP datagram is captured. The first 20 bytes are as follows:

  0x45 0x00 0x00 0x3C 0x1A 0x37 0x00 0x00 0x80 0x01 0x6E 0x31 0xC0 0xA8 0x01 0xD4 0xD3 0x9B 0x1C 0x41

Please try to analyze the value and meaning of each field in the IP datagram header.

IP 数据报

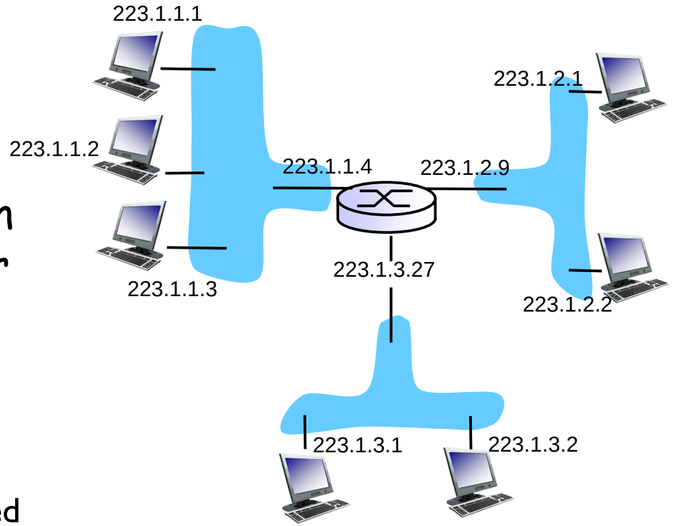
- 现在假设捕获了一个IP数据报。前20个字节如下：

  0x45 0x00 0x00 0x3C 0x1A 0x37 0x00 0x00 0x80 0x01 0x6E 0x31 0xC0 0xA8 0x01 0xD4 0xD3 0x9B 0x1C 0x41

请尝试分析IP数据报头部中每个字段的值和含义。

IP addressing: introduction

- *IP address:* 32-bit identifier for host, router *interface*
- *interface:* connection between host/router and physical link
  - router's typically have multiple interfaces
  - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)
- *IP addresses associated with each interface*

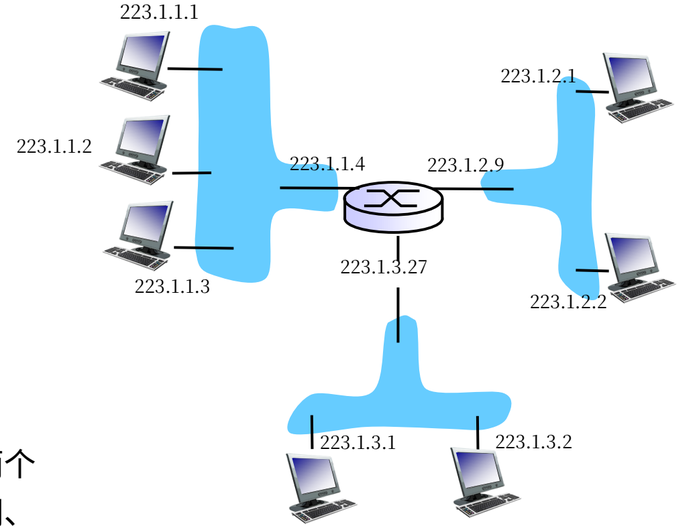


223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

IP寻址：简介

- IP地址：32位主机、路由器<样式 id='4'>接口</样式>
- 接口：主机/路由器与物理链路之间的连接
  - 路由器' 通常具有多个接口
  - 主机通常有一个或两个接口（例如，有线以太网、无线802.11）
- IP地址与每个接口相关联

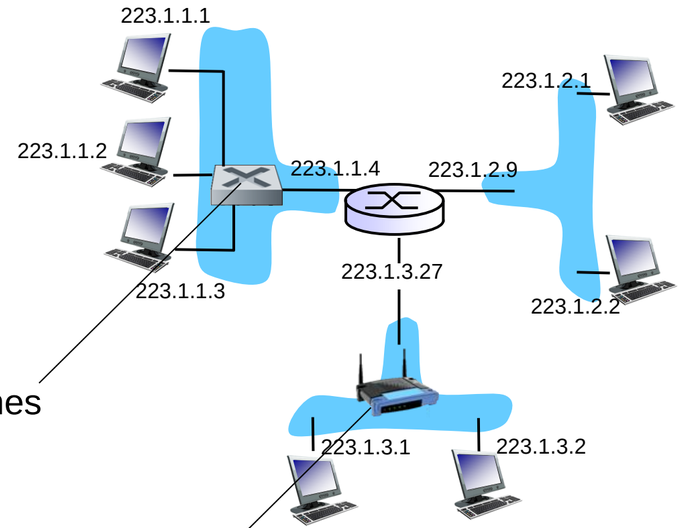


223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

IP addressing: introduction

Q: how are interfaces actually connected?



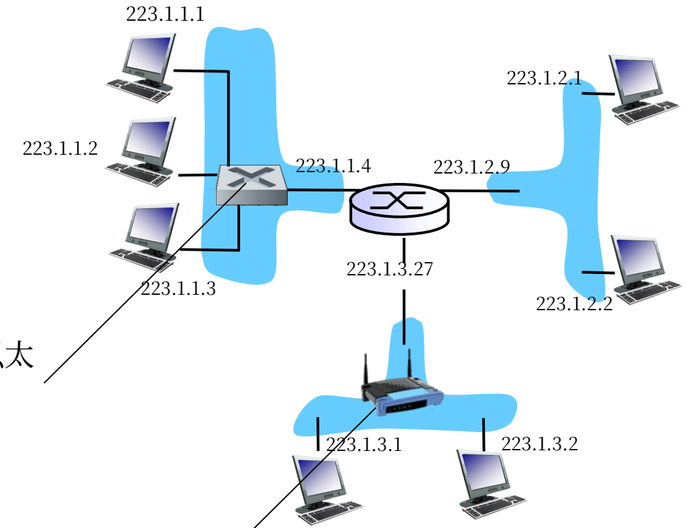
A: wired Ethernet interfaces connected by Ethernet switches

For now: don't need to worry about how one interface is connected to another (with no intervening router)

A: wireless WiFi interfaces connected by WiFi base station

IP寻址：简介

Q：接口实际是如何连接的？



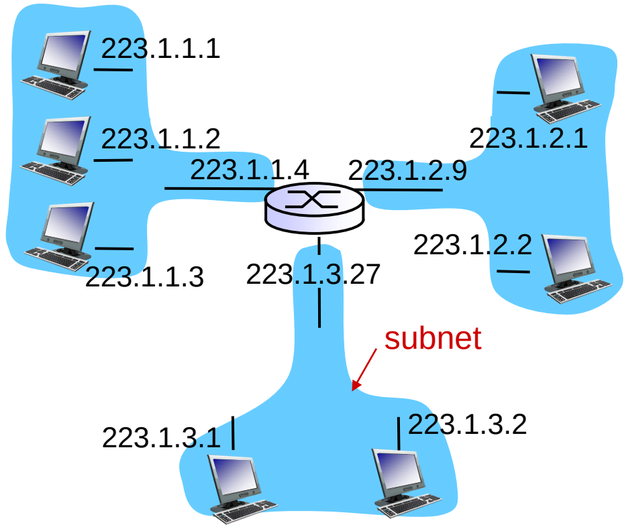
A: 通过以太网交换机连接的有线以太网接口

目前： 无需担心一个接口如何连接到另一个接口（没有中间的路由器）

A: 无线 Wi-Fi 接口通过 Wi-Fi 基站连接

Subnets

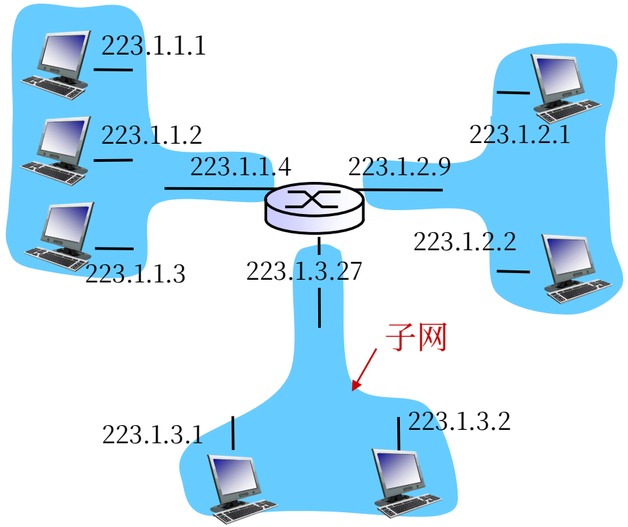
- IP address:
  - subnet part - high order bits
  - host part - low order bits
- *what's a subnet ?*
  - device interfaces with same subnet part of IP address
  - can physically reach each other *without intervening router*



network consisting of 3 subnets

子网

- IP地址：
  - 子网部分 - 高位 主机部分 - 低位
- 子网是什么’？
  - 具有相同子网部分的IP地址的设备接口
  - 可以物理上互相访问 而无需经过路由器

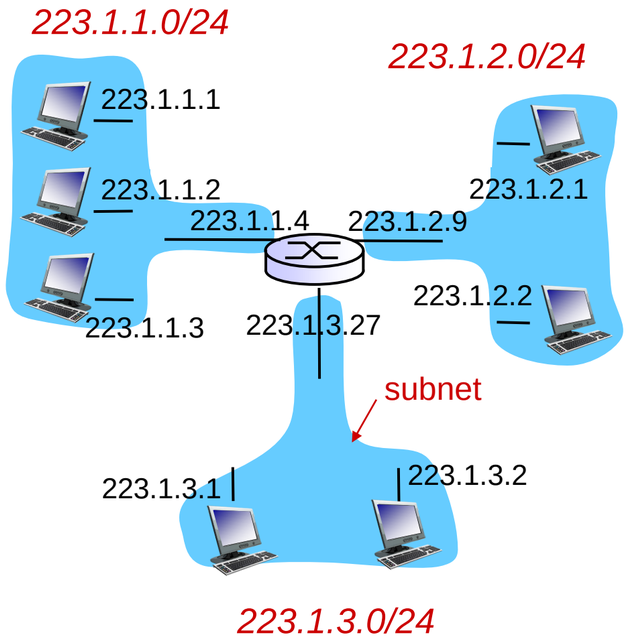


由 3 个子网组成的网络

Subnets

recipe

- to determine the subnets, detach each interface from its host or router, creating islands of isolated networks
- each isolated network is called a *subnet*

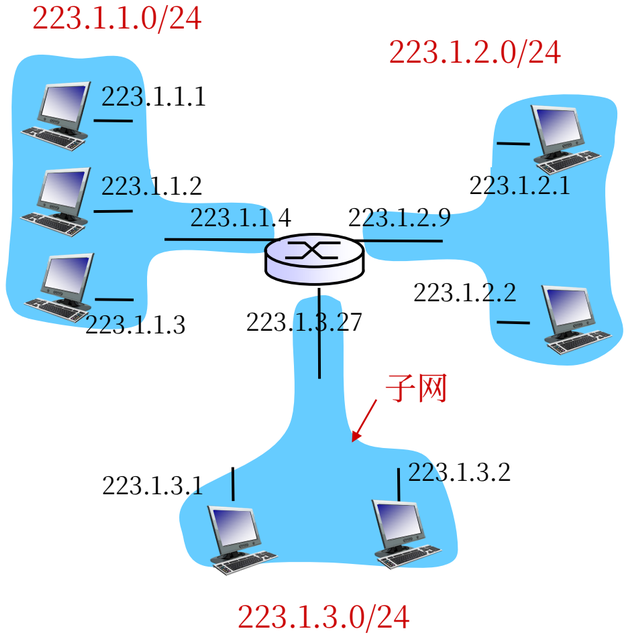


subnet mask: /24

子网

食谱

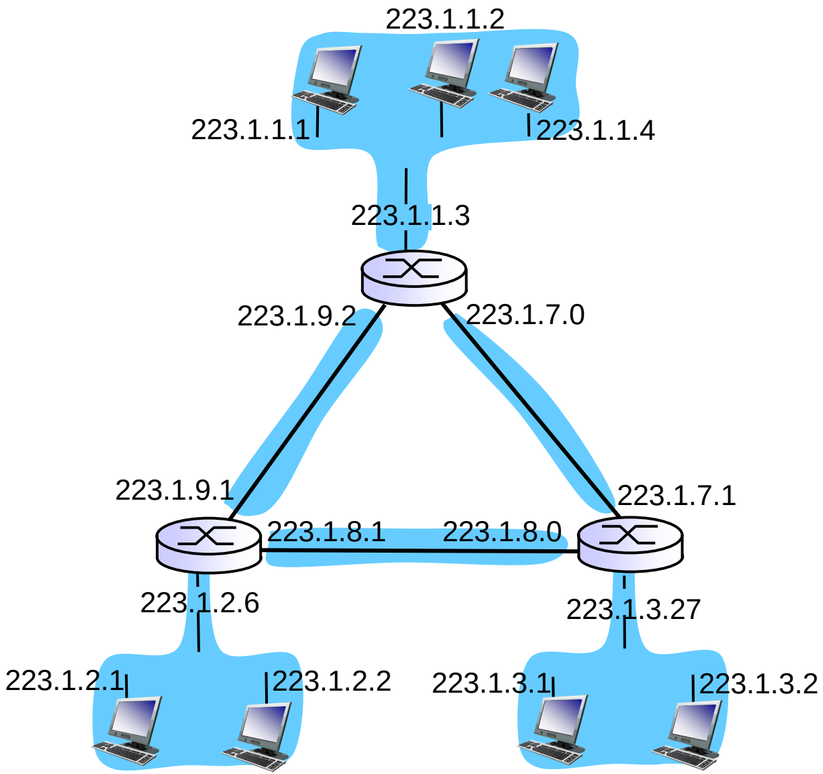
- 要确定子网，需要将每个接口与其主机或路由器分离，从而创建隔离的网络孤岛
- 每个隔离的网络被称为一个子网



子网掩码：/24

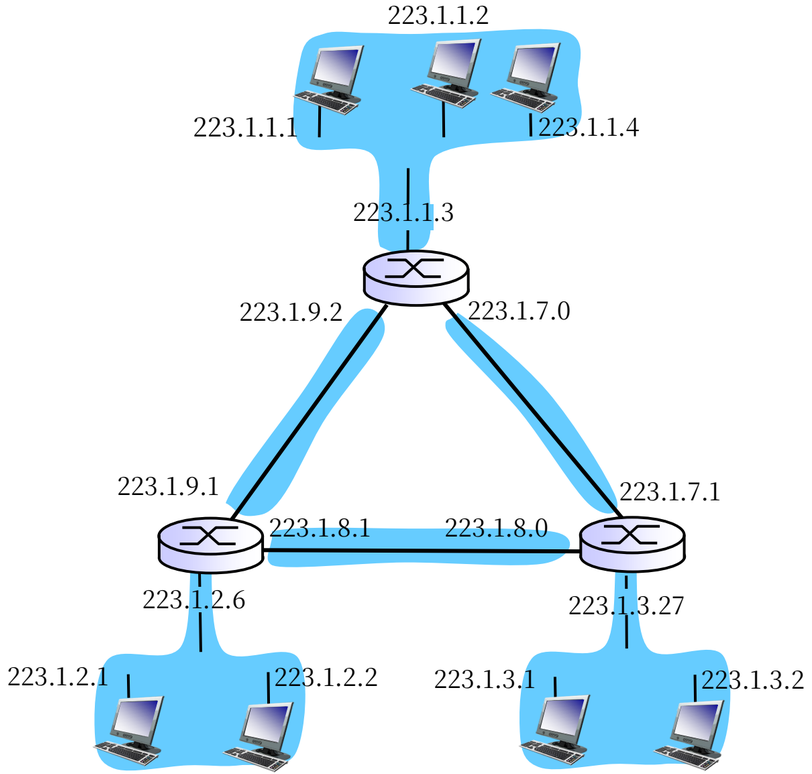
Subnets

how many?



子网

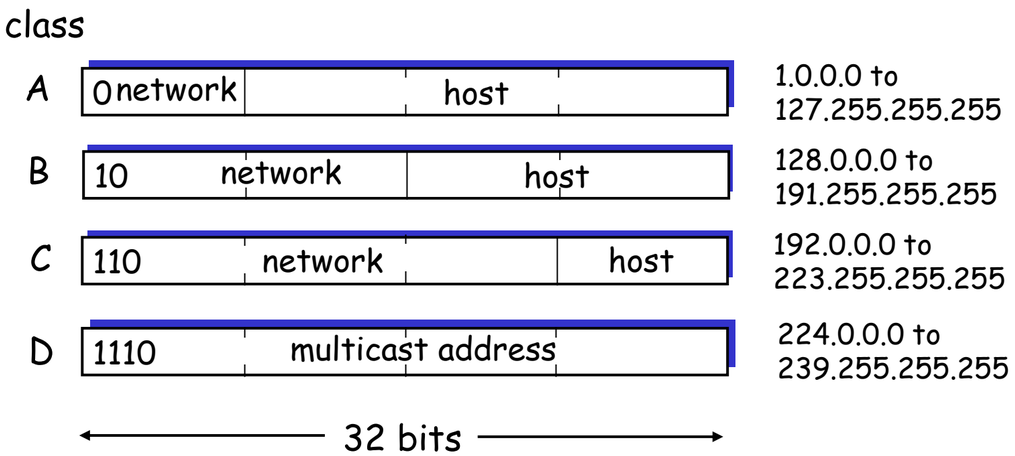
有多少?



IP Addresses

given notion of "network", let's re-examine IP addresses:

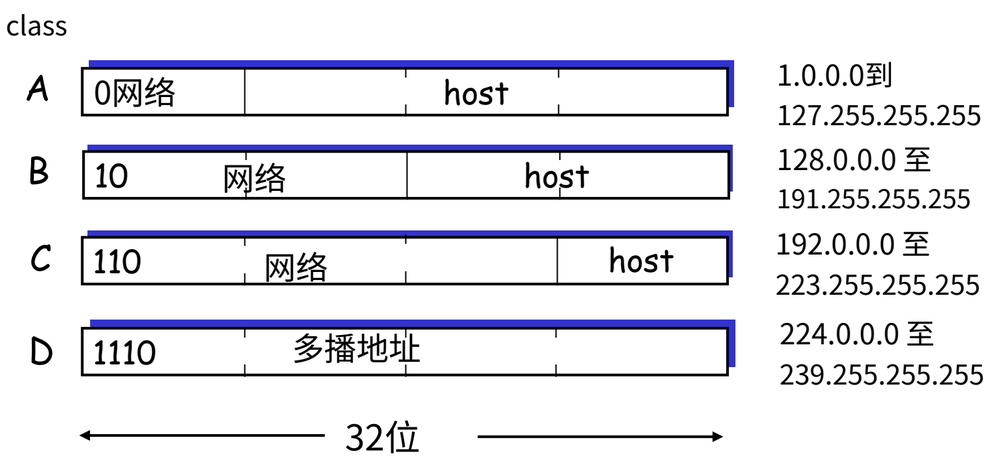
"class-full" addressing:



IP地址

在"网络"的概念下，让我们重新审视IP地址：

"类全址寻址：



网络号	主机号	地址类型	用途
全0	**全0**	**本机**	**启动时使用**
网络ID	**全0**	**网络**	**标识一个网络**
网络ID	**全1**	**直接广播**	**在指定网上广播**
全1	**全1**	**有限广播**	**在本地网上广播**
127	**任意**	**回送**	**环回测试（127.0.0.1）**

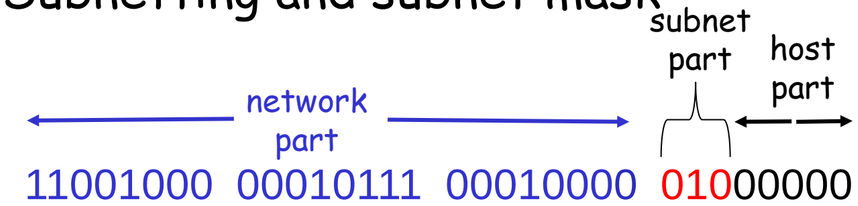
A类地址：10.0.0.0--10.255.255.255
B类地址：172.16.0.0--172.31.255.255
C类地址：192.168.0.0--192.168.255.255

网络号	主机号	地址类型	用途
全0	**全0**	本机	启动时使用
网络**ID**	**全0**	网络	标识一个网络
网络**ID**	**全1**	直接广播	在指定网广播
全1	**全1**	有限广播	在本地网广播
127	任意	回送	环回测试(**127.0.0.1**)

A类地址:10.0.0.0--10.255.255.255 B类地址:
172.16.0.0--172.31.255.255 C类地址:
192.168.0.0--192.168.255.255

IP addressing

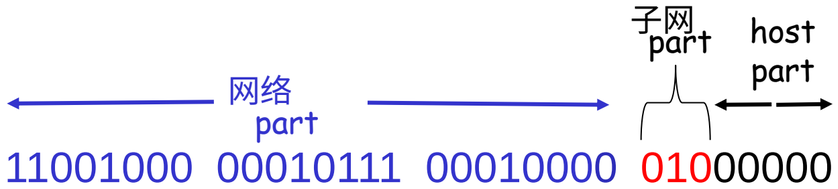
- Dotted-decimal notation:193.32.216.9
- classful addressing:
  - inefficient use of address space, address space exhaustion
  - e.g., class B net allocated enough addresses for 65K hosts, even if only 2K hosts in that network
- Subnetting and subnet mask



200.23.16.64/27

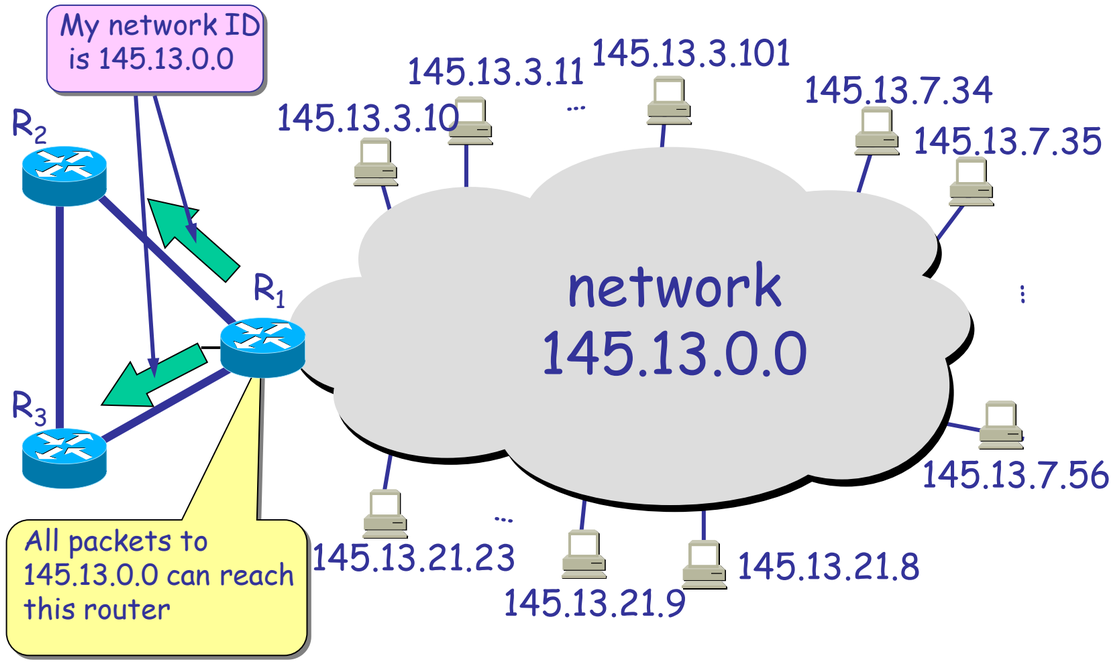
IP寻址

- 点分十进制表示法：193.32.216.9
- 分类寻址：
  - 地址空间使用效率低，地址空间枯竭
  - 例如，B类网络分配的地址足够支持65K个主机，即使该网络中只有2K个主机
- 子网划分和子网掩码

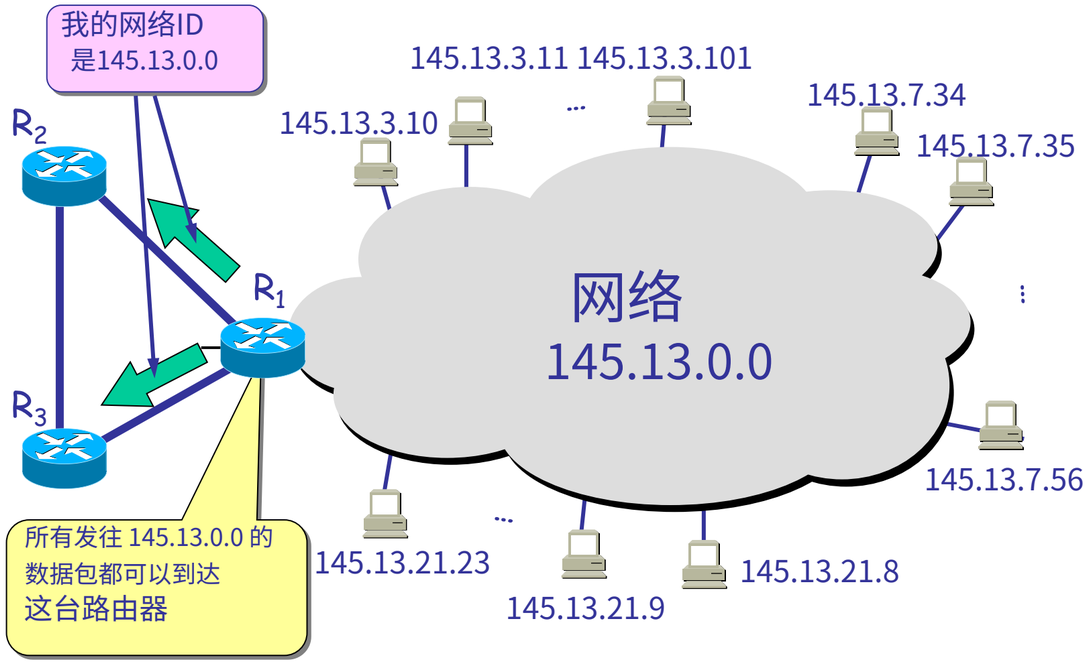


200.23.16.64/27

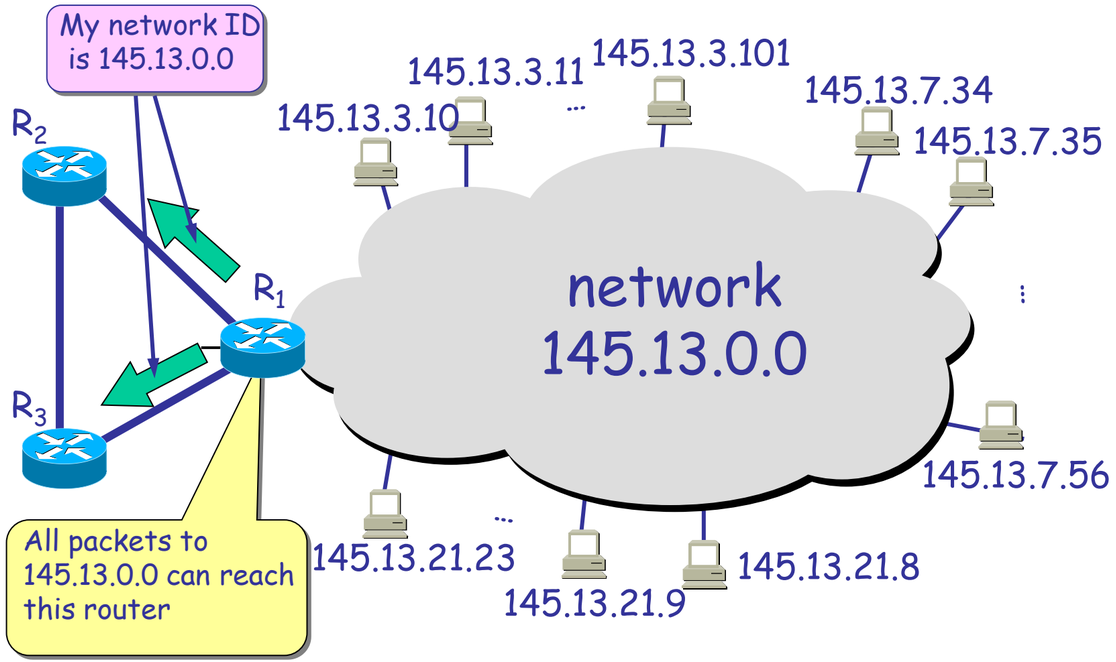
A class B network without subnet: 145.13.0.0



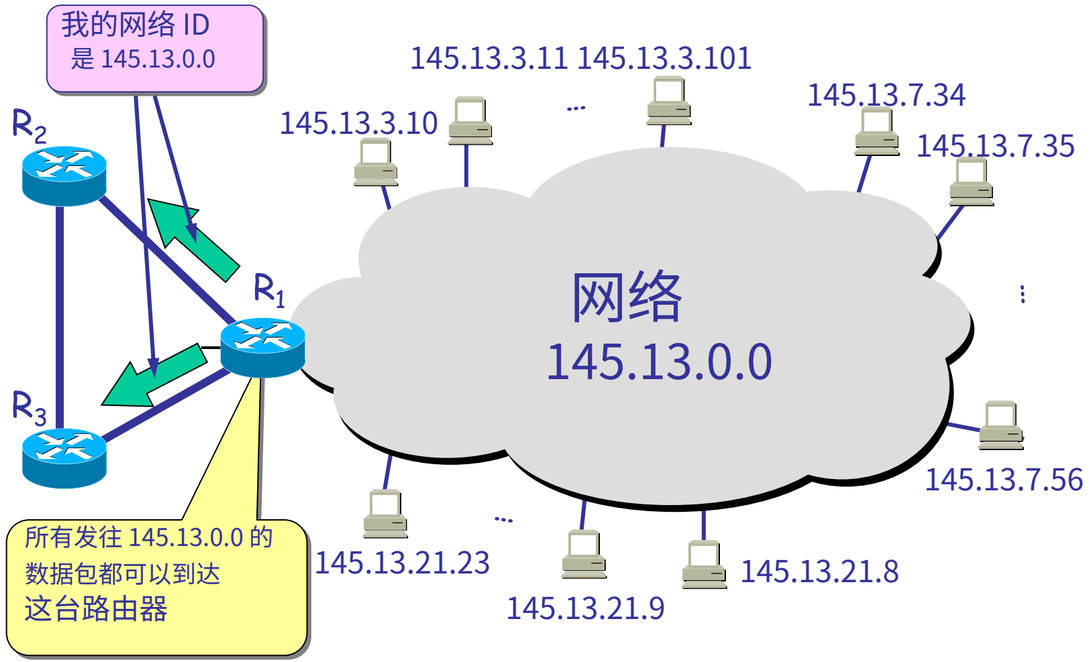
一个没有子网的B类网络：145.13.0.0



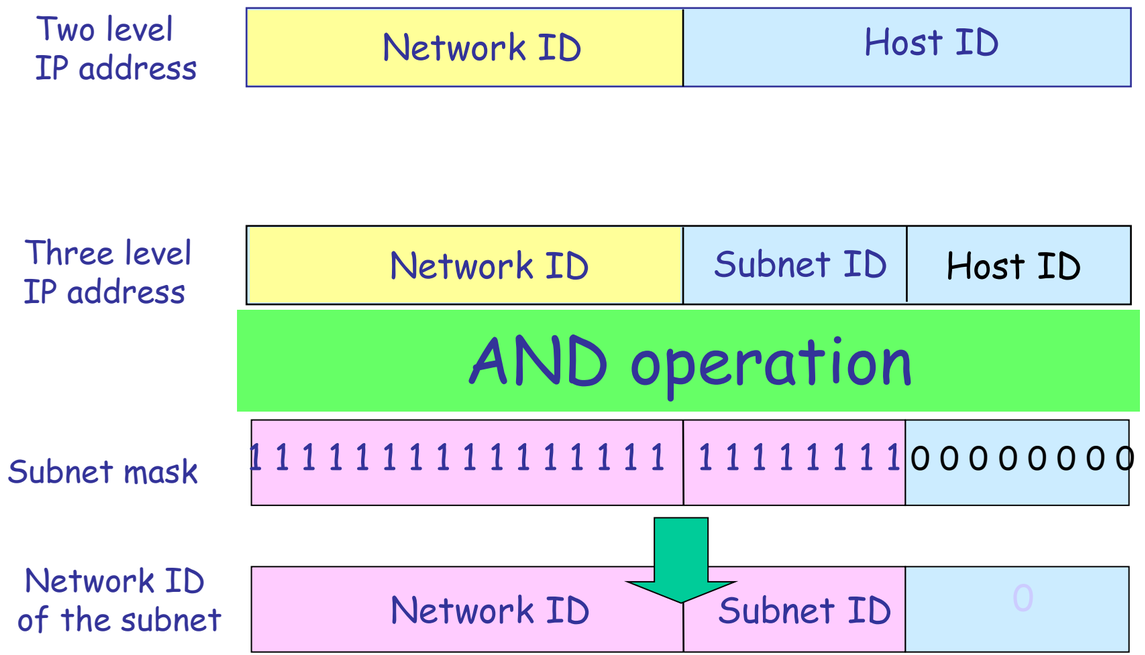
A network for others, the class B with 3 subnet



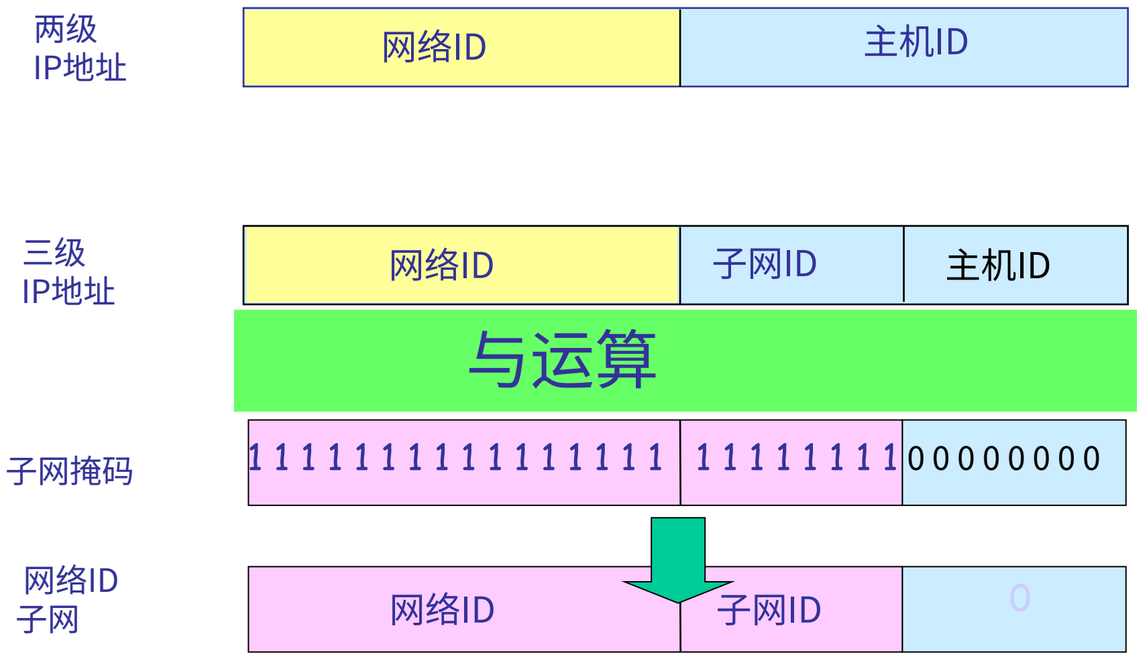
为他人而建的网络，B类拥有3个子网



Subnet mask



子网掩码



IP addressing: CIDR

- CIDR: Classless InterDomain Routing
  - Two level address: only two parts
  - network portion of address of arbitrary length
  - address format: a.b.c.d/x, where x is # bits in network portion of address
  - Network portion is often called prefix



200.23.16.0/23

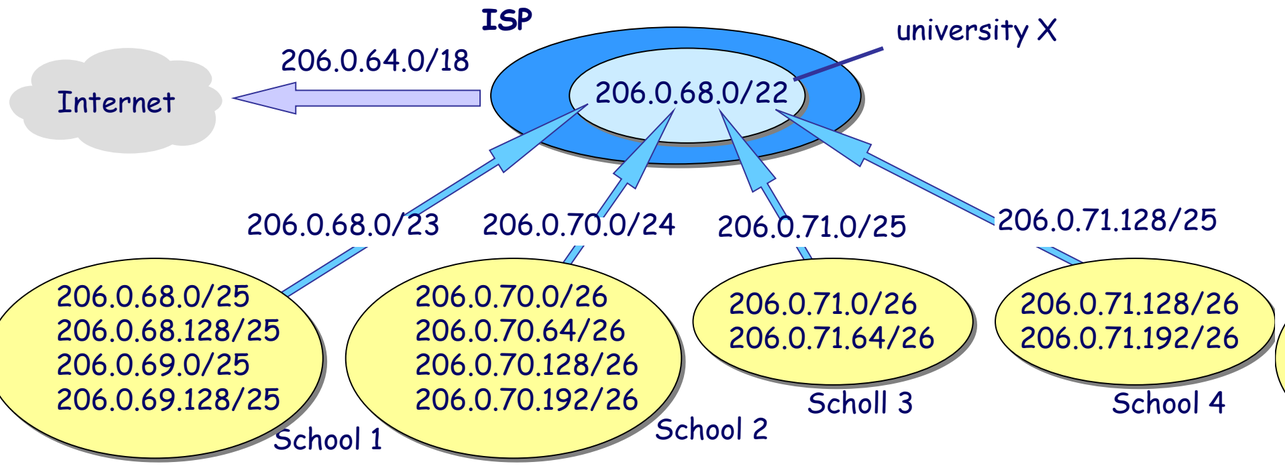
IP寻址：CIDR

- CIDR：无类域间路域路由由
  - 两级地址：仅包含两部分 地址的网络部分长度任意 地址格式：a.b.c.d/x，其中x表示网络部分的位数 网络部分通常称为前缀

网络 主机
11001000 00010111 00010000 00000000
部分 部分

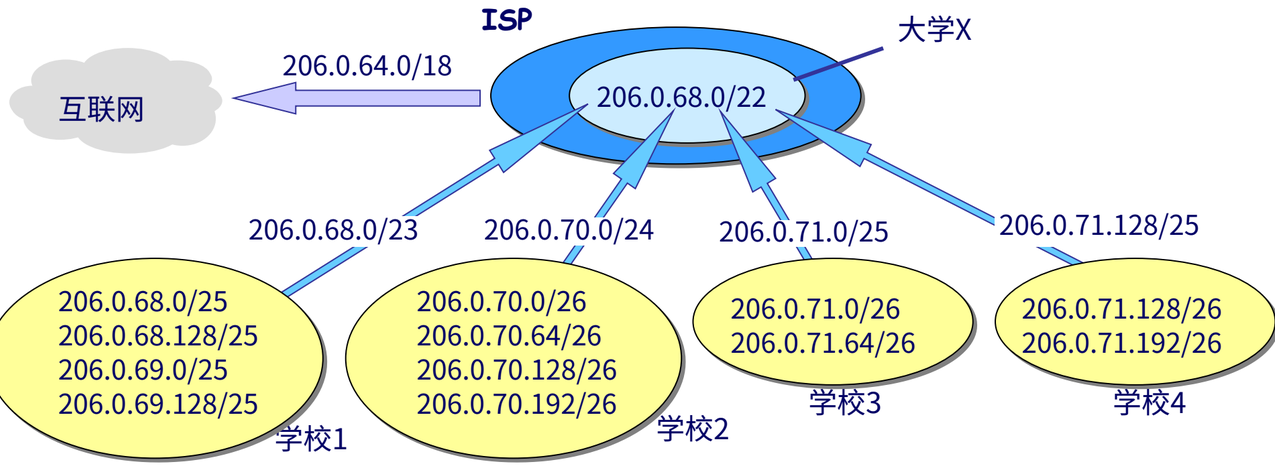
200.23.16.0/23

CIDR example



organizatio	address block	binary	number
ISP	206.0.64.0/18	11001110.00000000.01*	16384
Uni. X	206.0.68.0/22	11001110.00000000.010001*	1024
sch. 1	206.0.68.0/23	11001110.00000000.0100010*	512
Sch. 2	206.0.70.0/24	11001110.00000000.01000110.*	256
Sch.3	206.0.71.0/25	11001110.00000000.01000111.0*	128
Sch. 4	206.0.71.128/25	11001110.00000000.01000111.1*	128

CIDR示例



组织地址块	二进制数	
互联网服务提供商	206.0.64.0/18 11001110.00000000.01*	16384
Uni. X	206.0.68.0/22 11001110.00000000.010001*	1024
sch. 1	206.0.68.0/23 11001110.00000000.0100010*	512
Sch. 2	206.0.70.0/24 11001110.00000000.01000110.*	256
Sch.3	206.0.71.0/25 11001110.00000000.01000111.0*	128
Sch. 4	206.0.71.128/25 11001110.00000000.01000111.1*	128

IP addresses: how to get one?

Hosts (host portion):

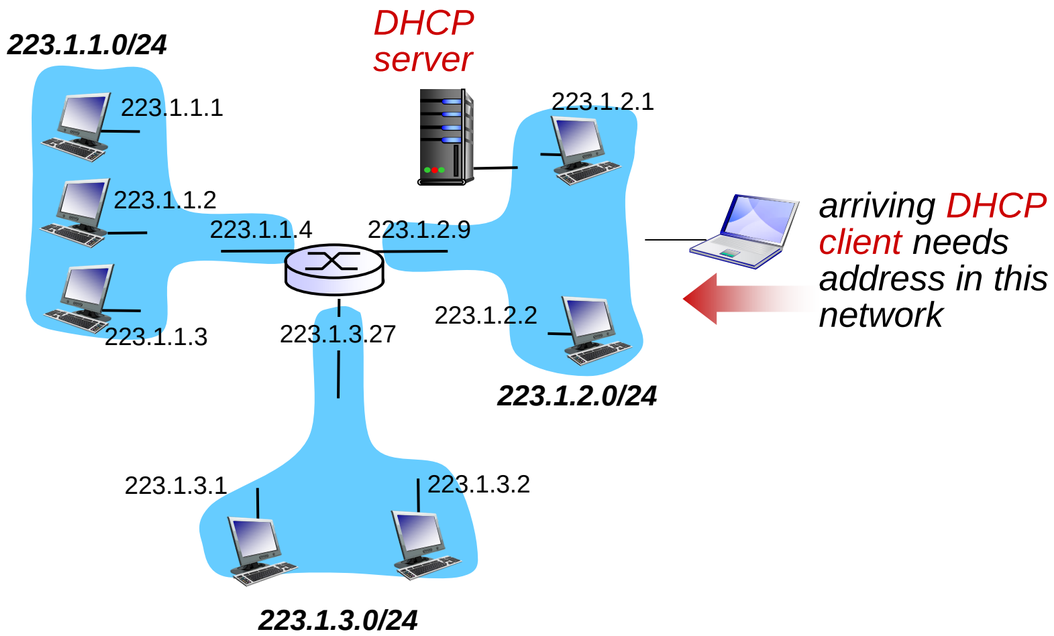
- hard-coded by system admin in a file
- DHCP: Dynamic Host Configuration Protocol: dynamically get address: "plug-and-play"
  - host broadcasts "DHCP discover" msg
  - DHCP server responds with "DHCP offer" msg
  - host requests IP address: "DHCP request" msg
  - DHCP server sends address: "DHCP ack" msg

IP地址：如何获取一个?

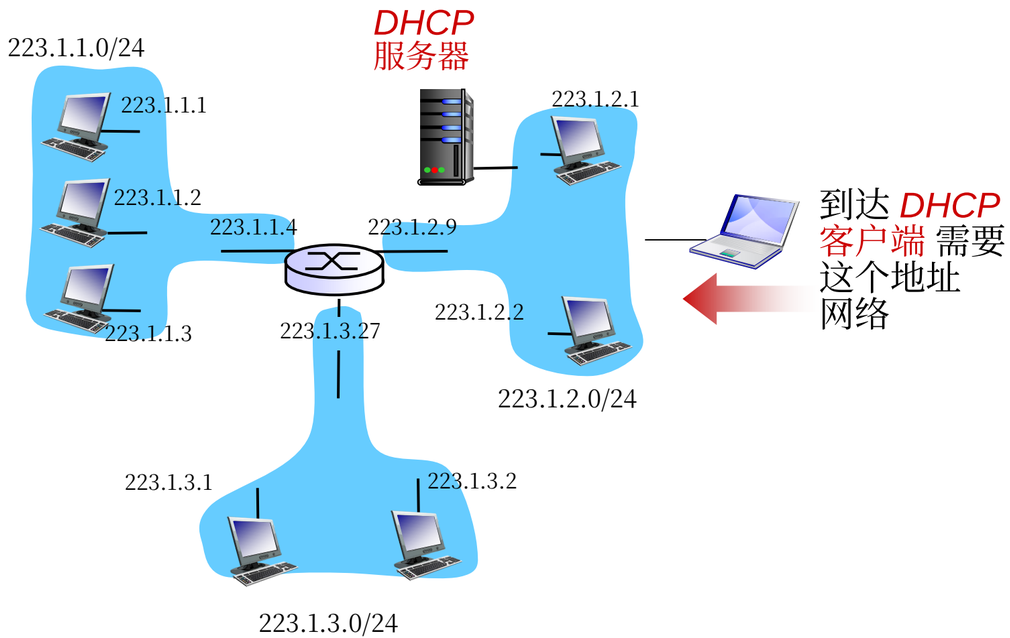
主机（主机部分）：

- 由系统管理员在文件中硬编码
- DHCP：动态 Host Configuration Protocol：动态获取地址："即插即用"
  - 主机广播"DHCP发现"消息
  - DHCP服务器响应"DHCP提供"消息
  - 主机请求IP地址："DHCP请求"消息
  - DHCP服务器发送地址："DHCP确认"消息

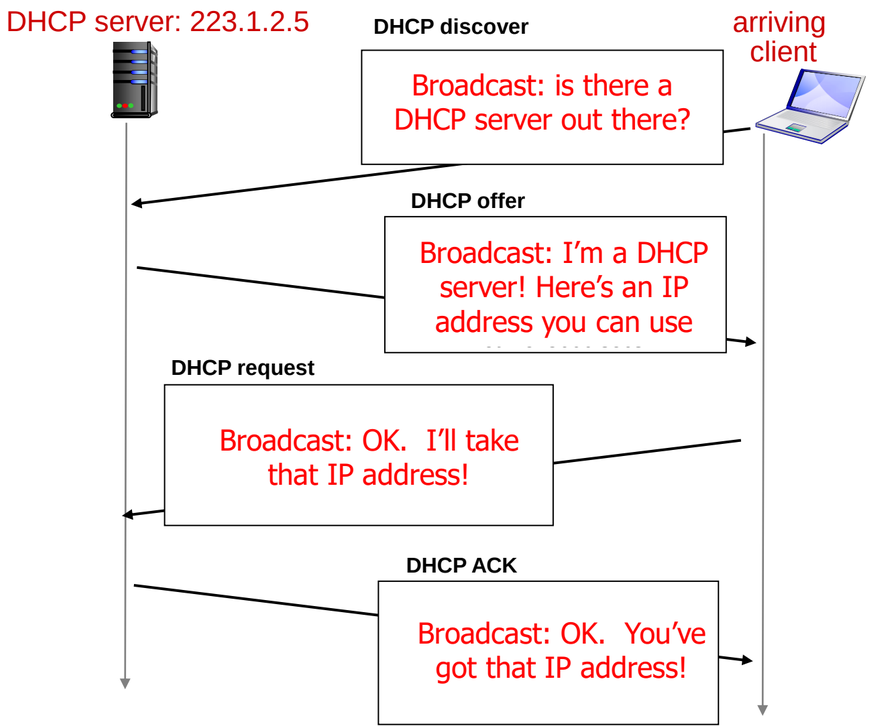
DHCP client-server scenario



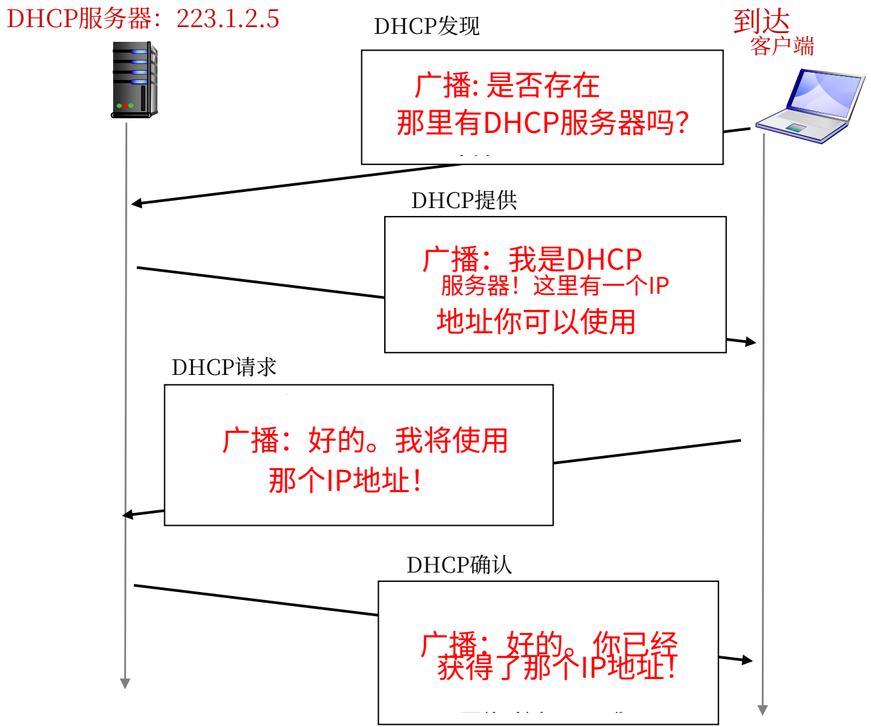
DHCP客户端-服务器场景



DHCP client-server scenario



DHCP客户端-服务器场景



IP addresses: how to get one?

Network (network portion):

- get allocated portion of ISP's address space:

ISP's block	11001000	00010111	00010000	00000000	200.23.16.0/20
Organization 0	11001000	00010111	00010000	00000000	200.23.16.0/23
Organization 1	11001000	00010111	00010010	00000000	200.23.18.0/23
Organization 2	11001000	00010111	00010100	00000000	200.23.20.0/23
...	.....	....	....
Organization 7	11001000	00010111	00011110	00000000	200.23.30.0/23

IP地址：如何获取一个?

网络（网络部分）：

- 获取ISP的地址空间分配部分：

互联网服务提供商的块 11001000 00010111 00010000 00000000 200.23.16.0/20

组织0 11001000 00010111 00010000 00000000 200.23.16.0/23

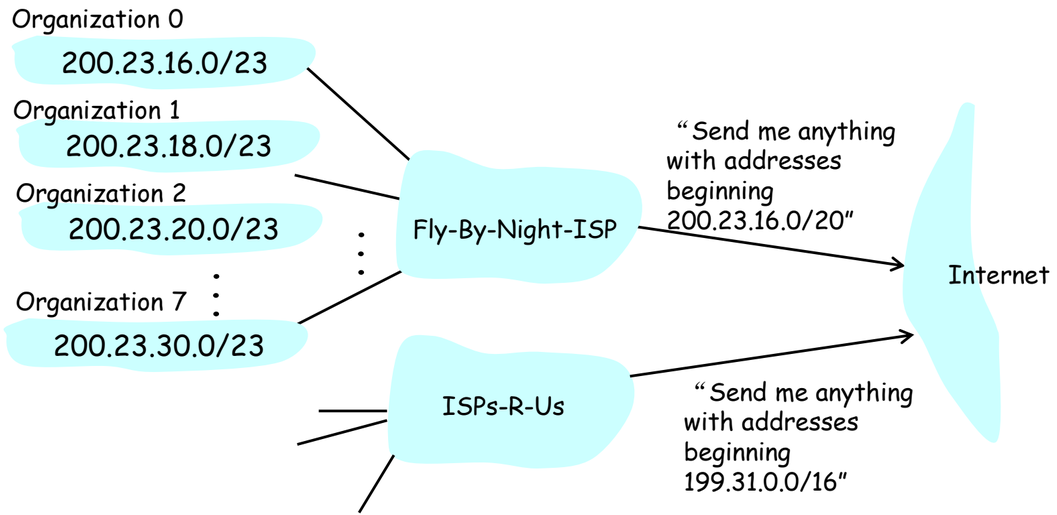
组织1 11001000 00010111 00010010 00000000 200.23.18.0/23

组织2 11001000 00010111 00010100 00000000 200.23.20.0/23 ... .

组织7 11001000 00010111 00011110 00000000 200.23.30.0/23

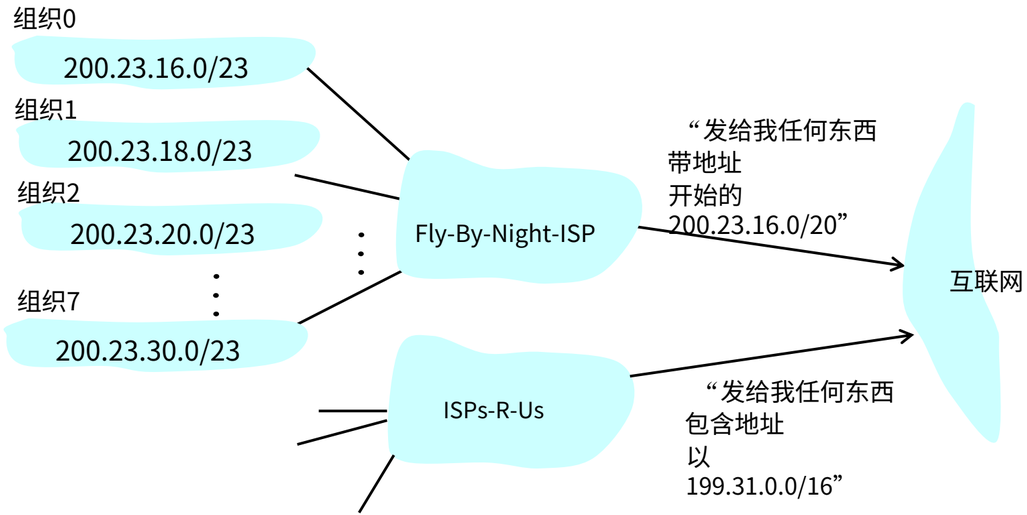
Hierarchical addressing: route aggregation

Hierarchical addressing allows efficient advertisement of routing information:



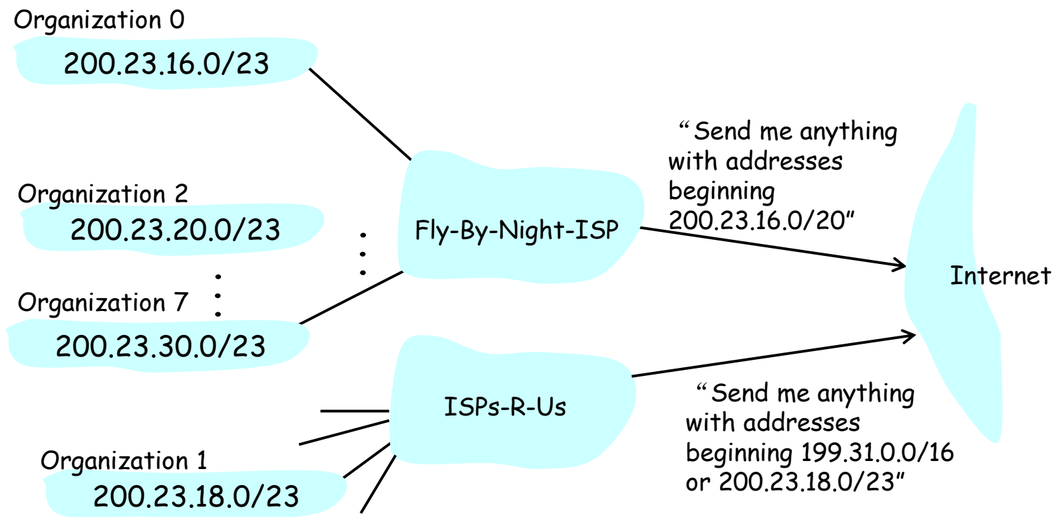
分层寻址：路由聚合

分层寻址允许高效发布路由信息：



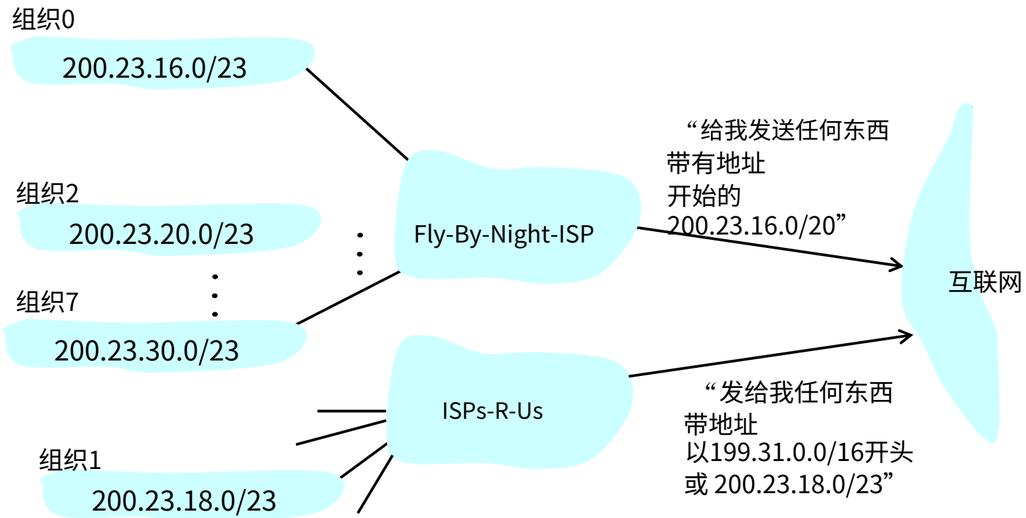
Hierarchical addressing: more specific routes

ISPs-R-Us has a more specific route to Organization 1



分层寻址：更具体的路由

ISPs-R-Us 有一个更具体的路由到组织1



IP addressing: the last word...

Q: How does an ISP get block of addresses?

A: ICANN: Internet Corporation for Assigned Names and Numbers

- allocates addresses
- manages DNS
- assigns domain names, resolves disputes

IP寻址：最后的词...

Q: 互联网服务提供商如何获得地址块?

A: ICANN: Internet Corporation for AssignedNames and Numbers分配地址管理DNS分配域名，解决争议

Exercise-2

1. In subnet 192.168.4.0/30, the maximum number of hosts that can accept IP packets with destination address 192.168.4.3 is

A.0 B.1 C.2 D.4

2. There are four routing table entries with the same forwarding interface in a routing table, and their destination network addresses are 35.230.32.0/21, 35.230.40.0/21, 35.230.48.0/21 and 35.230.56.0/21 respectively. After aggregating the four routes, the destination network address is

A.35.230.0.0/19 B. 35.230.0.0/20

C. 35.230.32.0/19 D. 35.230.32.0/20

练习-2

1. 在子网192.168.4.0/30中，能够接受目的地址为192.168.4.3的IP数据包的主机最大数量是

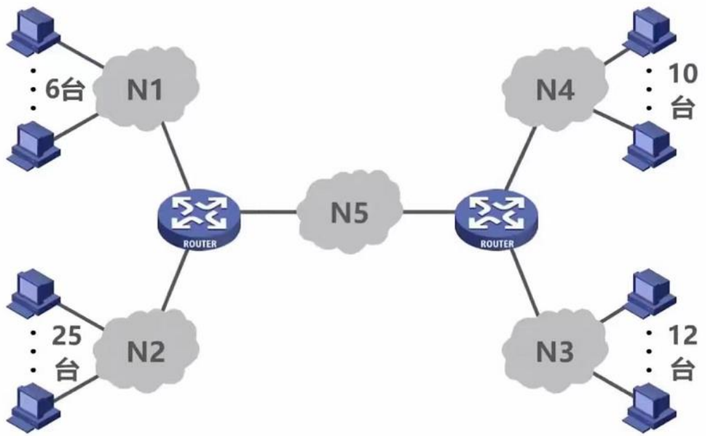
A.0 B.1 C.2 D.4

2. 在一个路由表中，有四个路由表条目使用了相同的转发接口，它们的各自目的网络地址分别是35.230.32.0/21、35.230.40.0/21、35.230.48.0/21和35.230.56.0/21。将这四个路由聚合后，目的网络地址是

A.35.230.0.0/19 B.35.230.0.0/20 C. 35.230.32.0/19 D. 35.230.32.0/20

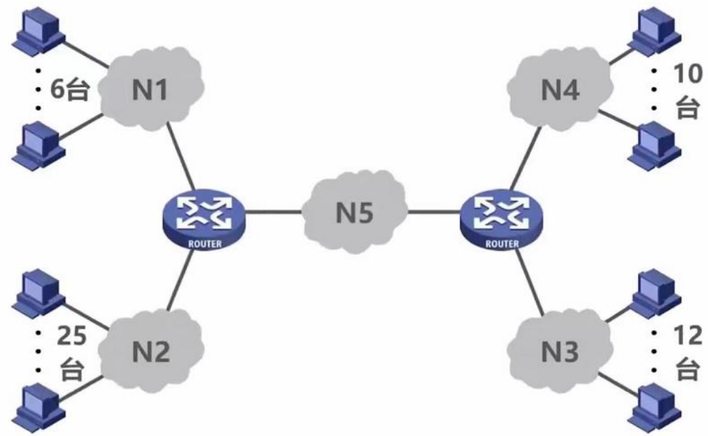
Exercises-3

Assuming that the applied IP address block is 218.75.230.0/24, please assign IP addresses to the devices in the figure below using fixed length and variable length subnet masks respectively.



练习-3

假设所应用的IP地址块是218.75.230.0/24请使用下面的图中所示设备，分别使用固定长度和可变长度子网掩码进行IP地址分配。



Exercises-3

Assuming that the applied IP address block is 218.75.230.0/24, please assign IP addresses to the devices in the figure below using fixed length and variable length subnet masks respectively.

使用子网掩码255.255.255.224 对C类网络218.75.230.0进行子网划分的细节			
子网	网络地址	广播地址	该子网可分配的地址
1	218.75.230.0	218.75.230.31	218.75.230.1 ~ 218.75.230.30
2	218.75.230.32	218.75.230.63	218.75.230.33 ~ 218.75.230.62
3	218.75.230.64	218.75.230.95	218.75.230.65 ~ 218.75.230.94
4	218.75.230.96	218.75.230.127	218.75.230.97 ~ 218.75.230.126
5	218.75.230.128	218.75.230.159	218.75.230.129 ~ 218.75.230.158
6	218.75.230.160	218.75.230.191	218.75.230.161 ~ 218.75.230.190
7	218.75.230.192	218.75.230.223	218.75.230.193 ~ 218.75.230.222
8	218.75.230.224	218.75.230.255	218.75.230.225 ~ 218.75.230.254

从子网1~8中任选5个分配给左图中的N1~N5。

练习-3

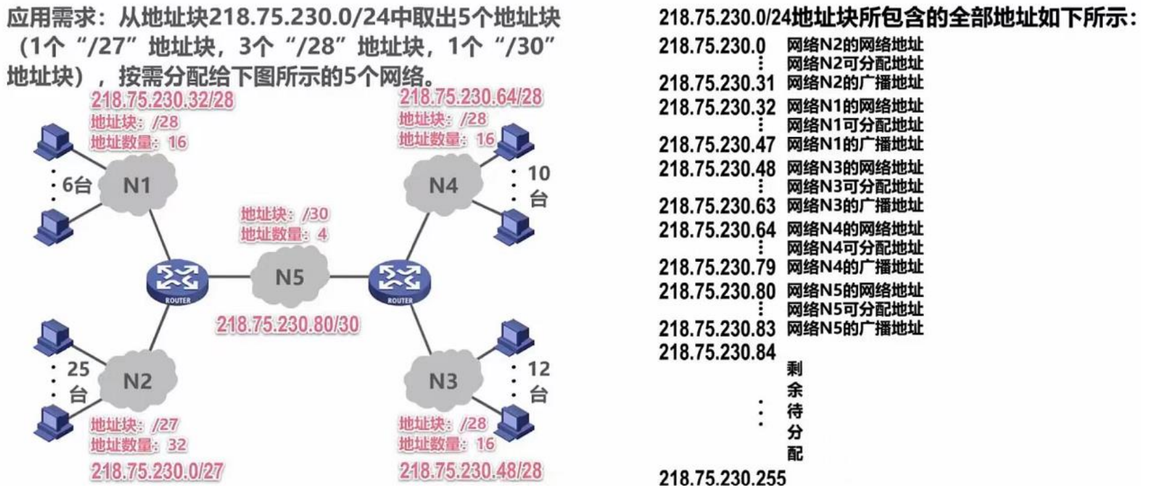
假设所应用的IP地址块是218.75.230.0/24，请分别使用固定长度和可变长度子网掩码为下图中的设备分配IP地址。

使用子网掩码255.255.255.224 对C类网络218.75.230.0进行子网划分的细节			
子网	网络地址	广播地址	该子网可分配的地址
1	218.75.230.0	218.75.230.31	218.75.230.1 ~ 218.75.230.30
2	218.75.230.32	218.75.230.63	218.75.230.33 ~ 218.75.230.62
3	218.75.230.64	218.75.230.95	218.75.230.65 ~ 218.75.230.94
4	218.75.230.96	218.75.230.127	218.75.230.97 ~ 218.75.230.126
5	218.75.230.128	218.75.230.159	218.75.230.129 ~ 218.75.230.158
6	218.75.230.160	218.75.230.191	218.75.230.161 ~ 218.75.230.190
7	218.75.230.192	218.75.230.223	218.75.230.193 ~ 218.75.230.222
8	218.75.230.224	218.75.230.255	218.75.230.225 ~ 218.75.230.254

从子网1~8中任选5个分配给左图中的N1~N5。

Exercises-3

Assuming that the applied IP address block is 218.75.230.0/24, please assign IP addresses to the devices in the figure below using fixed length and variable length subnet masks respectively.

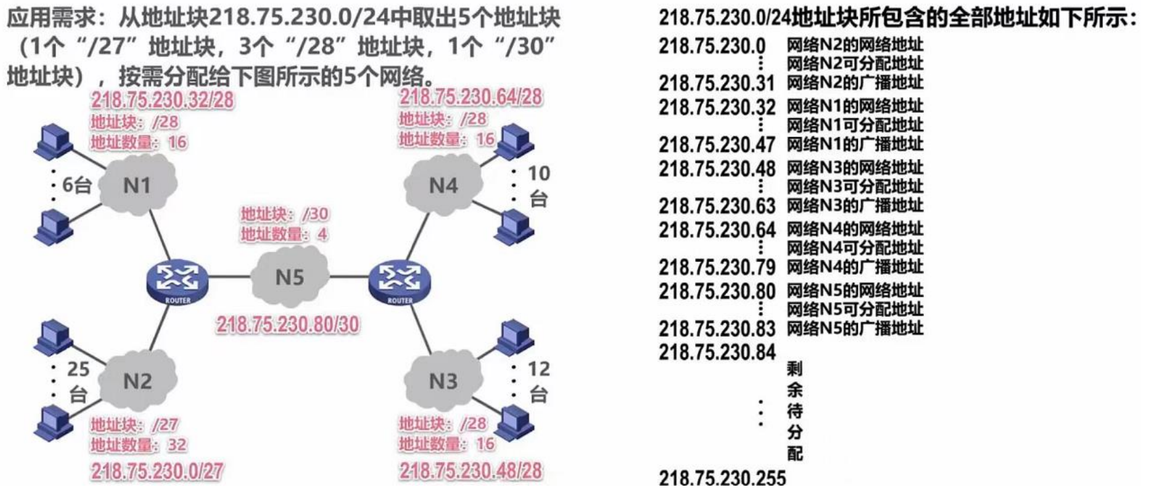


218.75.230.0/24地址块所包含的全部地址如下所示：

218.75.230.0	网络N2的网络地址
⋮	网络N2可分配地址
218.75.230.31	网络N2的广播地址
218.75.230.32	网络N1的网络地址
⋮	网络N1可分配地址
218.75.230.47	网络N1的广播地址
218.75.230.48	网络N3的网络地址
⋮	网络N3可分配地址
218.75.230.63	网络N3的广播地址
218.75.230.64	网络N4的网络地址
⋮	网络N4可分配地址
218.75.230.79	网络N4的广播地址
218.75.230.80	网络N5的网络地址
⋮	网络N5可分配地址
218.75.230.83	网络N5的广播地址
218.75.230.84	
⋮	剩余待分配
218.75.230.255	

习题-3

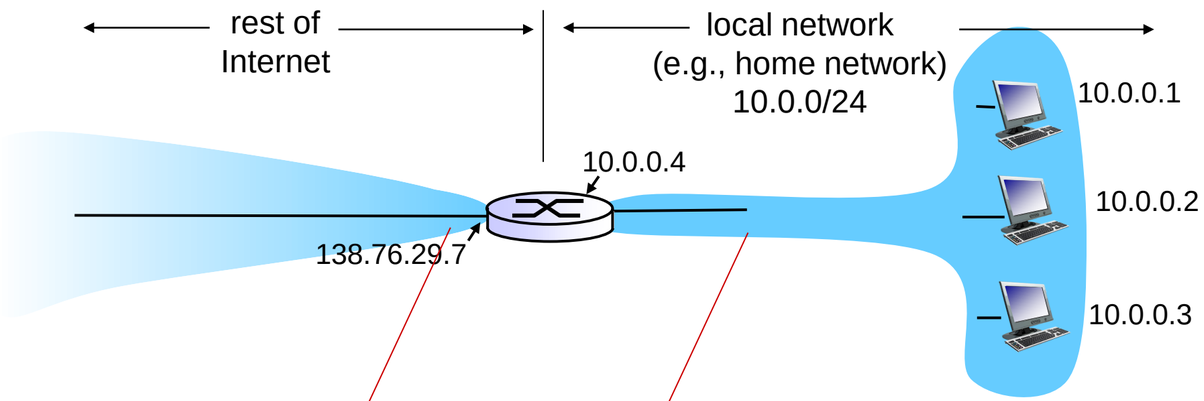
假设所应用的IP地址块是218.75.230.0/24，请使用固定长度和可变长度子网掩码分别给下图中的设备分配IP地址。



218.75.230.0/24地址块所包含的全部地址如下所示：

218.75.230.0	网络N2的网络地址
⋮	网络N2可分配地址
218.75.230.31	网络N2的广播地址
218.75.230.32	网络N1的网络地址
⋮	网络N1可分配地址
218.75.230.47	网络N1的广播地址
218.75.230.48	网络N3的网络地址
⋮	网络N3可分配地址
218.75.230.63	网络N3的广播地址
218.75.230.64	网络N4的网络地址
⋮	网络N4可分配地址
218.75.230.79	网络N4的广播地址
218.75.230.80	网络N5的网络地址
⋮	网络N5可分配地址
218.75.230.83	网络N5的广播地址
218.75.230.84	
⋮	剩余待分配
218.75.230.255	

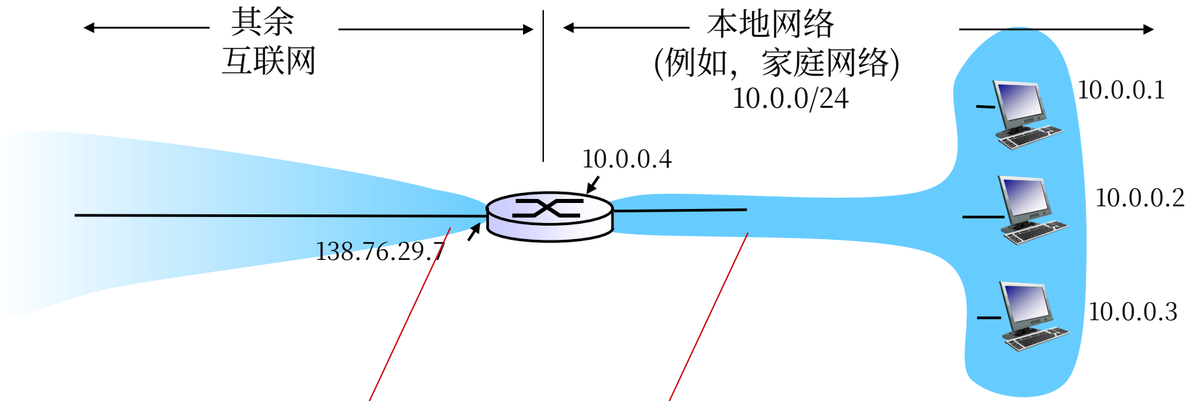
NAT: Network Address Translation



all datagrams *leaving* local network have *same* single source NAT IP address: 138.76.29.7,different source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT：网络地址转换



所有数据包 离开本地网络都有 相同的单一源网络地址转换IP地址：138.76.29.7，源端口号不同

源地址或目的地址在此网络中的数据报通常使用 10.0.0/24 作为源地址或目的地址

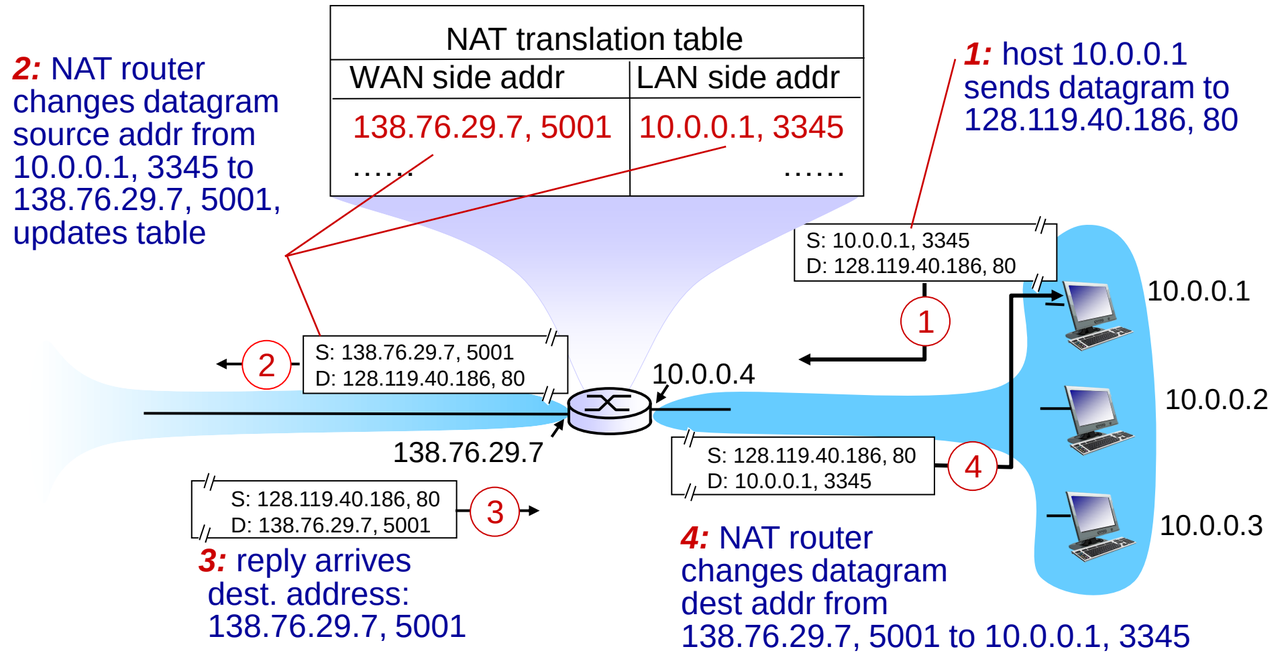
NAT: Network Address Translation

- Motivation: local network uses just one IP address as far as outside world is concerned
- Implementation: NAT router must:
  - Outgoing datagrams: replace (source IP address, port #) to (NAT IP address, new port #)
  - Remember (in NAT translation table) every translation pair
  - Incoming datagrams: replace (NAT IP address, new port #) with correspongding (source IP address, port #) stored in NAT table

NAT：网络地址转换

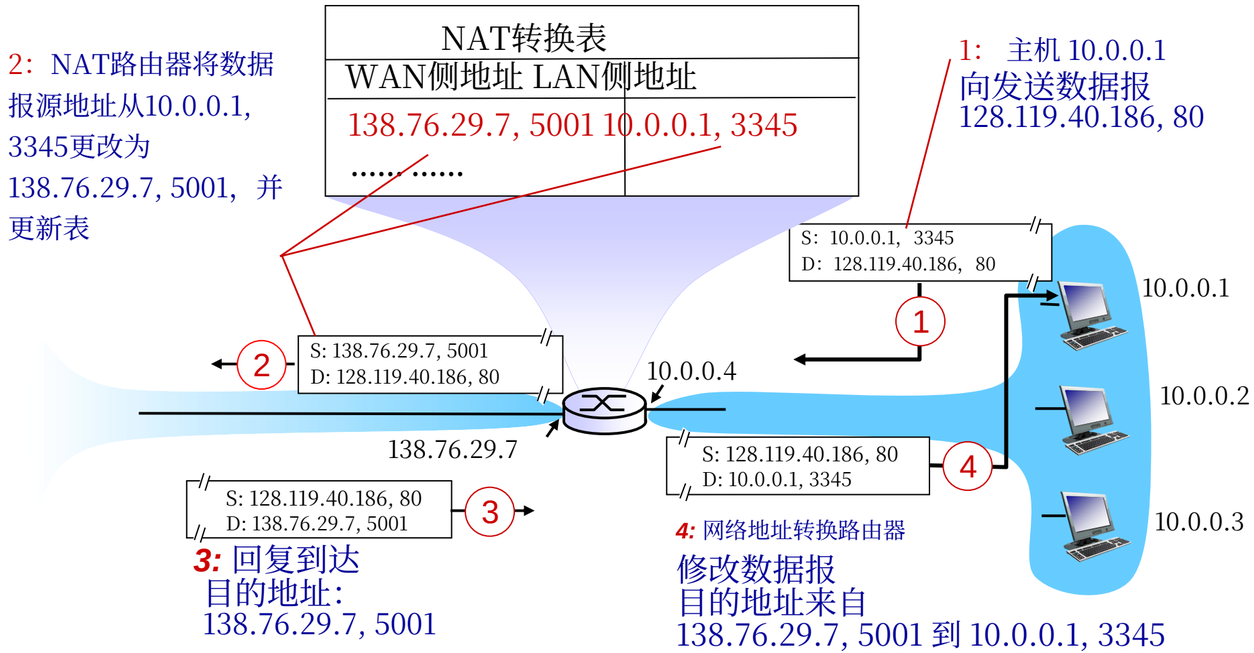
- 动机：就外部世界而言，本地网络只使用一个IP地址
- 实现：NAT路由器必须：
  - 出站数据报：替换(source IP地址, 端口号)为(NAT IP地址, 新端口号)
  - 记住(在NAT转换表中)每个转换对
  - 接收到的数据报：用相应的（源IP地址，端口#）替换（NAT IP地址，新端口#），这些信息存储在NAT表中

NAT: Network Address Translation



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

NAT：网络地址转换



* 查看在线互动练习获取更多示例：http://gaia.cs.umass.edu/kurose_ross/interactive/

NAT: Network Address Translation

- Outside node cannot initiate the communication

- Reserved addresses:
  - 10.0.0.0 – 10.255.255.255
  - 172.16.0.0 – 172.31.255.255
  - 192.168.0.0 – 192.168.255.255

NAT: 网络地址转换

- 外部节点不能发起通信

- 保留地址：
  - 10.0.0.0 – 10.255.255.255
  - 172.16.0.0 – 172.31.255.255
  - 192.168.0.0 – 192.168.255.255

LAN addresses and ARP

32-bit IP address:

- *network-layer* address
- used to get datagram to destination network (recall IP network definition)

LAN (or MAC or physical) address:

- used to get datagram from one interface to another physically-connected interface (same network)
- 48 bit MAC address (for most LANs) burned in the adapter ROM
- e.g.: 1A-2F-BB-76-09-AD

局域网地址和ARP

32位IP地址：

- 网络层地址
- 用于将数据报传输到目标网络（回顾IP网络定义）

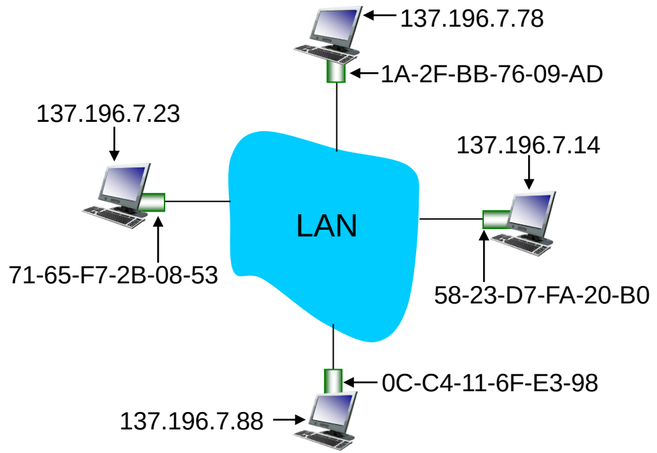
局域网（或MAC或物理）地址：

- 用于将数据报从一个接口传输到另一个物理连接的接口（同一网络）

- 48 用于大多数局域网的比特MAC地址（烧录在适配器ROM中）
- 例如：1A-2F-BB-76-09-AD

ARP: Address Resolution Protocol

Question: how to determine MAC address of B given B's IP address?



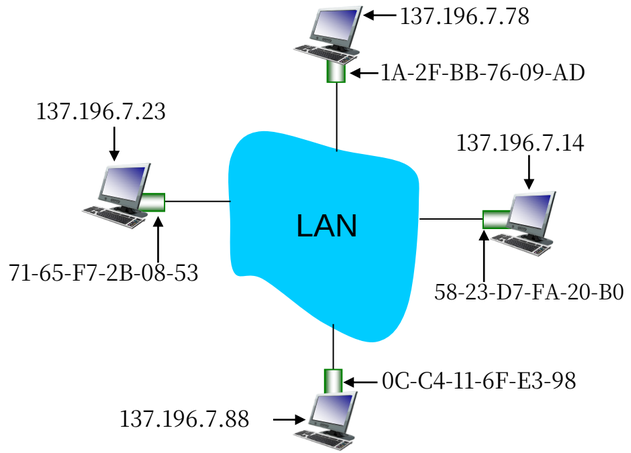
- Each IP node (Host, Router) on LAN has ARP module, table
- ARP Table: IP/MAC address mappings for some LAN nodes

  < IP address; MAC address; TTL>

  < ..... >
  - TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

ARP：地址解析协议

问题：如何根据B的IP地址确定<样式 id='1'>B的MAC地址</样式>?



- 局域网（LAN）上的每个IP节点（主机、路由器）都有ARP 模块，表- ARP表：一些局域网节点的IP/MAC地址映射< IP地址；MAC地址；生存时间>
  - 生存时间（Time To Live）：地址映射被遗忘的时间（通常为20分钟）

ARP protocol: same LAN

- A wants to send datagram to B
  - B's MAC address not in A's ARP table.
- A broadcasts ARP query packet, containing B's IP address
  - destination MAC address = FF-FF-FF-FF-FF-FF
  - all nodes on LAN receive ARP query
- B receives ARP packet, replies to A with its (B's) MAC address
  - frame sent to A's MAC address (unicast)
- A caches (saves) IP-to-MAC address pair in its ARP table until information becomes old (times out)
  - soft state: information that times out (goes away) unless refreshed
- ARP is "plug-and-play":
  - nodes create their ARP tables *without intervention from net administrator*

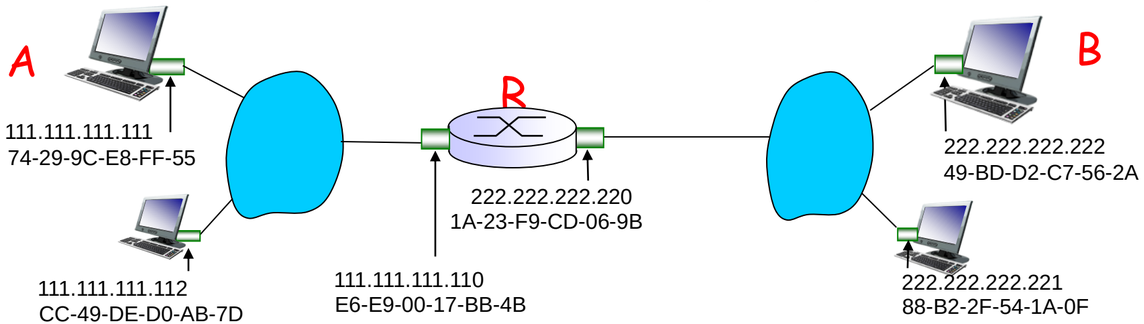
ARP协议：同一局域网

- A想要将数据报发送给B
  - B<样式 id='3'>的</样式>MAC地址不在A<样式 id='5'>的</样式>ARP表中。
- A 广播ARP查询包，其中包含B的IP地址
  - 目的MAC地址 = FF-FF-FF-FF-FF-FF
  - 局域网上的所有节点接收ARP查询
- B接收ARP数据包，向A回复其（B的）MAC地址
  - 发送到A'的MAC地址（单播）
- A在其ARP表中缓存（保存）IP地址到MAC地址对，直到信息过期（超时）
  - 软状态：除非刷新，否则会超时（消失）的信息
- ARP是"即插即用"：
  - 节点创建它们的ARP表无需网络管理员干预

Routing to another LAN

walkthrough: send datagram from A to B via R

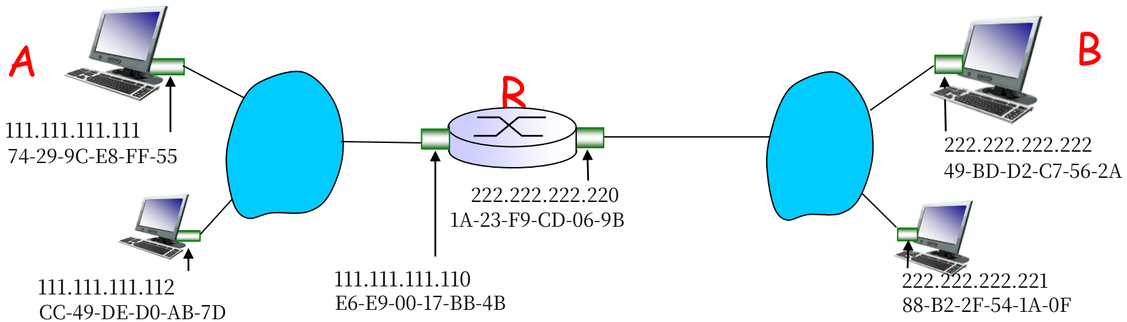
- focus on addressing – at IP (datagram) and MAC layer (frame)
- assume A knows B's IP address
- assume A knows IP address of first hop router, R (how?)
- assume A knows R's MAC address (how?)



路由到另一个局域网

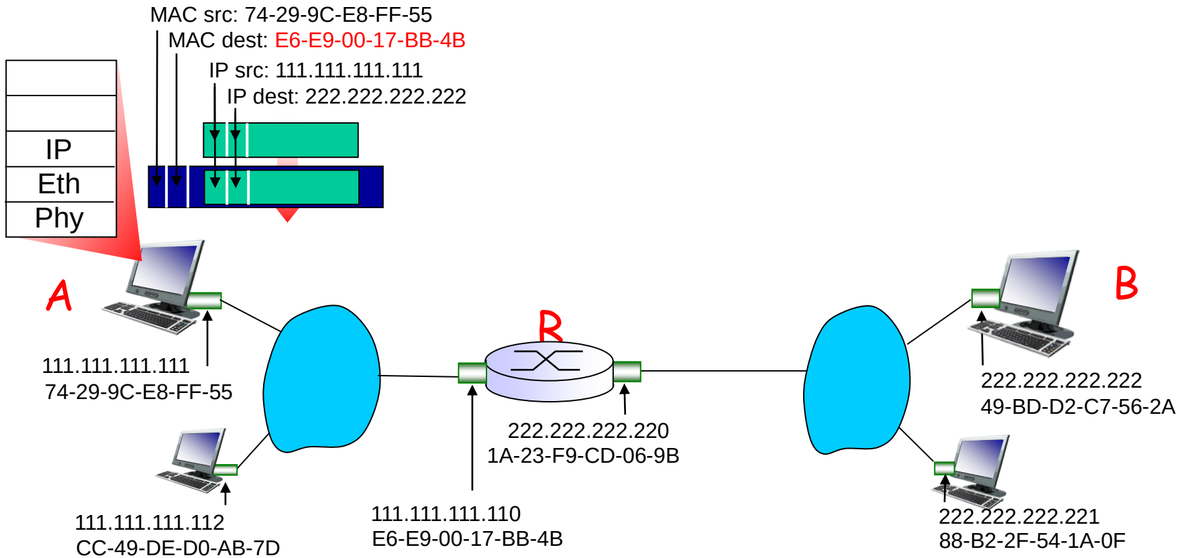
教程：从A通过R发送数据报到B

- 关注寻址 -IP（数据报）和MAC（帧）层
- 假设A知道B的IP地址
- 假设A知道第一跳路由器R的IP地址（如何？）
- 假设A知道R的MAC地址（如何做到？）



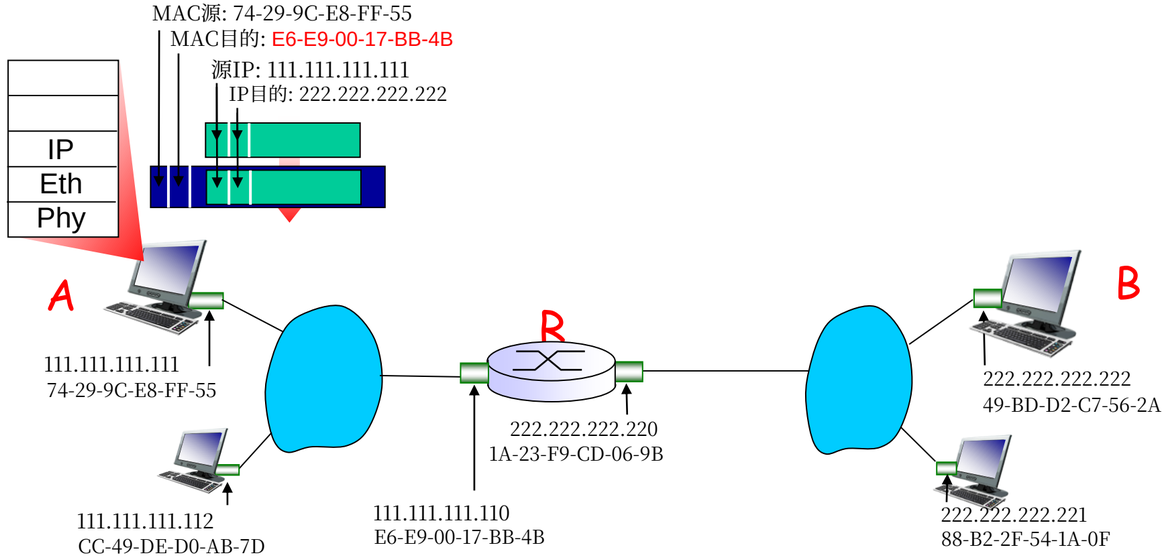
Routing to another LAN

- A creates IP datagram with IP source A, destination B
- A creates link-layer frame with R's MAC address as destination address, frame contains A-to-B IP datagram



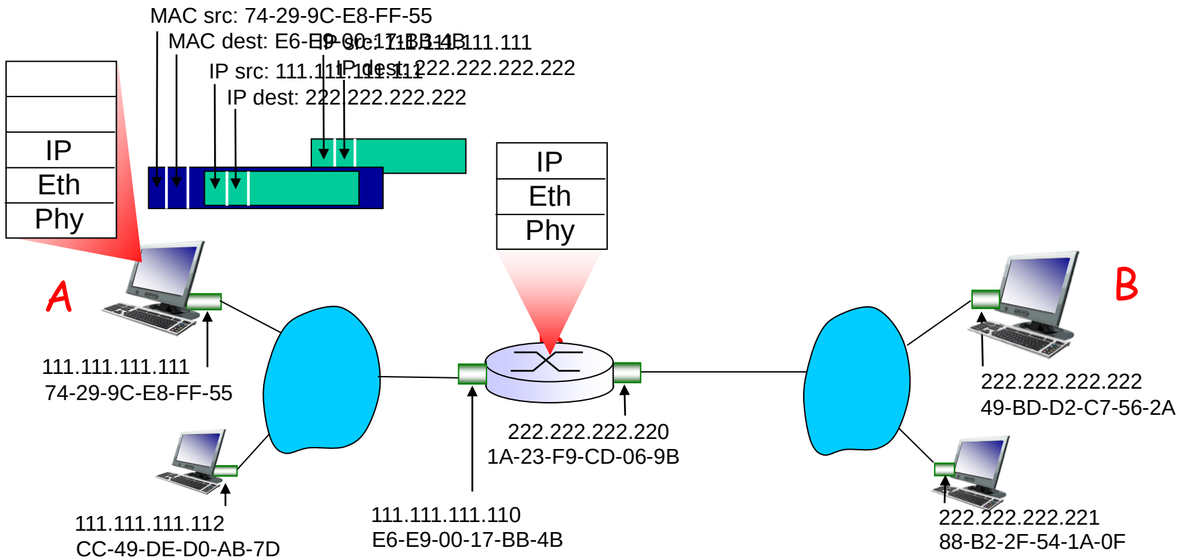
路由到另一个局域网

- A创建IP数据报，源IP为A，目的IP为B
- A 使用 R 的 MAC 地址作为目的地址创建链路层帧，帧中包含 A 到 B 的 IP 数据报



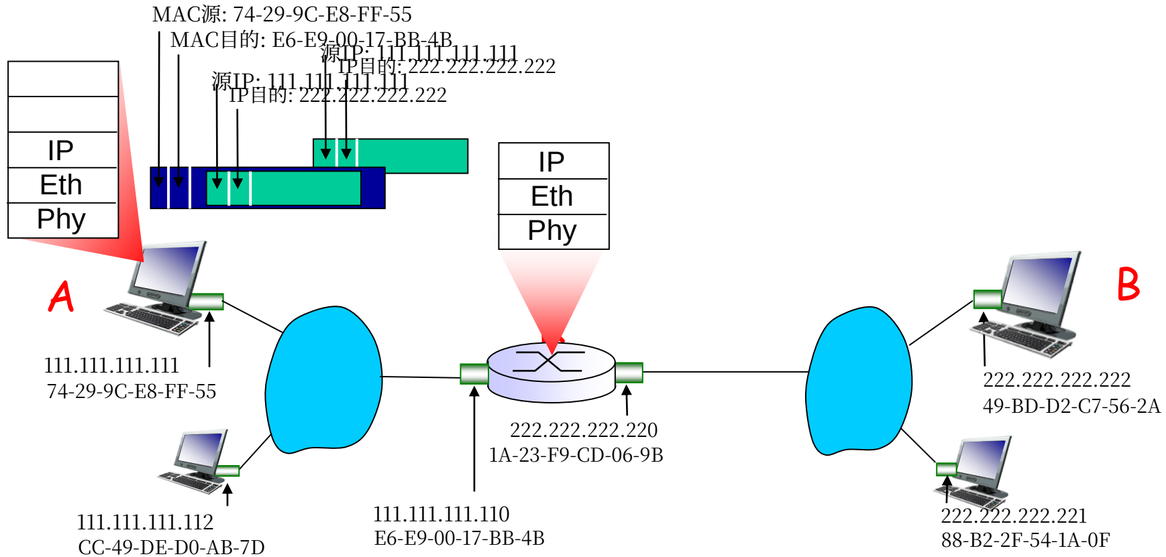
Routing to another LAN

- frame sent from A to R
- frame received at R, datagram removed, passed up to IP



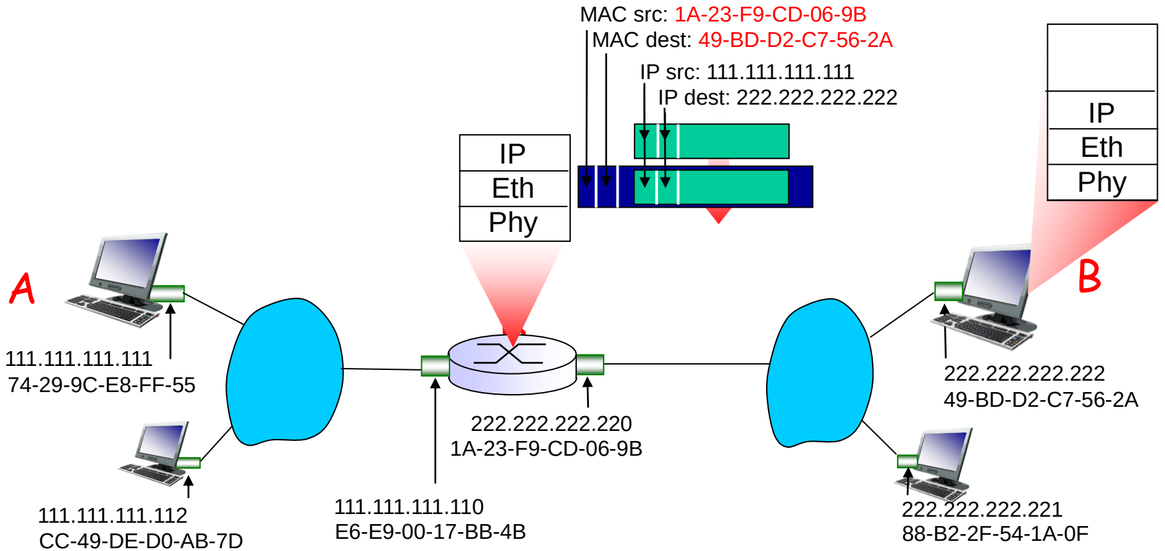
路由到另一个局域网

- 帧从 A 发送到 R
- 在R处接收到帧，数据报已移除，向上传递给IP



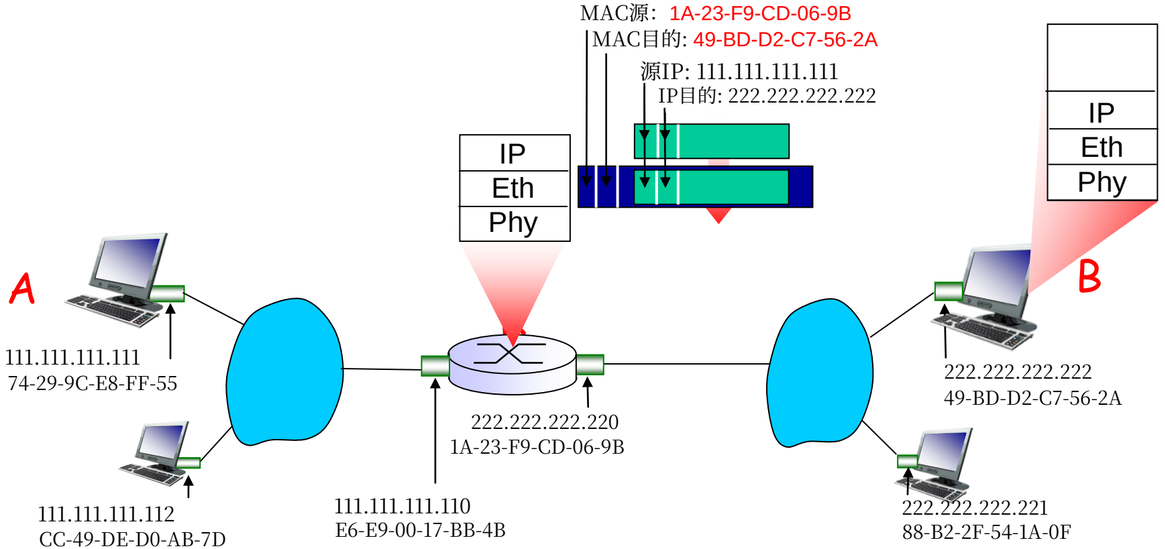
Routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as destination address, frame contains A-to-B IP datagram



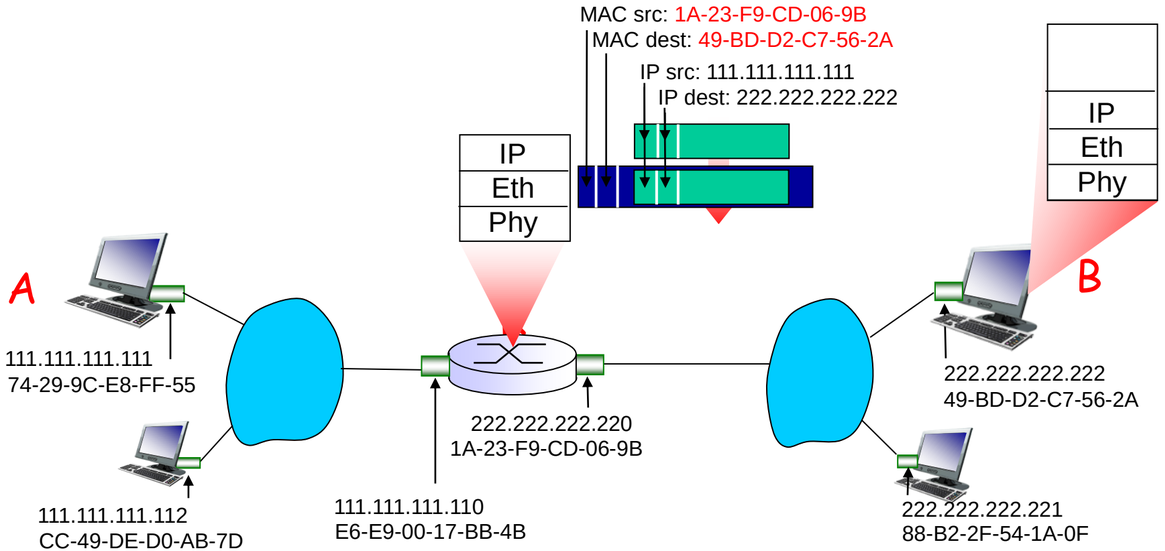
路由到另一个局域网

- R将数据报转发，IP源地址为A，目的地址为B
- R 使用 B 的 MAC 地址作为目的地址创建链路层帧，帧中包含从 A 到 B 的 IP 数据报



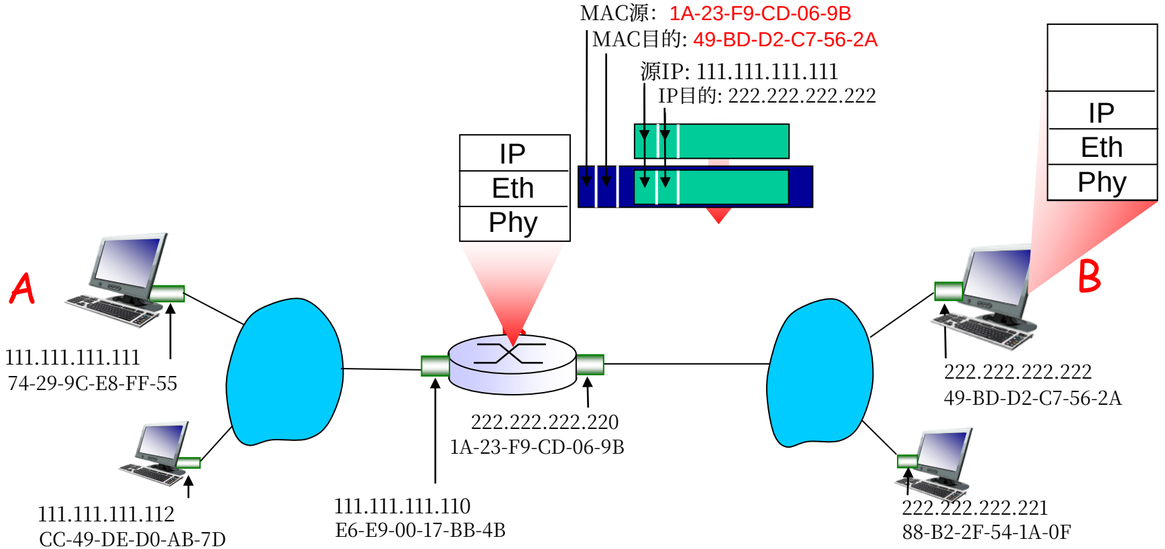
Routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as destination address, frame contains A-to-B IP datagram



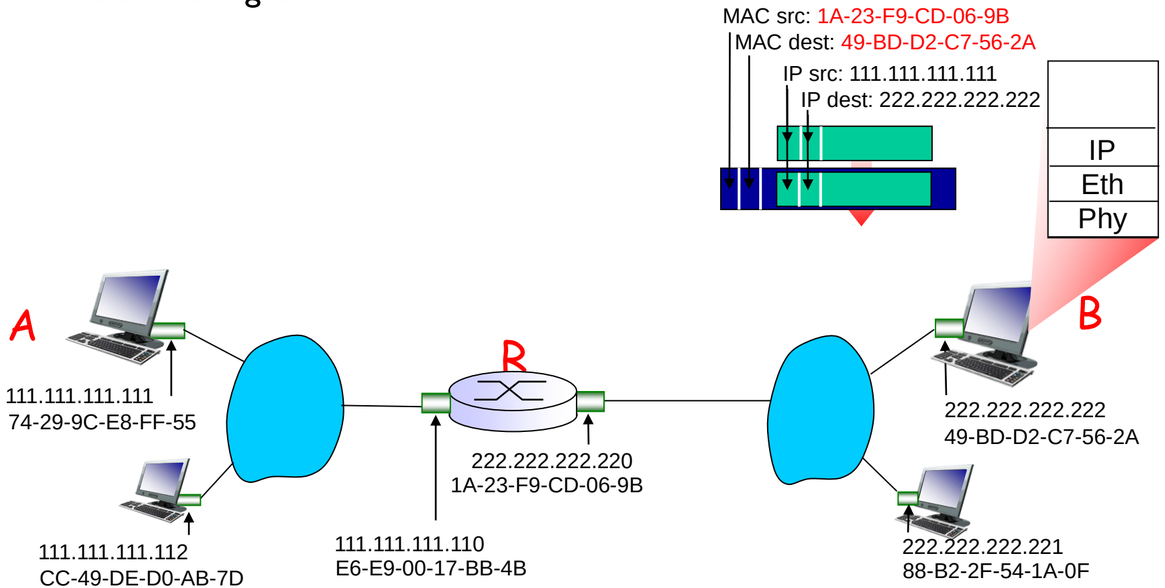
路由到另一个局域网

- R 转发源 IP 为 A、目的 IP 为 B 的数据报
- R 使用 B 的 MAC 地址作为目的地址创建链路层帧，帧中包含 A 到 B 的 IP 数据报



Routing to another LAN

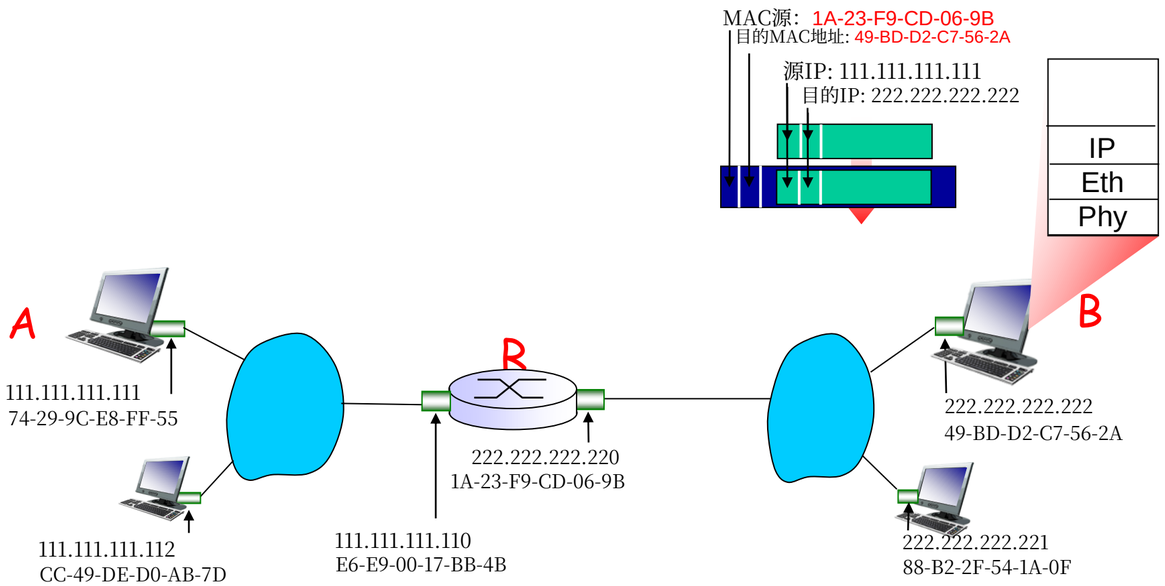
- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

路由到另一个局域网

- R 转发 IP 源地址为 A、目的地址为 B 的数据报
- R使用B的MAC地址作为目的MAC地址创建链路层帧，帧中包含从A到B的IP数据报



* 请访问在线交互式练习获取更多示例：
http://gaia.cs.umass.edu/kurose_ross/interactive/

Exercises-4

Figure 1 shows the network topology, and Figure 2 shows the hex-content of the first 80 bytes of an Ethernet frame of the host for a Web request.

(1) What is the IP address of the Web server? What is the MAC address of the default gateway of this host?

(2) When the IP packet encapsulated in this frame is forwarded through router R, which fields in the IP packet need to be modified?

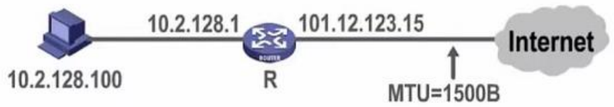


Fig. 1

00 21 27 21 51 ee 00 15	c5 c1 5e 28 08 00 45 00
01 ef 11 3b 40 00 80 06	ba 9d 0a 02 80 64 40 aa
62 20 04 ff 00 50 e0 e2	00 fa 7b f9 f8 05 50 18
fa f0 1a c4 00 00 47 45	54 20 2f 72 66 63 2e 68
74 6d 6c 20 48 54 54 50	2f 31 2e 31 0d 0a 41 63

Fig. 2

习题-4

图1显示了网络拓扑，图2显示了主机Web请求的以太网帧前80个字节的内容（十六进制）

(1) Web服务器的IP地址是什么？此主机的默认网关的MAC地址是什么？

(2) 当这个帧中封装的IP数据包通过路由器R转发时，IP数据包中哪些字段需要被修改？

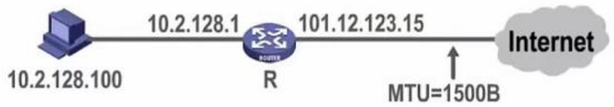


图1

00 21 27 21 51 ee 00 15	c5 c1 5e 28 08 00 45 00
01 ef 11 3b 40 00 80 06	ba 9d 0a 02 80 64 40 aa
62 20 04 ff 00 50 e0 e2	00 fa 7b f9 f8 05 50 18
fa f0 1a c4 00 00 47 45	54 20 2f 72 66 63 2e 68
74 6d 6c 20 48 54 54 50	2f 31 2e 31 0d 0a 41 63

图2

ICMP: Internet Control Message Protocol

- used by hosts, routers, gateways to communication network-level information
  - error reporting: unreachable host, network, port, protocol
  - echo request/reply (used by ping)
- network-layer "above" IP:
  - ICMP msgs carried in IP datagrams
- ICMP message: type, code plus first 8 bytes of IP datagram causing error

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

ICMP：互联网控制消息协议

- 被主机、路由器和网关用来通信网络层信息
  - 错误报告：无法到达的主机、网络、端口、协议
  - 回显请求/应答（用于ping工具）
- 网络层“位于”IP层之上：
  - ICMP消息承载在IP数据报中
- ICMP消息：类型、代码以及导致错误的IP数据报的前8个字节

类型代码描述 0 0 回显应答（ping） 3 0 目的. 网络不可达 3 1 目的主机不可达 3 2 目的协议不可达 3 3 目的端口不可达 3 6 目的网络未知 3 7 目的主机未知 4 0 源抑制（拥塞控制 - 未使用） 8 0 回显请求（ping） 9 0 路由发布 10 0 路由器发现 11 0 TTL 过期12 0 坏 IP 头部

ICMP:brief summary

- ICMP is the control sibling of IP
- ICMP is used by IP and uses IP as network layer protocol
- ICMP is used for ping, traceroute, and path MTU discovery
  - Ping: uses ICMP Echo request/reply messages
  - Path MTU discovery
    - Send a large IP datagram with"No fragment" bit set
    - Reduce size until success (No ICMP message received)

ICMP:简要概述

- ICMP 是 IP 的控制兄弟
- ICMP由IP使用，并将IP作为网络层协议
- ICMP用于ping、traceroute和路径MTU发现
  - Ping：使用ICMP回显请求/应答消息
  - 路径MTU发现
    - 发送带有“不分片”标志的大型IP数据报➢ 减小尺寸直至成功（未收到ICMP消息）

ping

```
C:\Documents and Settings\XXR>ping mail.sina.com.cn

Pinging mail.sina.com.cn [202.108.43.230] with 32 bytes of data:

Reply from 202.108.43.230: bytes=32 time=368ms TTL=242
Reply from 202.108.43.230: bytes=32 time=374ms TTL=242
Request timed out.
Reply from 202.108.43.230: bytes=32 time=374ms TTL=242

Ping statistics for 202.108.43.230:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 368ms, Maximum = 374ms, Average = 372ms
```

ping

C:\Documents and Settings\XXR>ping mail.sina.com.cn

Pinging mail.sina.com.cn [202.108.43.230] with 32 bytes of data:

Reply from 202.108.43.230: bytes=32 time=368ms TTL=242
Reply from 202.108.43.230: bytes=32 time=374ms TTL=242
Request timed out.
Reply from 202.108.43.230: bytes=32 time=374ms TTL=242

Ping statistics for 202.108.43.230:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 368ms, Maximum = 374ms, Average = 372ms

traceroute

```
C:\Documents and Settings\XXR>tracert mail.sina.com.cn

Tracing route to mail.sina.com.cn [202.108.43.230]
over a maximum of 30 hops:

  1    24 ms    24 ms    23 ms  222.95.172.1
  2    23 ms    24 ms    22 ms  221.231.204.129
  3    23 ms    22 ms    23 ms  221.231.206.9
  4    24 ms    23 ms    24 ms  202.97.27.37
  5    22 ms    23 ms    24 ms  202.97.41.226
  6    28 ms    28 ms    28 ms  202.97.35.25
  7    50 ms    50 ms    51 ms  202.97.36.86
  8   308 ms   311 ms   310 ms  219.158.32.1
  9   307 ms   305 ms   305 ms  219.158.13.17
 10   164 ms   164 ms   165 ms  202.96.12.154
 11   322 ms   320 ms  2988 ms  61.135.148.50
 12   321 ms   322 ms   320 ms  freemail43-230.sina.com [202.108.43.230]

Trace complete.
```


traceroute

```
C:\Documents and Settings\XXR>tracert mail.sina.com.cn

Tracing route to mail.sina.com.cn [202.108.43.230]
over a maximum of 30 hops:

  1    24 ms    24 ms    23 ms  222.95.172.1
  2    23 ms    24 ms    22 ms  221.231.204.129
  3    23 ms    22 ms    23 ms  221.231.206.9
  4    24 ms    23 ms    24 ms  202.97.27.37
  5    22 ms    23 ms    24 ms  202.97.41.226
  6    28 ms    28 ms    28 ms  202.97.35.25
  7    50 ms    50 ms    51 ms  202.97.36.86
  8   308 ms   311 ms   310 ms  219.158.32.1
  9   307 ms   305 ms   305 ms  219.158.13.17
 10   164 ms   164 ms   165 ms  202.96.12.154
 11   322 ms   320 ms  2988 ms  61.135.148.50
 12   321 ms   322 ms   320 ms  freemail43-230.sina.com [202.108.43.230]

Trace complete.
```

Traceroute and ICMP

- Source sends series of UDP segments to dest.
  - First has TTL=1
  - Second has TTL=2, etc.
  - Unlikely port number
- When nth datagram arrives to nth router:
  - Router discards datagram
  - And sends to source an ICMP message (type 11, code 0)
  - Message includes name of router & IP address
- When ICMP message arrives, source calculates RTT
- Traceroute does this 3 times
- Stopping criterion
- UDP segment eventually arrives at destination host
- Destination returns ICMP "host unreachable" packet (type 3, code 3)
- When source gets this ICMP, stops

traceroute 和 ICMP

- 源节点向目的节点发送一系列 UDP 数据报。
  - 第一个数据报的 TTL 为 =1
  - 第二个数据报的 TTL 为 =2，以此类推 不常见的端口号
- 当第 n 个数据报到达第 n 个路由器时：
  - 路由器丢弃数据报 并向源节点发送 ICMP 消息（类型 11，代码 0） 消息中包含路由器的名称和 IP 地址
- 当ICMP消息到达时，源节点计算RTT Traceroute会进行3次 停止条件 UDP数据段最终到达目的主机 目的主机返回ICMP<style id='8'> "主机不可达"数据包(类型3，代码3) 当源节点收到这个ICMP时，停止

IPv6

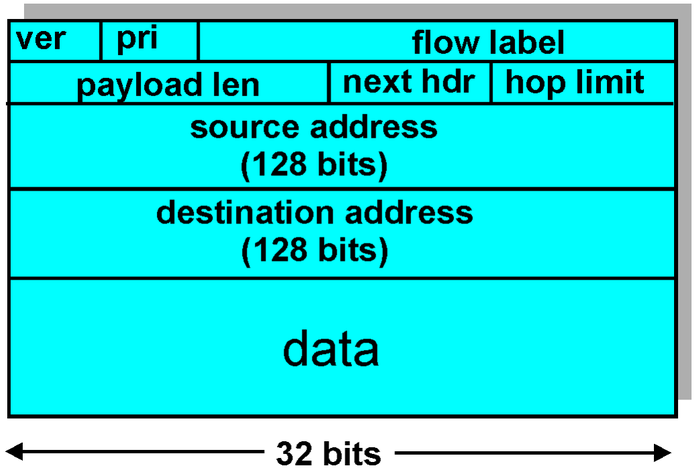
- Initial motivation: 32-bit address space completely allocated by 2008.
- Additional motivation:
  - header format helps speed processing/forwarding
  - header changes to facilitate QoS
  - new "anycast" address: route to "best" of several replicated servers
- IPv6 datagram format:
  - fixed-length 40 byte header
  - no fragmentation allowed

IPv6

- 初始动机：32位地址空间在2008年已完全分配
- 额外的动机：
  - 头部格式有助于加速处理/转发 头部变更以促进服务质量 新的"任播"地址：路由到多个复制的服务器中的"最佳"一个
- IPv6数据报格式：
  - 固定长度40字节的头部 不允许分片

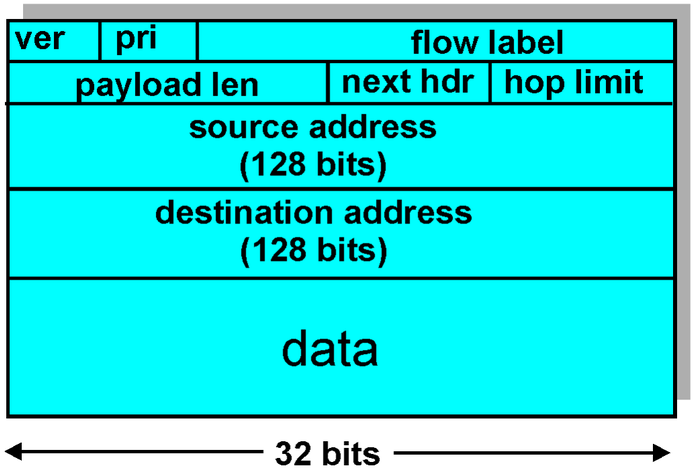
IPv6 Header (Cont)

Priority: identify priority among datagrams in flow
Flow Label: identify datagrams in same "flow."
(concept of"flow" not well defined).
Next header: identify upper layer protocol for data



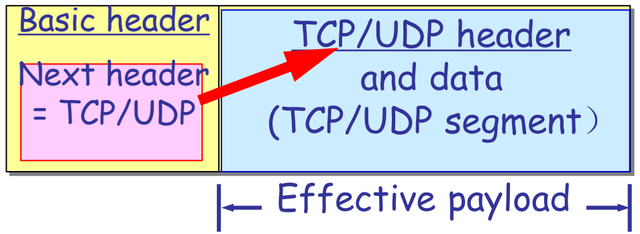
IPv6 头部 (续)

优先级：在流中识别数据包的优先级流标签：识别
y 同一“流”中的数据包。
（“流”的概念尚未明确定义）。
下一个头部： 识别用于数据的上层协议

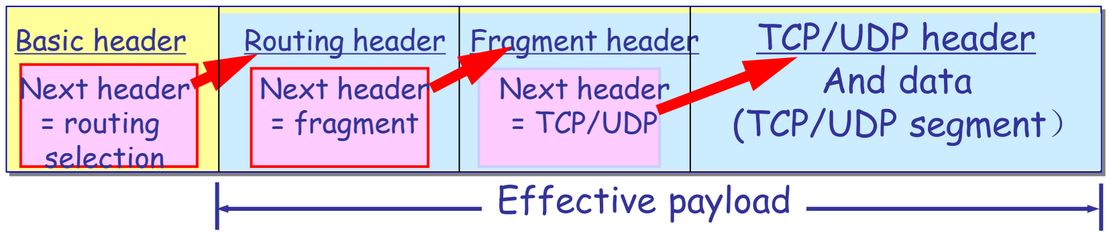


Next header

Without extension header

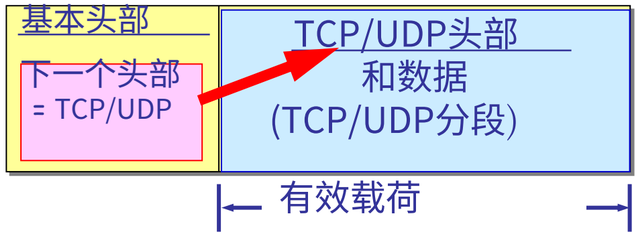


With extension header

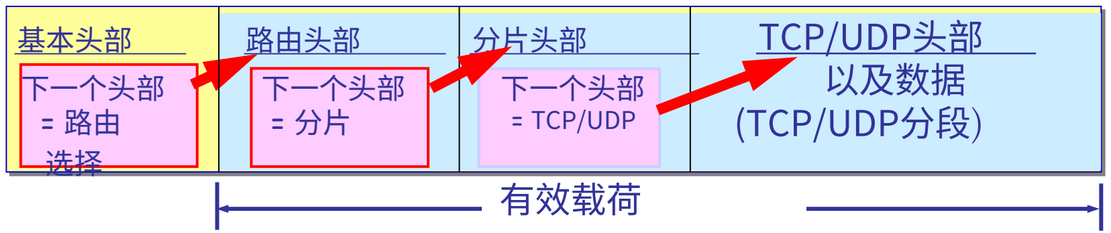


下一个头部

无扩展头部



有扩展头部



Other Changes from IPv4

- *Checksum:* removed entirely to reduce processing time at each hop
- *Options:* allowed, but outside of header, indicated by "Next Header" field
- *ICMPv6:* new version of ICMP
  - additional message types, e.g. "Packet Too Big"
  - multicast group management functions

IPv4 的其他变化

- 校验和： 完全移除以减少每个跳点的处理时间
- 选项： 允许，但位于头部之外，由“下一个头部”字段指示
- ICMPv6： ICMP 的新版本
  - 额外的消息类型，例如“数据包过大” ○ 多播组管理功能

IPv6 address

- Three types: unicast, multicast, anycast
- Colon hexadecimal notation:

68E6:8C64:FFFF:FFFF:0:1180:960A:FFFF

- Zero compression:

FF05:0:0:0:0:0:0:B3 == FF05::B3 ;

0:0:0:0:0:0:128.10.2.1 == ::128.10.2.1

12AB:0000:0000:CD30:0000:0000:0000:0000/60

== 12AB::CD30:0:0:0:0/60

== 12AB:0:0:CD30::/60

IPv6地址

- 三种类型：单播、多播、任播
- 十六进制冒号表示法：

68E6:8C64:FFFF:FFFF:0:1180:960A:FFFF

- 零压缩: FF05:0:0:0:0:0:0:B3 == FF05::B3 ;

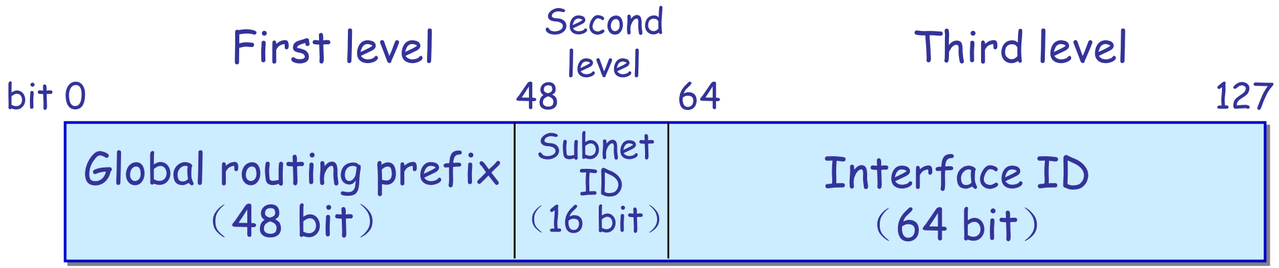
0:0:0:0:0:0:128.10.2.1 == ::128.10.2.1

12AB:0000:0000:CD30:0000:0000:0000:0000/60

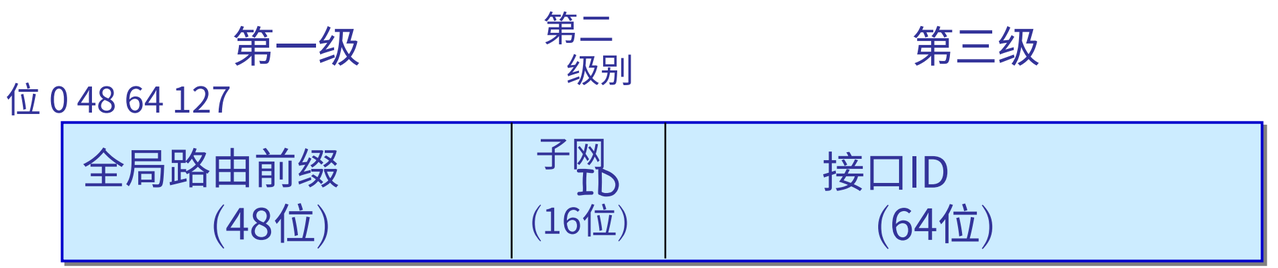
== 12AB::CD30:0:0:0:0/60

== 12AB:0:0:CD30::/60

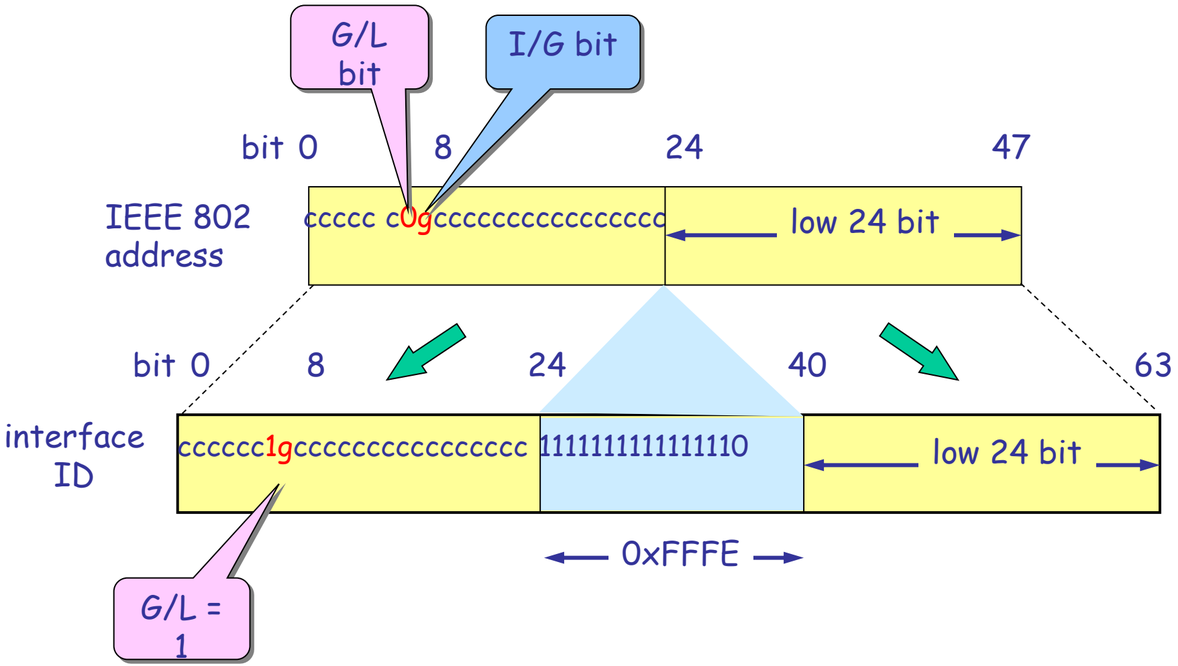
Unicast address



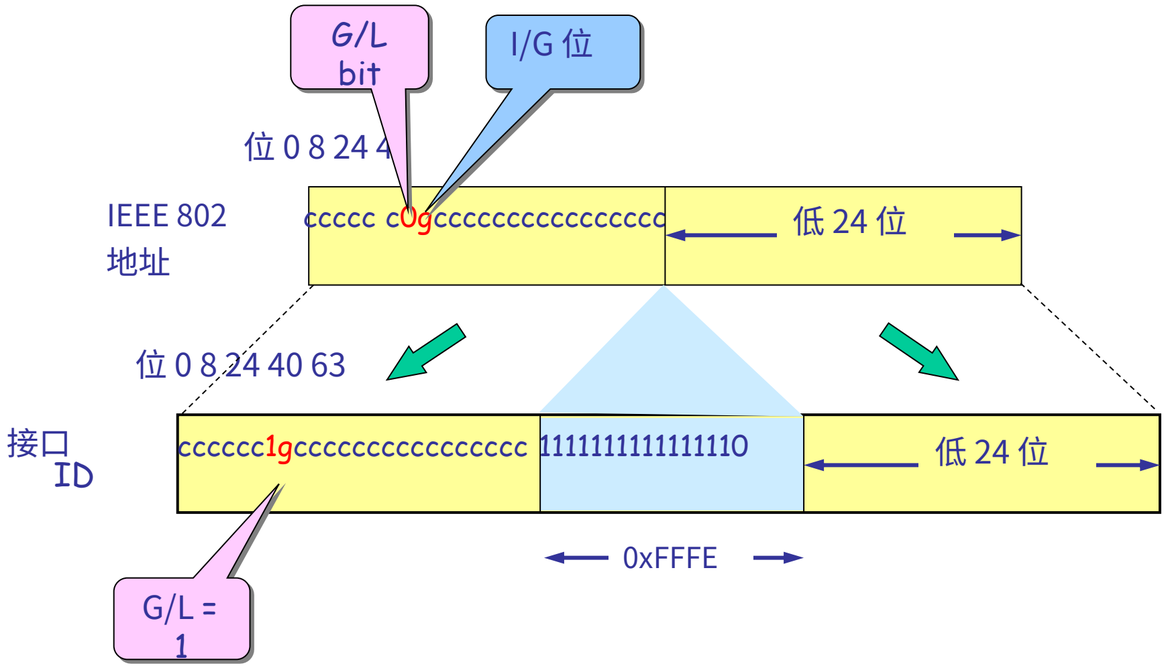
单播地址



EUI-64


G/L bit
I/G bit
bit 0
8
24
47
IEEE 802 address
cccccc c0gccccccccccccccc
low 24 bit
bit 0
8
24
40
63
interface ID
cccccc1gccccccccccccccc
1111111111111110
low 24 bit
0xFFFE
G/L = 1

EUI-64


G/L bit
I/G 位
位 0 8 24 4
IEEE 802 地址
cccccc c0gccccccccccccccc
低 24 位
位 0 8 24 40 63
接口 ID
cccccc1gccccccccccccccc
1111111111111110
低 24 位
0xFFFE
G/L = 1

Example:
host MAC address: 00:0C:85:AB:50:01
insert FFFE:
00:0C:85:FF:FE:AB:50:01
set G/L bit:
02:0C:85:FF:FE:AB:50:01
host EUI-64 is:
020C:85FF:FEAB:5001
router advertise the 64bit network prefix
Host unicast address: prefix+EUI-64

示例：主机MAC地址：
00:0C:85:AB:50:01 插入 FFFE：
00:0C:85:FF:FE:AB:50:01 设置 G/L 位：
02:0C:85:FF:FE:AB:50:01 主机 EUI-64 是：
020C:85FF:FEAB:5001 路由器发布 64位网络前缀 主机单播地址：前缀+EUI-64

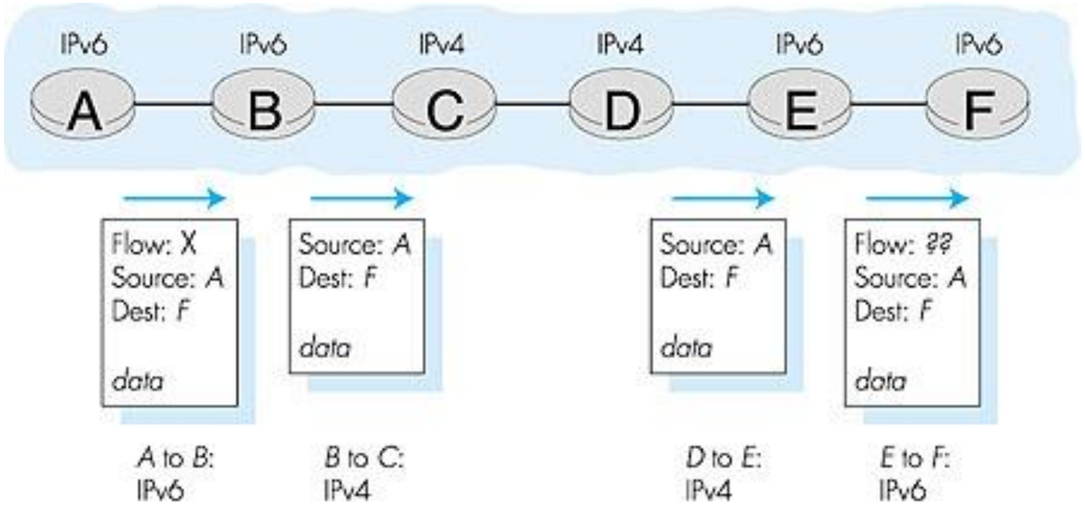
Transition From IPv4 To IPv6

- Not all routers can be upgraded simultaneous
  - no "flag days"
  - How will the network operatewith mixed IPv4 and IPv6 routers?
- Two proposed approaches:
  - *Dual Stack*: some routers with dual stack (v6, v4) can "translate" between formats
  - *Tunneling*: IPv6 carried as payload n IPv4 datagram among IPv4 routers

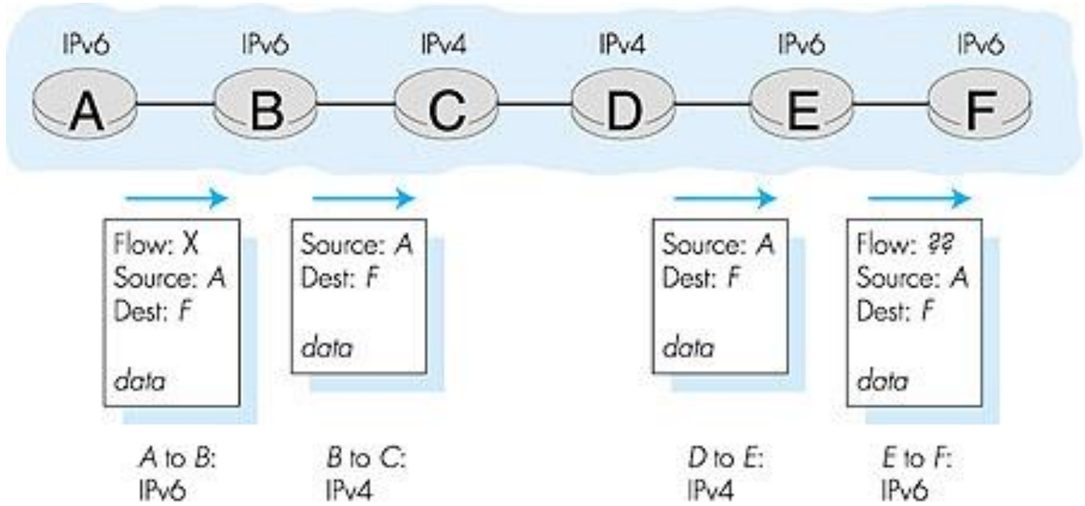
从 IPv4 过渡到 IPv6

- 并非所有路由器都能同时升级
  - 没有“旗帜日”
  - 网络将如何运行在混合 IPv4 和 IPv6 路由器的情况下?
- 两种提议的方法：
  - 双栈：一些具有双栈（v6, v4）的路由器可以“翻译”格式之间的差异
  - 隧道技术：IPv6 作为有效载荷在 IPv4 数据报中通过 IPv4 路由器传输

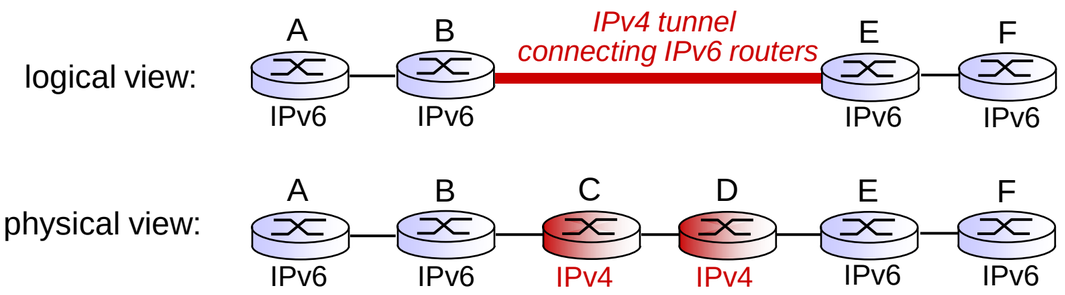
Dual Stack Approach



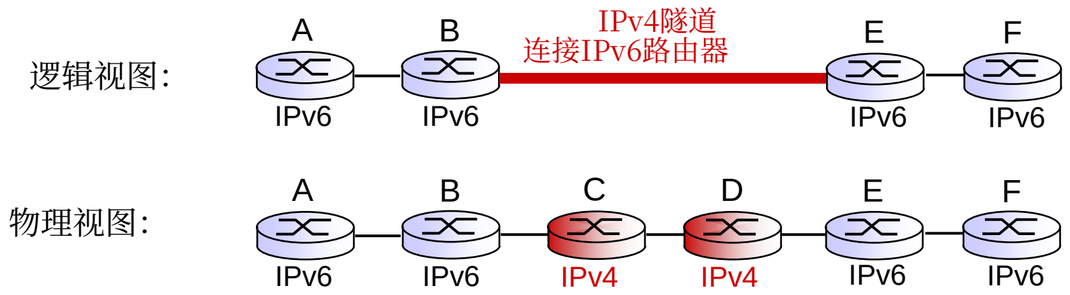
双栈方法



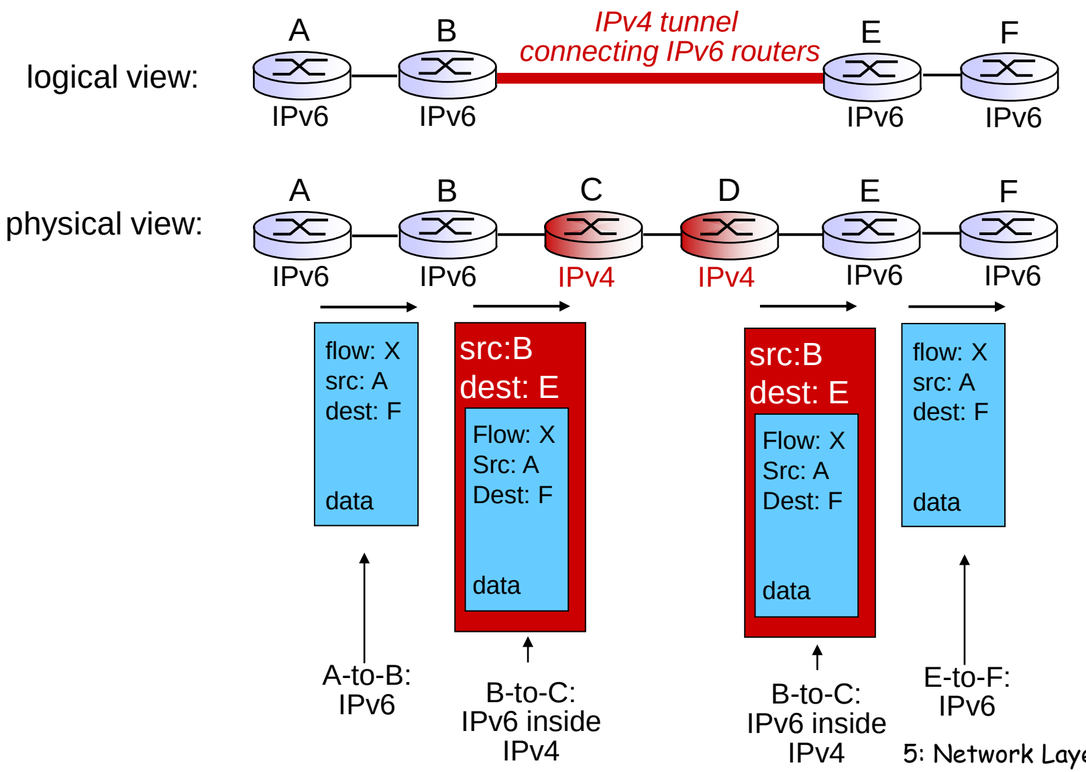
Tunneling



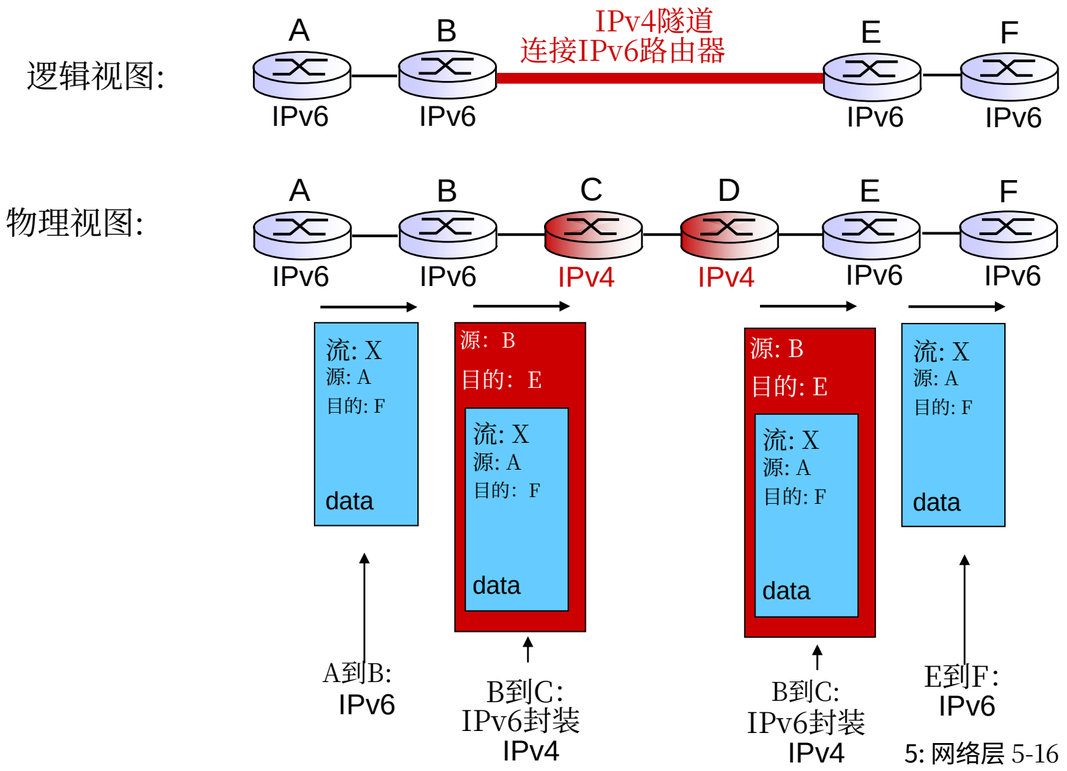
隧道技术



Tunneling



隧道技术



Summary

- Virtual circuit and datagram networks
- Routing algorithms:
  - Dijkstra's algorithm
  - Broadcast routing
  - Link state
  - Distance vector ("count to infinity" problem)
- Routing in the Internet (RIP, OSPF, BGP)
- IP: Internet protocol
  - IPv4 Datagram format
  - IPv4 addressing
  - IP fragment
  - NAT
  - ARP
  - ICMP
  - IPv6
  - From IPv4 to IPv6

摘要

- 虚拟电路和数据报网络
- 路由算法：
  - 迪杰斯特拉算法 广播路由 链路状态 距离向量("计数到无穷"问题)
- 互联网中的路由（RIP、OSPF、BGP）
- IP：互联网协议
  - IPv4数据报格式 IPv4地址 IP分片 网络地址转换 ARP ICMP IPv6 从IPv4到IPv6